



Meeting Booking System

Complete handbook

Employees

Booking, recurrence, approval and check-in

Front Desk / Administrators

Rooms, people, policy, reports and audit

IT / server operators

Deployment, logs, troubleshooting and recovery

Office site: <http://mbs.wdn.com.np>

Oracle Linux 9.8 | PostgreSQL | Docker | Branch mbs-prod

Documentation updated 8 October 2026, including per-room approval. See Verification evidence for tested behavior and remaining deployment checks.

Transgate Tech | Binayak Bhandari

Contents

Clickable chapter/section entries and PDF bookmarks are provided. Copy terminal commands from the linked Markdown guides; long lines wrap here for print. Initial deployment and routine operation use the HTTP profile. HTTPS is a separate future option.

1. Start here	6
Reading map	6
Current office installation	7
Rules and lifecycle at a glance	7
Where to go in the application	8
Quick operator commands (Linux)	9
Support boundaries and remaining operational decisions	9
2. Features and use cases	11
Contents	11
Branded web interface	11
Accounts, sessions and permissions	11
Page and feature directory	13
Room discovery and availability	16
Booking fields and validation	17
Approval and booking lifecycle	20
Recurring meetings	21
Check-in and automatic release	22
Email notifications	22
Staff administration	23
Reports and audit	25
Use-case catalogue	26
Implemented scope and limitations	28
3. Employee guide	29
Quick start	29
Navigation	29
1. Sign in using company email	29
2. Set your profile	30
3. Choose a room	30
4. Read the calendar	31
5. Request a single internal meeting	31
6. Request an external or mixed meeting	33
7. Create a recurring series	33

8. Follow approval and status	34
9. Edit or cancel	35
10. Check in at meeting start	35
11. Notifications and support	36
4. Staff guide	37
Staff navigation	37
1. First access and staff sign-in	37
2. Daily operating routine	38
3. Review a Pending request	39
4. Search, edit, cancel and check in company bookings	40
5. Book on behalf and manage exceptions	41
6. Add, edit or deactivate rooms	42
7. Block or release unavailable periods	43
8. Change booking rules	43
9. Manage company holidays	44
10. Manage people and access	44
11. Generate reports and export Excel	45
12. Use audit history	46
13. Email issues and escalation	46
14. Handover and retention	47
5. Office HTTP installation	48
Before starting	48
1. Get the HTTP deployment files	48
2. Configure the domain, binding, and SMTP	49
3. Build, migrate, and start	49
4. Verify mail and create staff access	50
5. Verify from an office laptop	50
Backups and updates	51
Troubleshooting and monitoring	52
6. Operations and recovery	53
Contents	53
1. Installation identity and architecture	53
2. Open an operator terminal	55
3. Configuration reference and safe checks	56
4. Starting, stopping, restarting, and updating	58
5. Staff access and password recovery	60
6. Service health, logs, and read-only database checks	61

7. Worker and notification operations	63
8. Backups, scheduling, and monitoring	64
9. Restore drill into a separate database	68
10. Approved live recovery	70
11. Daily, weekly, and release checklists	71
12. Incident handover and operational limits	72
13. Windows laptop differences	73
7. Troubleshooting and logs	74
1. First response	74
2. Logs and support evidence	75
3. Health and worker problems	76
4. Network, DNS and proxy	77
5. Session check and CSRF 403	78
6. Configuration and stale containers	79
7. Email sign-in and SMTP	79
8. Booking availability and business rules	82
9. Staff access and room editing	84
10. Application errors and static assets	85
11. Database, disk and Docker	85
12. Laptop problems (PowerShell)	86
13. Reports, audit and verification	87
8. Architecture	88
Purpose and launch scope	88
Architecture	88
Main user journeys	91
Current installation and remaining decisions	91
Acceptance checks before rollout	91
9. Database design	92
ERD	92
Photo files and database boundaries	92
Tables and keys	92
Reservation state and validation	94
Indexes and deletion	94
Migration and verification strategy	94
Approval migration	95
10. Laptop setup and development	96

Complete documentation	96
Features	96
Stack and requirements	97
Local setup	97
Environment and email	98
Verification and builds	99
Approval and booking lifecycle	99
Operations and limits	100

11. Verification evidence 101

Delivery and scope	101
Approval workflow	101
Reproduced failures and fixes	101
Security and database	102
Automated gates	103
Actual browser checks	104
Earlier isolated HTTPS production deployment rehearsal	104
Selected HTTP production deployment rehearsal	105
Native-browser CSRF correction	105
Deployment handover	106
Remaining verification	106
Documentation-only release verification - 7 October 2026	106
Booking wizard and room photos - 8 October 2026	107
Per-room approval - 8 October 2026	108

1. Start here

Source: docs/index.md

Transgate Tech | Binayak Bhandari Reviewed: 8 October 2026. **Application branch:** mbs-prod.

This is the documentation entry point for the existing Meeting Booking System: employee tasks, Front Desk/Administrator workflows, feature definitions, business examples, server operation, troubleshooting and recovery. It describes implemented behavior, including approval, rather than only the original requirements. The owner's later decisions take precedence over the questionnaire.

Reading map

Document	Audience	What it contains
Employee guide	Employees and organizers	Sign-in, rooms/calendar, three-step internal/external/mixed requests, individual/recurring meetings, edits, cancellation, check-in, profile and common questions.
Staff guide	Front Desk and Administrators	Password/email-code sign-in, dashboard, approval/rejection, rooms/photos, closures, on-behalf bookings, overrides, people, policy/holidays, reports and audit.
Feature catalogue and use cases	Product owner, staff, testers and developers	Permissions, rules, lifecycle, notifications, detailed success/failure examples and supported limits.
HTTP installation	Linux operator	Selected Oracle Linux private-network installation, domain/IP/SMTP setup, build, migrations, initial staff and pilot.
Operations runbook	Linux operator and IT	Configuration, background operation, updates, safe SQL/health checks, email recovery, backup scheduling, paired database/photo recovery, isolated restore and controlled live recovery.
Troubleshooting and logs	Support and IT	Symptoms, exact diagnostic commands, safe log capture, CSRF, SMTP, Docker, DNS, worker, database, booking/report issues and laptop differences.
Architecture	Technical owner/developers	Components, transaction/occupancy design, authentication, privacy, worker, deployment and limitations.
Database design	Developers and database operators	Tables, relationships, constraints, statuses, indexes, runtime permissions and migration/rollback behavior.
Verification report	Owner and reviewers	Historical/current test results, browser checks and isolated deployment/restore evidence; office pilot remains separate.

Document	Audience	What it contains
Alternative HTTPS installation	IT planning a future HTTPS change	Certificate-based profile, trust and deployment procedures. This is not the selected HTTP installation.
Repository README	Laptop developers and new operators	Local Docker setup, configuration overview, builds and test commands.

Read employees/staff first for everyday use. For a server incident, begin with troubleshooting's first-response commands. For a planned update/restore, read the operations procedure completely before executing it. Markdown command blocks are the authoritative copyable commands; the PDF is a printable companion.

Current office installation

Item	Selected configuration
Application	Meeting Booking System (MBS), Transgate branding, Times New Roman/system fallback
Site	http://mbs.wdn.com.np on the office network/VPN
Server	Oracle Linux 9.8, x86_64; office IP 192.168.50.222
Repository/branch	BINAYAK016/MeetingRoomSystemWDN , mbs-prod
Server checkout	/opt/mbs/MeetingRoomSystemWDN
Production Compose	compose.prod.http.yaml
Laptop Compose	compose.yaml; http://127.0.0.1:8000/
Employee domains	wdn.com.np, transgate.com.np
SMTP	maildc01.wdn.com.np:25, sender mbs@wdn.com.np, no relay username/password, TLS/SSL false
Timezone	Asia/Kathmandu for screens/rules; timezone-aware database instants
Staff role	Front Desk and Administrators have identical powers; individual accounts

HTTP intentionally requires no certificate/private CA; browsers show **Not secure**. The selected profile still enforces production restrictions, CSRF, staff password plus email code, ownership and database conflict constraints. Keep access restricted to the approved office network/VPN. SMTP settings describe the previously accepted relay; new-server authorization and actual mailbox delivery must be checked on that installation.

Rules and lifecycle at a glance

These are initial defaults, not immutable promises. Staff can change policy through the application; confirm current values before troubleshooting.

Rule	Default
Working calendar	Monday-Friday, 09:00-17:00 Nepal time, excluding configured holidays
Meeting duration	30-120 minutes

Rule	Default
Start/end increment	15 minutes
Gap after meeting	15 minutes, counted once
Advance horizon	14 days, including recurring occurrences
Recurrence	Daily/weekly/monthly; finite occurrences, no automatic future renewal
Room Requires approval	Checked by default; staff can change each room
Employee request	Checked room: Pending hold until review/expiry; unchecked room: Approved immediately
Staff-created booking	Approved immediately
Unreviewed request	Cancelled at meeting start when reconciliation runs
Check-in	From start until before start + 15 minutes
No check-in	No show; occupancy released, history preserved

Substantive employee edits follow the selected room's current approval setting. Changing a room's setting alone, or saving a booking unchanged, preserves saved statuses; existing Pending requests are not bulk-approved. Recurring occurrences follow that room's policy at creation, are reviewed individually when Pending, and are later edited/cancelled individually. Booking and closure conflicts share a PostgreSQL exclusion constraint. Staff overrides never allow simultaneous occupancy; WDN third-floor senior-management priority is handled manually.

Where to go in the application

Append paths to <http://mbs.wdn.com.np>. Email-token URLs are generated by the system and must not be guessed/bookmarked/shared.

Task	Path	Access
Employee sign-in	/sign-in/	Allowed company email
Overview	/	Public landing page; workspace after sign-in
Rooms/directory	/rooms/	Signed-in user; no room-edit controls here
Calendar	/calendar/	Signed-in user; broader staff view
New booking/request	/bookings/new/	Signed-in user; staff can book on behalf
Own bookings	/bookings/mine/	Organizer's records
Profile	/profile/	Own names/department
Staff sign-in/code	/staff/sign-in/, /staff/code/	Active staff, password plus emailed code
Staff dashboard	/staff/	Verified staff
Add/edit rooms	/staff/rooms/, /staff/rooms/new/, /staff/rooms/<room_id>/	Verified staff; Edit link in room row

Task	Path	Access
All bookings/pending	/staff/bookings/, /staff/bookings/pending/	Verified staff
Closures	/staff/blocks/, /staff/blocks/new/	Verified staff
Booking rules/holidays	/staff/policy/, /staff/holidays/	Verified staff
People/access/setup emails	/staff/users/	Verified staff
Reports/Excel	/staff/reports/, /staff/reports.xlsx	Verified staff
Audit	/staff/audit/	Verified staff
Health	/livez/, /healthz/	Read-only operational endpoints
Development email inbox	/dev/mail/	Loopback local file-mail only; unavailable in production

The product's admin interface is **Staff desk**. There is no configured Django `/admin/` route. A staff account using employee email-link sign-in gets employee capabilities; use staff password/code sign-in for management.

Quick operator commands (Linux)

All commands below use the selected production profile. Read the full guide for initial setup, schema updates and recovery.

BASH / Display wraps; copy complete commands from the Markdown source.

```
cd /opt/mbs/MeetingRoomSystemWDN
```

```
# Read-only status and recent logs.
```

```
docker compose -f compose.prod.http.yaml ps -a
```

```
docker compose -f compose.prod.http.yaml logs --since 15m --tail=200 --timestamps web worker proxy db
```

```
curl --fail --show-error --max-time 15 http://mbs.wdn.com.np/healthz/
```

```
# Start already-installed services in the background.
```

```
docker compose -f compose.prod.http.yaml up -d web worker proxy
```

```
# Stop without deleting data (brief outage).
```

```
docker compose -f compose.prod.http.yaml stop proxy web worker
```

```
# Create a private database backup (writes a dump, does not stop the app).
```

```
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
```

Followed logs can be left with Ctrl+C without stopping services. If attached to interactive `docker compose up`, press **d** when its menu offers **Detach**; future starts should use `up -d`. Avoid `down --volumes`, global Docker pruning, direct SQL business-state edits and production test runs.

Support boundaries and remaining operational decisions

The application includes configurable room records, finite recurrence, manual priority, approval, notifications, reports and audit; it does not include automatic monthly renewal, attendee personal-calendar conflict checks, ICS/HCL/Outlook integration, separate Front Desk permissions, attachments, a public REST API, high availability or automatic data-retention purging.

IT/company owners still choose backup destination/retention and acceptable recovery objectives, monitoring/log retention, actual room/holiday/policy records, priority procedures and pilot acceptance. Tests/rehearsals validate implementation; they cannot establish employee-device DNS, this server's relay authorization or company-wide readiness by themselves. No passwords, SMTP credentials or raw authentication links are included in these guides.

When behavior changes, update the relevant guides and regenerate the handbook with `python docs/build_handbook.py` using the optional documentation dependency. See [handbook build instructions](#).

2. Features and use cases

Source: docs/features-and-use-cases.md

Meeting Booking System (MBS) Transgate Tech | Binayak Bhandari Documentation checked against application source on **8 October 2026**.

This reference describes implemented behavior. The selected office deployment is **http://mbs.wdn.com.np**, bound to **192.168.50.222:80**, with PostgreSQL and Docker Compose. Both **@wdn.com.np** and **@transgate.com.np** are accepted employee domains in the production profile. Staff means Front Desk or Administrator: both have the same permissions.

Use the [employee guide](#) for employee workflows, the [staff guide](#) for administration, the [HTTP deployment guide](#) for installation, and [troubleshooting](#) for diagnostic commands. This is a feature reference, including behavior that is easy to miss in normal use.

Contents

1. [Branded web interface](#)
2. [Accounts, sessions and permissions](#)
3. [Page and feature directory](#)
4. [Room discovery and availability](#)
5. [Booking fields and validation](#)
6. [Approval and booking lifecycle](#)
7. [Recurring meetings](#)
8. [Check-in and automatic release](#)
9. [Email notifications](#)
10. [Staff administration](#)
11. [Reports and audit](#)
12. [Use-case catalogue](#)
13. [Implemented scope and limitations](#)

Branded web interface

The application includes the Transgate logo, navy/red brand colors and the requested Times New Roman font (with Times/serif fallbacks). It provides responsive form/cards/table layouts, keyboard focus styling, a Skip to content link, named table regions, accessible status/error messages and confirmation dialogs for consequential form actions. Core pages/forms work without JavaScript; JavaScript adds the three-step booking wizard, room availability previews, confirmations and repeated-click prevention during a submitted request. Wide calendar/report tables scroll horizontally on smaller screens. No external font installation, browser plug-in or webcam/location access is required.

Accounts, sessions and permissions

Employee access

An eligible employee enters a company email at [Employee sign-in](#). Email addresses are trimmed and normalized to lowercase. Eligibility checks the complete domain after @; a different domain or a subdomain is not automatically accepted.

The employee receives a one-time link valid for **15 minutes**, opens it and presses **Continue to workspace**. Opening the link alone does not authenticate the user or consume it; the confirmation button performs that action. This accommodates mailbox scanners that open links. The account is created on successful confirmation if it does not exist. There is no employee password requirement or independent employee sign-in code entry screen.

The request page uses a generic response. "Check your email" is not proof that the account is eligible or that the relay delivered the message. Invalid domains, inactive accounts, throttling and delivery failures can prevent a usable link. Request limits are **three requests per address** and **20 per source IP** in a 15-minute window.

Staff access

Staff sign in at [Staff sign-in](#) using their company email and password. The second factor is an **eight-digit email code**, valid for **10 minutes**. A challenge permits at most **five failed attempts**. Starting another successful password challenge invalidates older staff codes for that account.

A staff account first receives a password setup link from an existing verified staff member or the server's `create_staff_account` command. The setup/reset link lasts **one hour**. Passwords must match, contain at least **12 characters**, and pass the application's password validators (including common, numeric-only and account-similarity checks). There is no public "forgot password" self-service form; staff request another setup link from a colleague or IT.

The password stage limits requests to **five per address** and **30 per source IP** in a 15-minute window. The employee link route can sign a staff member in as an employee, but does not grant verified staff powers. A staff member who uses employee link sign-in must complete staff password-plus-code sign-in before using management pages.

Session lifecycle

Successful employee or staff sign-in creates an **eight-hour** session. This is an expiry from sign-in, not a documented sliding idle timer. Sign out removes the current browser session. Cookies must be enabled for sign-in confirmation, form submission and email check-in.

Staff permissions are checked against the active account and the session's verified staff status. Active/staff access changes in People revoke existing authentication sessions and pending sign-in challenges. Password changes also invalidate existing sessions. The account history remains. Account deactivation does not itself cancel its meetings; staff review those separately.

Permission matrix

Action or information	Employee session	Verified Front Desk / Administrator
Browse active rooms, facilities and instructions	Yes	Yes
Day and week availability	Yes	Yes
Month availability	No; a month request falls back to week	Yes
See another organizer's private booking detail	No	Yes
See occupancy of another employee's slot	Yes, as unavailable	Yes, with a link to detail
Read own meeting title, attendees, notes and decisions	Yes	Yes
Submit own booking request	Pending if selected room requires approval; otherwise Approved	Yes, staff creation starts Approved
Book on behalf of another employee	No	Yes
Edit/cancel another organizer's eligible booking	No	Yes
Approve/reject requests	No	Yes
Enter a booking-policy exception	No	Yes, with an override reason
Manual check-in within the check-in window	No	Yes
Manage rooms, closures, rules, holidays and access	No	Yes
View reports, Excel exports and audit log	No	Yes
Use local test inbox on the office production server	No	No

An attendee's invitation does not grant access to the organizer's detail page or editing rights. Staff and organizer can read Front Desk notes; those notes are not a separate staff-private comment field.

Page and feature directory

Paths below are relative to <http://mbs.wdn.com.np>. Numeric identifiers in {braces} are assigned by the application; use page links rather than guessing IDs.

Page / path	Purpose	Access
/	Public landing page, or authenticated workspace overview	Public / signed-in employee

Page / path	Purpose	Access
/sign-in/	Request employee email link	Public
/sign-in/confirm/	Confirm an email link opened in this browser	Pending email-link session
/profile/	Update own first name, last name and department	Signed in
/rooms/	Search/filter active room directory	Signed in
/rooms/{room_id}/	Facilities, description and instructions	Signed in; active rooms
/calendar/	Day/week availability; staff month view	Signed in
/bookings/new/	Three-step meeting request or approved staff booking	Signed in
/bookings/availability/	Read-only room availability for the full selected schedule/series	Signed in
/rooms/{room_id}/photo/	Authenticated optional room photo	Signed in; active room, or verified staff for inactive room
/bookings/mine/	Own Upcoming and History lists	Signed in
/bookings/{booking_id}/	Private detail, decision and available management actions	Organizer or verified staff
/bookings/{booking_id}/edit/	Edit one future pending/approved occurrence	Organizer or verified staff
/check-in/confirm/	Confirm arrival after opening emailed check-in link	Valid organizer link and browser session
/staff/sign-in/	Staff password stage	Public
/staff/code/	Complete pending staff email-code challenge	Pending staff challenge in same browser
/staff/	Staff overview and operational indicators	Verified staff
/staff/bookings/pending/	Pending queue, earliest start first	Verified staff
/staff/bookings/	Search/filter all booking statuses	Verified staff
/staff/rooms/	Active and inactive room administration	Verified staff
/staff/rooms/new/	Add a room	Verified staff
/staff/rooms/{room_id}/	Edit a room	Verified staff
/staff/blocks/	List room closures and remove them	Verified staff
/staff/blocks/new/	Reserve an unavailable period	Verified staff
/staff/policy/	Configure booking rules	Verified staff
/staff/holidays/	Add/update/delete company holidays	Verified staff

Page / path	Purpose	Access
/staff/users/	Employee directory, profile fields and access controls	Verified staff
/staff/reports/	Date-range usage reports	Verified staff
/staff/reports.xlsx	Download selected report as Excel	Verified staff
/staff/audit/	Read-only, filterable action history	Verified staff
/healthz/	Application/database readiness	Health tooling; no booking information
/livez/	Process liveness	Health tooling; no booking information

Email sign-in, check-in and password setup also have generated token-bearing routes. These are private links, not permanent navigation URLs. Do not copy them into logs or documentation. The custom staff pages are the administration interface; **no /admin/ route is exposed**.

Action and generated-link routes

These routes complement the page directory. Buttons/forms supply the required session-backed CSRF token; do not invoke management POSTs as unauthenticated GET links.

Route	Method	Purpose / access
/sign-out/	POST	End current login session
/bookings/{booking_id}/cancel/	POST	Organizer/verified staff cancels eligible record
/staff/bookings/{booking_id}/approve/	POST	Verified staff approves future Pending
/staff/bookings/{booking_id}/reject/	POST	Verified staff rejects Pending with reason
/staff/bookings/{booking_id}/check-in/	POST	Verified staff manual check-in during window
/staff/rooms/{room_id}/approval/	POST	Verified staff changes Requires approval; existing statuses retained
/staff/blocks/{block_id}/cancel/	POST	Verified staff removes active closure
/staff/holidays/{holiday_id}/delete/	POST	Verified staff deletes holiday entry
/staff/users/save/	POST	Verified staff saves profile/access state
/staff/users/{user_id}/setup/	POST	Verified staff sends active staff setup/reset mail
/sign-in/link/{private_token}/	GET	Stage email token and redirect to confirmation; does not consume it
/check-in/link/{private_token}/	GET	Stage check-in token and redirect to confirmation; does not consume it

Route	Method	Purpose / access
/staff/set-password/{private_uid}/{private_token}/	GET / POST	Validate private setup link; POST sets password
/dev/mail/	GET	Loopback file-mail development only; production returns 404

Room discovery and availability

Workspace overview

After sign-in, the overview shows active room count, upcoming approved meetings, pending request count, booking-window length and office hours. It previews up to six active rooms and links to the full directory, calendar and booking form. The staff overview is a separate page with company-wide operational counts.

Room directory and detail

The employee room directory supports:

- Search by room name, floor or description.
- Minimum seat count.
- Exact location and facility filters selected from active room values.
- Resetting filters and pagination, with **12 rooms per page**.
- Room cards with name, location, floor, capacity, facilities and **Staff approval required** or **Automatic approval**; staff can supply an optional real photo used by the booking wizard.
- Detail pages with descriptions, instructions, equipment labels and **Book this room**.

Facilities are descriptive free-text labels such as "Projector" or "Whiteboard". They are not separate reservable inventory items. Minimum seats filters room capacity; it does not reserve seats or check attendee schedules. Only active rooms appear in employee search, detail and calendar views.

Calendar

All booking times are **Nepal time (Asia/Kathmandu, NPT)**. The day view displays one column per active room and one row per configured booking increment. A free cell offers **Available +**, which preselects room/date/start time on the booking form. The wizard keeps those selections, proposes a duration/end time under the current policy, and lets the organizer adjust them and complete the remaining fields.

Calendar markings distinguish an available start, unavailable occupancy, the current user's own booking, and dates/times outside normal rules. Pending holds, approved bookings, checked-in/completed occupied intervals, room closures and meeting buffers can all make a cell unavailable. Other employees see generic occupancy. Organizers and verified staff can open booking detail links for accessible meetings.

Week view shows counts by day and room, with links to day view. Staff also get a month view. Counts cover active reservation records, including closures; they are not a count of approved meetings only. Day view marks closed weekends, holidays, past times, dates outside the horizon, insufficient time before closing, and insufficient time for a minimum meeting plus buffer before another reservation.

The calendar reflects the database when loaded. It is not a live push-updated feed or a guarantee that another person cannot reserve a slot. Availability is checked again when a form is saved. Reload after other users' changes. Dates more than 366 days from today fall back to today.

Booking fields and validation

Three-step booking flow

The booking page has workspace navigation and three steps:

Step	Actions	Before continuing
Schedule & details	Choose date, duration/start or custom times; select a room; enter title, meeting type, department, purpose, guest company, refreshments and special requirements; choose recurrence when creating	Check required meeting details and current room/time availability
Attendees	Add email addresses and additional guests without individual email entries; verified staff can set organizer for an employee	Check the displayed total against room capacity; organizer is already counted
Review & confirm	Review room, date/time, actual recurring occurrences, meeting information and attendees; use Edit/Back to correct values	Submit once; only successful saving holds the interval

Duration/time choices follow the current policy rather than a fixed quarter-hour rule. New booking defaults aim for a one-hour meeting within minimum/maximum duration and office hours, with an aligned future start on a working day. Room/date/start links from the calendar take precedence as selections; final validation still applies. An edit preserves its existing values and changes one occurrence only.

The room preview checks the **whole selected schedule**, including every eligible recurring occurrence, against Pending requests, approved/checked-in/completed occupied intervals, closures and required turnaround gaps. A room is available only if all the proposed occurrences fit. The review shows the actual occurrence list, including daily skips. Schedule changes refresh the preview. Preview is read-only: it neither holds a room nor sends invitations. The server repeats authoritative checks on final submission, so another user's intervening reservation can still produce an error.

Availability reveals room metadata and occupied time ranges; it does not disclose another organizer's email, title, description, notes, booking state or closure reason. During an authorized edit it excludes the occurrence being edited, but keeps other occurrences in the same series. Attendee capacity is checked separately and again on save.

If JavaScript is unavailable, the underlying native form displays room/date/start/end and meeting/attendee fields with the same Submit action. It retains server validation and the same request/approval behavior; live preview and staged review require JavaScript. A failed request displays field/general errors and retains entered values so they can be corrected.

Fields

Field	Meaning / rule
Room	An active room, required
Date, start time, end time	Same-day future meeting in Nepal time; end after start
Title	Required, at most 200 characters
Description	Optional, at most 10,000 characters
Meeting type	Internal, External, or Internal + External (mixed)
Guest company name	Required for External and Internal + External; at most 200 characters
Additional guests without email addresses	Stored as external attendee count; numeric value from 0 to 500; do not double-count email-listed people
Department	Optional, at most 120 characters; initially copied from profile
Attendees	Up to 50 distinct valid email addresses; newline, comma, semicolon or whitespace separated
Refreshments requested	A request flag visible to Front Desk, not a fulfilment or stock workflow
Front Desk notes	Optional preparation information, at most 10,000 characters; visible to organizer and staff
Recurrence	One meeting, Daily, Weekly or Monthly
Until date	Required for recurring creation; cannot precede first date
Organizer email	Verified staff only; blank means current staff user; otherwise eligible active employee
Override reason	Verified staff only, at most 500 characters; a nonempty reason enables policy exceptions

Attendee addresses are normalized to lowercase and deduplicated. Attendees may use external domains; the company-domain restriction applies to account sign-in and organizer selection. The application does not check attendees' personal calendars. External attendee count represents additional people without individual listed addresses: avoid counting a person both in the email list and in the external count.

Capacity calculation: one organizer + distinct listed addresses other than the organizer + external attendee count. An organizer, five other listed emails and two unlisted external guests require eight seats. Staff exceptions cannot bypass capacity.

Default booking rules

These are the source/migration defaults. Staff can change numeric/time values under **Rules**; the running database is authoritative.

Rule	Initial value	Behavior
Working days	Monday-Friday	Weekends excluded by application logic; no working-weekday editor
Opening / closing	09:00 / 17:00	Meeting start/end must fit inside these hours

Rule	Initial value	Behavior
Minimum duration	30 minutes	Shorter normal bookings rejected
Maximum duration	120 minutes	Longer normal bookings rejected
Time increment	15 minutes	Start/end align to minute offsets from midnight, such as 10:00 or 10:15
Gap after meeting	15 minutes	Room remains occupied after meeting end
Advance window	14 days	Every occurrence, and recurring until date, must stay inside calendar-date limit
Check-in deadline	15 minutes after scheduled start	Confirmation must occur strictly before deadline
Holidays	Dates entered by staff	Normal bookings on listed holidays rejected

The horizon uses Nepal calendar dates rather than an exact rolling 336-hour duration. If today is 7 October and the rule is 14 days, 21 October is the last eligible booking date. Past start times are always rejected, including a start earlier today.

A 10:00-11:00 meeting with the default gap occupies **10:00-11:15**. The next meeting can begin at 11:15 if its full occupied interval is available. The gap is appended after the end; it is not added both before and after. A meeting ending at office closing can have its buffer continue after closing.

Concurrency and conflicts

Pending requests already hold room time. The application validates overlaps and PostgreSQL also rejects overlapping active occupied intervals, including simultaneous requests. Room locks and the database constraint cover bookings and closures. A stale calendar cannot create two reservations for the same occupied time. Choose another room/time if an error says unavailable. A conflict notice is also queued to the user who made the failed create/edit attempt.

Attendee count, future time, active room/organizer and room conflict rules still apply to staff. An exception reason bypasses normal advance-window, weekend/holiday, office-hour, duration and increment checks; it does not bypass those fundamental constraints. The configured gap still applies.

Approval and booking lifecycle

State reference

Status	How reached	Holds occupied interval?	Normal available next actions
Pending approval (pending)	Employee creation/substantive edit in a room requiring approval; existing Pending retained after room-policy toggle or unchanged save	Yes	Future edit; staff approve/reject; cancel; automatic expiry at start
Approved (approved)	Staff approves Pending; staff creates; employee creates/substantively edits in an unchecked room; substantive staff edit of Pending in an unchecked room	Yes	Future edit; cancel; organizer or manual staff check-in during window; automatic No show if missed
Rejected (rejected)	Staff rejects Pending with reason	No	Read history/reason; submit a separate new request
Checked in (checked_in)	Valid email check-in or staff manual check-in	Yes	Cancel if necessary; automatic completion after end
Completed (completed)	Worker reconciles a checked-in meeting after scheduled end	Yes, its historical occupied interval	View/report; no normal edit/cancel
Cancelled (cancelled)	Organizer/staff cancellation or approval-window expiry	No	View history; create a new request
No show (no_show)	Approved meeting misses check-in deadline	No	View history; create new booking if required
Blocked (blocked)	Staff room closure	Yes	Remove closure
Closure cancelled (block_cancelled)	Staff removes closure	No	View closure history

Completed meetings keep their historical interval to preserve conflict integrity. This does not block future dates. No show is the release state for a missed check-in; it is distinct from cancelled even though it releases the room.

Submission and review

The selected room's **Requires approval** setting determines employee submission status. It defaults to checked: submission creates Pending, queues a request message to the organizer and active staff, and does not issue approved invitations to attendees. The detail page shows that review is required. Staff review each future Pending request and approve it or reject it with an explanation of up to 500 characters.

For an unchecked room, a valid employee submission starts **Approved** immediately and queues normal confirmation to organizer/attendees/staff, including eligible organizer check-in instructions. No staff reviewer is invented: automatic approval records a timestamp with no approval actor. The current room setting is read under a transaction lock when saving; a preview does not override it. Capacity, conflicts, buffers, date/duration rules and ownership checks are identical for both policies.

Approval records the staff actor and time, changes state to **Approved** and queues confirmation. The room and organizer must still be active; the meeting must not have started. Approval confirms the stored request; it does not re-run every changed holiday/policy/capacity value. If room details or rules changed after submission, staff should review the request manually.

Rejection records staff actor, time and reason, releases the interval and queues a rejection message to the organizer and active staff. The organizer can read the reason. A rejected occurrence is not editable or approvable again; submit a new request if arrangements change.

Modification

A substantive change includes room, organizer, time/occupied buffer, meeting fields (for example title, department, purpose or refreshments), or the normalized attendee set. A new configured gap can change the saved occupied interval even when the visible meeting times stay the same.

Only future **Pending** or **Approved** bookings may be edited. Ordinary employees edit their own bookings. Verified staff can edit any eligible booking, including changing organizer. Editing a recurring occurrence changes that occurrence only; recurrence fields are not offered on the edit form.

A substantive employee edit follows the **selected room's current policy**, even when moving to another room. Checked means **Pending** with approval metadata cleared and an updated request queued; unchecked means **Approved** with an automatic approval timestamp and no attributed reviewer. Old check-in links and obsolete queued mail are invalidated.

A form saved without substantive changes keeps its original status, including a **Pending** request whose room was later unchecked. Staff editing **Approved** keeps it **Approved** and queues changes. A substantive staff edit to **Pending** in a checked room leaves it **Pending** for a separate **Approve** action; in an unchecked room it becomes **Approved** automatically. A **Pending-to-Approved** edit queues confirmation; an **Approved-to-Approved** edit queues change notices. Changing the room setting alone does not rewrite any booking or queue approval mail.

Availability, capacity and applicable policy are checked for the proposed new state. Failed edits do not discard the existing booking. Revision numbers prevent outdated pending email from being sent as if it described the current booking. Change messages for approved bookings include previous attendees/organizer when those recipients were removed or changed.

Cancellation and expiry

Organizers and verified staff may cancel **Pending**, **Approved** or **Checked** in bookings. The reason is stored (up to 240 characters), the interval is released, old check-in tokens are invalidated and cancellation messages are queued. Cancellation is an explicit confirmed action, not a delete.

Pending requests still pending at their scheduled start are cancelled with **"Approval window expired before the meeting started"**. The worker performs reconciliation, and booking creation/edit also releases overdue pending/no-show holds before checking availability. This is not a waitlist or automatic approval process.

Recurring meetings

Recurring creation supports **Daily**, **Weekly** and **Monthly**, using a first date and inclusive until date. Each saved occurrence has its own status, approval, check-in, emails and management actions, associated with a series record.

Pattern	Actual behavior
Daily, normal rules	Creates each company working day between first/until dates; skips weekends and configured holidays
Weekly	Every seven days from first date; a closed occurrence makes the whole normal request invalid
Monthly	Same day number in later months; clamps to the last day of a shorter month
Staff exception	Can include normally excluded dates, within remaining recurrence safety limits

All occurrences must validate and be available. **The series saves all-or-nothing**: one conflict rejects the entire creation; no partial series is kept. Creation is limited to **15 occurrences** and a date span of at most **366 days**, even with an override.

For normal users the **until date itself** must be inside the configured advance window. The system rejects a distant until date; it does not silently truncate a year-long series to two weeks. Under the default 14-day window, a monthly request usually creates only its first occurrence. The organizer books later months manually when they enter the window. A month-end first date can, depending on the current date/window, produce a second short-month occurrence.

There is no automatic rolling creation, "cancel whole series", series-wide edit, series-wide approval or background extension into later months. Manage and review occurrences individually.

Check-in and automatic release

The organizer's approved confirmation/change emails and reminder may include a **one-time check-in link**. Attendees and staff receive event notifications, but only the organizer recipient gets that link. The link opens a confirmation page; arrival is recorded only when **Check in now** is pressed.

Check-in is allowed from scheduled start **inclusive** until the configured deadline **exclusive**. A 10:00 meeting with a 15-minute rule can be checked in at 10:00 through just before 10:15; at 10:15 it is too late. Early confirmation is rejected and can be retried from the same staged page after start.

The check-in link proves possession of the organizer's emailed token. The current implementation **does not require a prior employee login session**. The browser still needs its session cookie and CSRF-protected confirmation form. Organizer/account/token/reservation are checked, and expired, used, cancelled, modified or mismatched tokens are rejected. Treat the link as private.

Verified staff can use **Manual check-in** on an Approved booking detail page during the identical window. The button can be visible earlier, but the server rejects an out-of-window action. Manual check-in invalidates outstanding email check-in tokens and records the staff actor.

After the deadline an unchecked Approved meeting becomes **No show**, releases the whole occupied interval and queues release notifications. A late click cannot revive it. Checked-in meetings become Completed when their scheduled end passes. The worker normally reconciles about every **30 seconds**, so state changes occur on the next successful cycle, not necessarily at the exact boundary. Worker and SMTP health matter; delivery failures do not extend the check-in deadline.

Email notifications

Who receives what

Recipients are deduplicated by address. "Staff" below means every currently active staff account when that event/reminder is queued.

Event	Organizer	Listed attendees	Active staff	Check-in link
Employee request / updated Pending request	Yes	No	Yes	No
Rejection	Yes, with reason	No	Yes	No
Approval / automatic approval / staff creation confirmation	Yes	Yes	Yes	Organizer recipient only while valid
Approved booking change	Yes	Yes, including former removed recipients	Yes	Organizer recipient only while valid
Cancellation	Yes, with reason	Yes	Yes	No
No show release	Yes	Yes	Yes	No
Successful check-in	Yes	Yes	Yes	No
Reminder	Yes	Yes	Yes	Organizer recipient only while valid
Conflict on failed create/edit	User who attempted action	No	No extra staff broadcast	No

Login links, staff codes and password setup emails are sent directly during their requests. Booking events use a separate database queue, so a saved meeting is not rolled back because subsequent SMTP delivery fails.

Timing and retry behavior

Approved future meetings become eligible for a reminder within **one hour** of start. The worker queues it once per recipient, booking revision and scheduled start. This is not an exact timer guaranteeing delivery at T-60 minutes. A booking approved inside that hour becomes eligible on the next reconciliation. Pending/rejected/cancelled meetings do not get an approved reminder.

The worker normally reconciles states/reminders every 30 seconds and attempts one queued email per iteration. Backlog and SMTP latency can delay delivery. Failed queued mail retries up to **five total attempts**, with automatic retry delays of 2, 4, 8 and 16 minutes after failures one through four. After the fifth failure it remains Failed until IT requeues an eligible job; its recorded next-attempt timestamp is 32 minutes ahead but does not trigger an automatic sixth send. Recovery uses leases to avoid concurrent claims; a crash after SMTP acceptance can still cause duplicate mail when its lease is recovered. Mail delivery is not exactly-once.

Obsolete events are marked **Superseded** when state/revision changes. Worker checks current booking state before delivering related queued mail. Disabled/unconfigured SMTP leaves notifications waiting; it does not verify mailbox delivery. An SMTP "accepted" response proves relay acceptance, not final inbox placement. See [troubleshooting](#) for queue inspection, worker checks and retry commands.

Staff administration

Dashboard and bookings

Staff Overview displays all Pending requests, today's approved/checked-in/completed meeting count, active rooms, today's No shows, failed mail and up to 12 upcoming Approved meetings. The failed-mail number is a signal to investigate with IT; the dashboard is not a notification body/token viewer.

Bookings filters by title/room/organizer search, status, room and inclusive start-date bounds. General Bookings is latest meeting first; Pending queue is earliest scheduled start first. Lists paginate (normally 25 items) while preserving filters. Staff can open details, review, edit/cancel and book on behalf.

Rooms

Staff add/edit room name, location, floor, positive capacity, description, instructions, facilities, optional real room photo, Active state and **Requires approval**. The combination of name/location/floor must be unique. Facilities accept comma, semicolon or newline separators, up to 50 names of at most 120 characters, deduplicated case-insensitively.

Photo upload accepts a still JPEG, PNG or WebP up to 5 MiB and 12 million pixels. The server generates a JPEG no larger than 1920 pixels on its longest edge, removes supplied metadata and gives it a generated filename. Existing rooms without photos remain valid. Edit can replace a photo or select Remove current photo; selecting both is rejected. Replacement/removal deletes the previous file only after the database change commits. Photo delivery requires authentication; ordinary employees cannot fetch inactive room photos, while verified staff can preview them. No public media directory is served.

Deactivation removes the room from employee discovery/new selection and prevents approval while inactive. It preserves bookings/history; it does not automatically cancel meetings or prevent a previously approved meeting reaching its check-in flow. Reconcile affected meetings before deactivating. There is no hard-delete room action. Edit the same record or deactivate it when its history should remain associated.

Room approval policy

Staff desk -> Rooms contains a **Requires approval** checkbox for each active/inactive room. With JavaScript, changing it submits that row immediately; without JavaScript, select the desired state and press that row's **Save** button. The add/edit form provides the same field. Checked is the default for a new room and for every existing room upgraded through migration 0011.

Checked rooms require staff review of employee creations/substantive edits; unchecked rooms confirm those valid requests immediately. Staff-created bookings remain Approved for either setting. A saved toggle takes effect for later writes without a process restart, and is audited as `room_approval_changed` with its old/new values. The staff-only POST validates CSRF/current verified access and locks the room. It changes no booking status: an existing Pending request still needs explicit review or a substantive eligible edit; an existing Approved meeting is not demoted. Use the Pending queue to resolve the backlog deliberately.

Closures

Staff may reserve a future same-day start/end period with a reason (up to 240 characters). Closures use the room-conflict constraint and cannot displace a Pending or Approved hold. They block exactly the chosen interval without appending the meeting gap.

Closures do not apply normal meeting horizon, working-day, office-hour, minimum/maximum-duration or server slot-alignment checks, although the time widget uses the configured step. They are intended for maintenance and other unavailable periods. The UI supports one date per closure; split a multi-day closure into separate entries. Removing one releases it and keeps a "Closure cancelled" history entry. There is no dedicated closure edit form; remove and recreate if necessary.

Rules and holidays

Timing rules are one office-wide policy, not different limits per room/user. Editable fields are opening/closing times, minimum/maximum meeting minutes, slot increment, post-meeting gap, advance days and check-in minutes. The separate Requires approval checkbox is managed per room, not in Rules.

The form enforces ordered office hours, ordered duration limits, duration limits as whole increments, office hours aligned to increments, minimum duration fitting within office hours, and check-in closing within the shortest permitted meeting. Numeric bounds are minimum/maximum duration 1-1440, increment 1-60, gap 0-120, advance

1-366 and check-in 1-1440, subject to those cross-field checks.

Changes apply to subsequent validation. Existing occupied buffers are stored with each booking and are not rebuilt when the gap changes. Check-in uses current policy at confirmation/reconciliation, so changing its deadline affects existing Approved meetings. Adding a holiday or changing hours does not cancel saved bookings. Review affected requests/meetings.

Holidays use a unique date and a name (up to 160 characters). Saving an existing date updates its name. Remove deletes that entry and records an audit event. Weekends remain closed independently of holidays.

People and access

People searches email/name/department and filters Everyone, Staff or Inactive. Staff create/update names, department, Active state and Front Desk / Administrator flag. Email identifies the account: changing the email field creates or updates the account at that address; it is **not** an email-rename feature.

Granting staff does not automatically send setup mail. Save the active staff account, then use **Email setup link** in the row. The same button resets a password. Inactive/nonstaff rows cannot request staff setup.

Removing staff keeps employee access when Active remains selected; deactivation prevents normal sign-in. Access changes revoke current authentication and pending challenges. Existing bookings remain for separate review. You cannot deactivate/remove your own staff access on this page. There is no user hard-delete action or approval chain for staff grants: all verified staff have these powers.

Manual senior-management priority

There is no automatic rank/priority engine. Front Desk handles WDN third-floor senior-management priority manually: review plans, reject/cancel or reschedule the displaced booking with a reason, then make the replacement. An override cannot overlap the still-held interval. Communicate through the company's normal process; MBS records the reservation decision/reason and queues event emails.

Reports and audit

Reports

Report ranges are inclusive Nepal start dates. Presets are Today, This week, This month, This quarter and This year, with period-to-date end points. Default is this month's first day through today. Custom ranges accept an end minus start span of at most 366 days; reversed/excessive ranges show an error and fall back on the report page. Excel rejects an invalid range.

Booking counts include **all booking statuses**, including Pending, Rejected, Cancelled and No show. Closures are not bookings. Department and organizer totals count those records; blank department becomes "Unspecified". Room rows include configured inactive rooms and are sorted most booked first.

Utilization is an **estimate of checked-in/completed scheduled meeting time** divided by bookable time in the full selected period. It uses current Monday-Friday office hours, excludes current holidays and active closures, and clips verified meeting intervals to those windows. It excludes meeting buffers, Pending/Approved-only time and No shows. A checked-in meeting contributes its scheduled full interval, not a sensor measurement of actual attendance or early departure. Overlapping intervals are merged to prevent duplicate minute counting. Meetings beginning before the range can contribute clipped usage minutes without counting as a start-date booking in that range.

Historical policy, room activation, holiday changes and cancelled-closure availability are not reconstructed. A policy edit can change historical utilization. A future-inclusive range increases available time before it has been used; choose an appropriate period when interpreting percentages.

Excel export contains Bookings, Room usage, Departments and Organizers worksheets. Booking fields are ID, date, start/end, room, organizer, department, raw status and title. Headers, column sizing, filters and frozen first rows are

included; user text is emitted as literal strings instead of executable spreadsheet formulas. It is not a database backup or a full meeting-notes export.

Audit log

The read-only staff Audit log records timestamp, actor (or System), action, target type/ID and outcome. Filter by exact action and actor-email substring; pagination uses 50 events per page. It covers sign-in/access/password setup outcomes, room/policy/holiday administration, booking creation/changes/decisions/cancellation/check-in/expiry and relevant mail/security events. System/anonymous events can have no actor. It is not a copy of every page view or every email message.

Structured details may exist in the database for authorized IT diagnosis, but the UI displays summary columns. There is no edit/delete/export button for audit records. The application does not automatically purge records after one year; backup/retention policy and any future cleanup are operational responsibilities.

Use-case catalogue

Examples assume an illustrative eight-seat room and default policy. Adjust dates so they are future working dates within the current window; sample email addresses are placeholders, not actual recipients. Step-by-step versions are in the employee/staff guides.

ID	Scenario	Actor and preconditions	Workflow / result
UC-01	First employee visit	Eligible active mailbox	Request link -> open within 15 minutes -> confirm -> account/workspace; no password
UC-02	Invalid/expired employee link	Employee	Request new link; generic request screen does not confirm eligibility/delivery
UC-03	Staff first login	Access granted, setup email received	Set valid 12+ character password -> staff password sign-in -> newest eight-digit code -> staff desk
UC-04	Staff uses employee login	Staff email, employee session	Employee functions available; management redirects to staff sign-in until MFA completed
UC-05	Find projector room for six	Signed-in employee	Rooms -> seats 6 -> Projector -> instructions -> calendar; no match means change filters/contact staff
UC-06	Reserve reviewed team meeting	Employee, free room with Requires approval checked	Future working Thursday, 10:00-11:00, Internal, "Team planning", colleagues listed -> Pending hold -> review
UC-07	External or mixed client visit	Employee, adequate seats	External or Internal + External, "Example Partners", three unlisted guests + two listed colleagues -> six seats; missing company rejected
UC-08	Concurrent same-time requests	Concurrent signed-in users	One transaction holds interval; other create fails without duplicate; choose another time/room
UC-09	Approve request	Future Pending, active room/organizer	Pending queue -> detail -> Approve -> Approved, actor/time and confirmation mail
UC-10	Reject request	Pending	Reason "Use larger room for visitor count" -> Reject -> Rejected, slot free, reason visible
UC-11	Edit Approved meeting	Organizer, future Approved	Change end 11:00 to 11:30 -> validate -> Pending if selected room is checked, otherwise Approved; old check-in link invalid

ID	Scenario	Actor and preconditions	Workflow / result
UC-12	Staff edits Approved	Verified staff, future Approved	Change eligible fields -> Approved retained -> change mail includes removed recipients
UC-13	Daily stand-up	Employee, all slots free	Daily 09:00-09:30 for future week -> skip closed dates -> all Pending in checked room or Approved in unchecked room; any conflict rolls back the whole series
UC-14	Weekly holiday conflict	Employee	Weekly dates include holiday -> whole normal series rejected; choose dates or ask staff about exception
UC-15	Monthly with 14-day limit	Employee	Until inside window -> usually first only; later month manual; distant until rejected
UC-16	Cancel one recurrence	Organizer/staff, eligible state	Open occurrence -> reason -> Cancel; other occurrences unchanged
UC-17	Email check-in	Organizer holds valid link	Confirm between 10:00 and before 10:15 -> Checked in; prior employee login not required
UC-18	Missing email but present	Verified staff, Approved in window	Verify arrival -> Manual check-in before deadline -> audited; email tokens invalidated
UC-19	No approval by start	Pending	Worker cancels at/after start; approval no longer possible; create new meeting
UC-20	Missed check-in	Approved	Worker marks No show -> room released; late link cannot revive; new booking if still free
UC-21	Equipment repair	Verified staff, future free interval	Closures -> reason/time -> Block; overlap rejected
UC-22	Retire room	Verified staff	Review bookings -> Rooms Edit -> clear Active -> records preserved; manage meetings separately
UC-23	Priority meeting	Verified staff	Resolve hold first -> book on behalf -> reason if policy exception -> Approved; capacity/conflict still enforced
UC-24	New Front Desk colleague	Verified staff, eligible mailbox	People -> Active + Staff -> Save -> Email setup link; permissions equal administrators
UC-25	Departing employee	Verified staff	Cancel/reassign future meetings separately -> deactivate/revoke -> sessions invalidated; history retained
UC-26	Monthly usage review	Verified staff	Reports -> range -> counts/estimated utilization -> Excel; percentages are not actual presence
UC-27	Investigate changed booking	Verified staff	Audit actor/action + detail + IT logs if needed; read-only history
UC-28	Email outage	Web/database operational	Saved state survives; queue waits/retries; IT checks mail/worker; deadline still applies

ID	Scenario	Actor and preconditions	Workflow / result
UC-29	Confirm without staff review	Employee, free room with Requires approval unchecked	Submit valid team meeting -> Approved immediately -> confirmation and organizer check-in; normal capacity/conflict rules still apply
UC-30	Change room approval requirement	Verified staff, room has existing Pending/Approved records	Rooms -> change checkbox -> saved/audited; saved bookings keep status; future submissions follow new policy
UC-31	Resubmit a retained Pending request	Organizer/staff, future Pending in now-unchecked room	Unchanged save stays Pending; substantive valid edit becomes Approved and queues confirmation

Implemented scope and limitations

- First launch includes own calendar, the three-step booking flow with real schedule/series availability, internal/external/mixed meetings, recurring creation, approval, email notifications/check-in and staff administration.
- Room photos are optional staff uploads. They persist in the `room_media` Docker volume; database dumps contain file references, not the photo files. IT backs up/restores both together.
- HCL/Outlook/Exchange integration, calendar invitations/ICS and automatic sync are **deferred**. Email messages do not promise an entry in another calendar.
- Employees use links; staff use password plus email code. SSO, SMS/TOTP and independent employee numeric-code entry are not implemented.
- Recurrence is finite, limited to 15 occurrences; normal dates remain inside the horizon. No rolling monthly scheduler or bulk series action.
- Senior-management priority, refreshment fulfilment and equipment preparation are manual. No automatic displacement, catering integration or equipment inventory reservation.
- Only room intervals are conflict-protected. Attendee availability, concurrent meetings in different rooms for the same person, RSVP and waitlists are not managed.
- Room/user records are retained through inactivity; no normal hard-delete actions. Account/room/holiday changes do not retroactively cancel reservations.
- Reports use current policy/availability and verified scheduled time; they are not sensor occupancy or historical policy ledgers.
- The office uses the selected HTTP profile. A browser "Not secure" label is expected. Production safeguards remain enabled; local inbox/demo seeding are unavailable and forms require the exact site origin.
- Backups, restores, log rotation, monitoring, mailbox delivery, network access and retention are IT responsibilities described in operations documentation.

3. Employee guide

Source: docs/user-guide.md

Meeting Booking System (MBS) Transgate Tech | Binayak Bhandari Checked against application source on **8 October 2026**.

This guide explains employee use of <http://mbs.wdn.com.np> on the office network. Use a company email ending in @wdn.com.np or @transgate.com.np. All displayed meeting times are **Nepal time (NPT)**. For the complete behavior reference see [Features and use cases](#); staff workflows are in the [staff guide](#).

Quick start

1. Open [Employee sign-in](#).
2. Enter company email, request the link, open it within 15 minutes and press **Continue to workspace**.
3. Open **My profile** and set name/department.
4. Find a room using **Rooms** and a time using **Calendar**.
5. Submit **New booking**. A room marked **Staff approval required** starts **Pending approval**; wait for staff. A room marked **Automatic approval** starts **Approved** immediately.
6. At meeting start, open the organizer check-in link in the approved email and press **Check in now** before the deadline.
7. Use **My bookings** for changes, cancellation, status and history.

Navigation

Menu / page	What to do there
Overview	Room count, upcoming meetings, pending count and current booking window
Rooms	Search room, seat count, location and facilities; read instructions
Calendar	Day/week occupancy and available starts
My bookings	Your Upcoming or History list
New booking	Schedule & details -> Attendees -> Review & confirm for one meeting or recurring occurrences
My profile	Your name and department
Staff desk	Management interface; separate staff password/code authentication required
Sign out	End this browser's session

1. Sign in using company email

First or returning sign-in

1. Connect to the office network, or the company-approved connection that reaches the internal site.

2. Open **http://mbs.wdn.com.np/sign-in/** in Chrome or Edge with cookies enabled. Keep this hostname throughout the workflow.
3. Enter your exact company email. Capitalization is normalized; either approved domain is accepted.
4. Submit and check that mailbox. The page says **Check your email** if delivery is configured.
5. Open the link within **15 minutes**. It opens the confirmation page.
6. Press **Continue to workspace**. You are now signed in; the overview shows your account email.

An account is created on first successful confirmation. There is no employee password to remember. Possession of the emailed one-time link verifies access to that mailbox. Opening it alone does not complete sign-in, so mail scanners that inspect links do not normally consume it.

Each completed session lasts **eight hours**. Use **Sign out** on shared devices. An account deactivated by staff cannot sign in. Account permissions/password changes can end an existing session and require fresh authentication.

If the link does not work

Situation	Action
Expired or already used link	Return to sign-in and request a new one; a completed link cannot be reused
No mail arrives	Check Junk/quarantine and spelling/domain; contact Front Desk or IT
Repeated requests	Limit is three per address per 15 minutes; wait for the window rather than repeatedly submitting
"Email unavailable"	IT must configure/repair SMTP; there is no production local test inbox
"Let's start fresh" / session check	Reopen the form at the same hostname, keep cookies enabled and retry; ask IT to inspect CSRF logs if persistent
Staff pages require login again	Employee email-link authentication does not unlock staff powers; use staff password plus email code

The generic mail-request screen does not guarantee a message was sent, accepted or delivered. Share the screen text, time and ordinary page path with IT; keep email sign-in/check-in links and codes private. See [troubleshooting](#) for the server-side checks.

2. Set your profile

1. Choose **My profile**.
2. Enter first name, last name and department, for example "Operations".
3. Save and check the **Your profile has been saved** message.

Department is used as the initial value on future booking forms. You can change it on a meeting where needed. Updating your profile does not rewrite the department already saved on earlier bookings. Email and access flags cannot be changed from this page; contact staff.

3. Choose a room

1. Open **Rooms**.
2. Enter a search such as "3" for floor or a room name/description term.
3. Set **Minimum seats**, for example 6.
4. Select a location or facility, for example "Projector", when offered.

5. Press **Find rooms**. Use **Reset** to clear filters; use pagination for additional results.
6. Open **Room details** to read capacity, facilities, description and instructions.
7. Use **Book this room** to preselect it, or **View availability** to compare times.

Only active rooms are listed. A room's listed facility is a descriptive label; contact Front Desk if you need to verify equipment is ready. Employees cannot edit room records. If no room matches, widen filters or ask Front Desk about available rooms.

4. Read the calendar

1. Open **Calendar**.
2. Select a date and **Day** or **Week**, then press **Show**. **Today** resets the selected date.
3. In day view, find the row/time and room column.
4. **Available +** opens the booking wizard with room/date/start preselected. Check the proposed duration/end and adjust if needed.
5. In week view, select a day to see its detailed slots.

Calendar marking	Meaning
Available +	A permitted start for at least a minimum meeting and required buffer appears free at page load
Your booking	Occupied by a meeting you organize; select to open details
Unavailable	Another booking/request, closure or buffer occupies the interval; details remain private
Outside rule / past / holiday explanation	This start cannot be booked normally; read the displayed reason

Calendar availability is checked again on submission. Another person may hold the slot after you load the page. Staff can see month view; employees use day/week. Other employees see occupancy without your meeting title, notes or attendee details. Being on someone else's attendee list does not grant access to their booking details.

5. Request a single internal meeting

The following is an illustrative request. Choose a **future company working date within the current booking window** and an active room with enough seats; do not submit the placeholder addresses as real invitations.

Field	Example
Room	An available eight-seat room
Date	A future working Thursday
Start / End	10:00 / 11:00
Title	Team planning
Description	Review next week's work and outstanding actions
Meeting type	Internal
Additional guests without email addresses	0

Field	Example
Department	Operations
Attendees	colleague@wdn.com.np, teammate@transgate.com.np (replace with real colleagues)
Refreshments requested	Select only if needed
Special requirements	Projector needed; please check remote batteries
Recurrence	One meeting
Until date	Leave blank

1. Open **New booking**, or use the room/calendar link.
2. In **Schedule & details**, choose a date, duration and start time. Use **Custom** for explicit start/end. All times are Nepal time; the choices follow current office rules.
3. Wait for the room cards to update. Check capacity, location, facilities and any real photo; select a room marked available for the selected schedule.
4. Enter meeting title and information, select **Internal**, set refreshments to Yes/No, and add special requirements. Keep Repeat meeting as **One meeting**.
5. Select **Next: attendees**. Add colleague emails separated by commas or new lines. Duplicate addresses are normalized and removed; the organizer is already included. Enter 0 additional guests and check the displayed headcount fits the room.
6. Continue to **Review & confirm**. Read the room, date/time, meeting details and attendee list. Use **Edit** or **Back** to correct them without restarting.
7. Press **Submit booking request** for a room requiring review, or **Confirm booking** for an automatically approved room, once. Availability and the current room policy are checked again while saving.
8. On success, check the saved status. A room requiring approval shows **Pending approval** and holds the room/time; an unchecked room shows **Approved** immediately.
9. If Pending, wait for staff approval before treating the meeting as confirmed. If Approved, read the confirmation/check-in instructions. Follow the status in **My bookings**.

Room cards and the review screen show the room's approval requirement. Both reviewed and immediately confirmed bookings must satisfy the same capacity, availability and booking rules; staff creation uses its own verified staff path.

The room preview does not reserve space. If the schedule changes, wait for its updated result; an intervening booking can still be rejected at final submission. Field/general errors preserve your entries so you can correct and retry. A photo is optional; a room without a photo can be booked normally.

With JavaScript disabled or unavailable, the page presents one native form with room/date/start/end and all meeting/attendee fields. Enter end time explicitly and submit the same request; server rules and approval still apply. The three steps, duration buttons and live availability preview require JavaScript.

Normal initial rules

Staff can change these under Rules. Current form/calendar messages and office policy take precedence over this default table.

Rule	Initial setting
Working days / hours	Monday-Friday, 09:00-17:00; configured holidays excluded

Rule	Initial setting
Meeting length	30-120 minutes
Start/end increments	15 minutes
Gap after end	15 minutes
Advance limit	14 calendar days
Check-in deadline	Before 15 minutes after start

A 10:00-11:00 meeting holds the room until 11:15 under the default gap. The organizer counts as one person. Listed attendees other than organizer and the external attendee count are added to room capacity. Max 50 distinct listed addresses are permitted. End must be after start, on the same date; past start times are rejected.

Common submission failures

Message / cause	How to resolve
Room unavailable	Another hold/booking/closure or required gap conflicts; choose another room/time
Room seats fewer than listed	Choose a larger room or correct attendee/guest count
Meeting length outside permitted range	Adjust start/end to current duration rules
Outside office hours or increments	Choose aligned times inside current hours
Not a company working day	Choose weekday not listed as a holiday; staff can review a justified exception
Too far ahead	Choose a date within current horizon
Invalid attendee email	Correct the identified address; text names alone are not email addresses

A failed submission does not create a booking. On a failed edit, the previously saved booking remains. Refresh the calendar when correcting a conflict; do not create repeated duplicate requests for the same planned meeting.

6. Request an external or mixed meeting

1. In Schedule & details, select **External** for a guest meeting, or **Internal + External** for a mixed meeting. Both require a guest company name.
2. Enter guest company name, for example **Example Partners**.
3. In Attendees, enter the number of additional guests who are not individually listed by email.
4. List colleague/guest email addresses that should receive approved meeting notifications.
5. Select refreshments if needed and enter preparation needs in **Special requirements**, review, then submit.

Example: organizer + two listed colleagues + three unlisted guests = **six people**. If a guest has already been counted as a listed email attendee, do not count them again as an unlisted external attendee. External addresses may receive attendee notifications, but cannot use account sign-in unless their domain is in the employee allowlist. Refreshments are a request for Front Desk to handle manually, not an automatically fulfilled order.

7. Create a recurring series

1. In Schedule & details, enter first date, time and common meeting details.

2. Set **Repeat meeting** to **Daily**, **Weekly** or **Monthly**.
3. Choose an inclusive **Last date in series** inside the current booking horizon.
4. Wait for availability: each room must fit every eligible occurrence, not just the first. A conflict on a later date also makes that room unavailable.
5. Add attendees, then review the actual occurrence dates/times in **Review & confirm**. Daily skips appear in the resulting list; no hidden later months are reserved.
6. Submit; the detail page opens the first created occurrence.
7. Open **My bookings** to inspect all occurrences. Each starts Pending if the selected room requires approval, otherwise Approved. Pending occurrences need individual review; every Approved occurrence has its own check-in.

Example	Expected behavior
Daily 09:00-09:30 through a future working week	Creates eligible weekdays; skips holidays/weekends
Weekly next Thursday and the following Thursday	Creates both if dates are working days and both slots free
Monthly with until date inside next two weeks	Usually creates first occurrence only; book later months manually when eligible
Any requested occurrence conflicts	Entire series fails; no partial series is saved
Weekly/monthly date lands on holiday/weekend	Entire normal request rejected; alter dates or ask staff about an exception
Until date three months away with 14-day rule	Rejected; the application does not truncate a distant series automatically

At most **15 occurrences** are allowed in one creation. Monthly dates use the original day number, clamped to shorter month-end when necessary. The application does not extend a series automatically. Editing/cancelling one occurrence leaves other occurrences unchanged. There is no whole-series edit/cancel button.

8. Follow approval and status

Open **My bookings** and select the meeting title. The detail page shows room, date/time, requester, request time, department, guests, attendees, description, preparation notes, state and available actions.

State	Meaning / next step
Pending approval	Slot held; contact Front Desk before start if approval is delayed
Approved	Meeting confirmed; organizer receives check-in instructions
Rejected	Read staff reason; slot released; make a new request if needed
Checked in	Arrival confirmed; meeting becomes Completed after scheduled end
Completed	Historical record; cannot edit/cancel normally
Cancelled	Released by cancellation or approval-window expiry; reason shown
No show	Deadline missed; room released; old check-in link cannot restore it

Upcoming lists active Pending/Approved/Checked in meetings whose end has not passed. **History** shows all your booking records, including future records in other states; it is not exclusively past meetings. You cannot open another organizer's private record, even if you know its ID or were invited.

A request still Pending at meeting start is automatically cancelled because its approval window expired. Staff changing the room to **Automatic approval** does not automatically approve saved Pending requests. They remain Pending until staff review or a substantive valid edit; an unchanged save keeps their state.

9. Edit or cancel

Edit

1. Open your meeting from **My bookings**.
2. For a future Pending or Approved record, select **Edit booking**.
3. Use the same three steps to adjust fields, review, and press **Save booking changes**. The original date/start/end, room and meeting information are prefilled; recurrence controls are omitted because this edits one occurrence.
4. Check the resulting state and confirmation message.

A substantive employee edit uses the **selected room's current approval requirement**. Checked means **Pending** for staff review; unchecked means **Approved** immediately. This applies to a change of room as well as details, times or attendees, and invalidates earlier check-in links. An unchanged save retains the state, including an existing Pending request after staff unchecked its room. Current room capacity, availability and policy apply to edits. Started, Checked in, Completed, Rejected, Cancelled and No show records cannot be edited through normal workflow.

Cancel

1. Open the eligible Pending, Approved or Checked in record.
2. Enter a cancellation reason, for example **Client visit postponed**.
3. Select **Cancel booking** and confirm the prompt.
4. Check the Cancelled state and reason.

The room is released; history remains and event emails are queued. Cancellation affects one occurrence only. There is no undo button; make a new request if needed.

10. Check in at meeting start

1. Find the organizer check-in link in the latest approved confirmation/change email or reminder.
2. Open it at the scheduled start time.
3. Press **Check in now** before the configured deadline.
4. Check the **You're all set** confirmation and meeting/room name.

For a 10:00 meeting with the default rule, check-in opens at **10:00** and closes at **10:15**. The deadline is exclusive: at 10:15 it is too late. Opening the link before start is allowed, but confirming early is rejected; keep the page and use **Try check-in again** after start.

Check-in uses possession of the emailed organizer link; a prior employee login is not required. It still requires browser cookies and the confirmation button. Attendees do not receive the organizer's private check-in link. A used or superseded link will fail, and a cancelled/rejected/No show booking cannot be checked in.

If mail is missing while attendees have arrived, contact Front Desk **before the deadline**. Verified staff can perform Manual check-in during the same window. Email problems do not automatically extend it. A missed deadline marks the record No show and releases the room on the next successful worker reconciliation, normally about every 30 seconds.

11. Notifications and support

You receive submission/decision/change/cancellation/check-in/no-show messages for your meetings. After approval, listed attendees and active staff receive the corresponding meeting events; unapproved submission and rejection are sent to organizer/staff, not attendees. Reminder scheduling begins within one hour of an Approved start and depends on worker/relay delivery; it is not guaranteed at precisely 60 minutes before.

A saved booking remains saved during a mail outage. Use the application state as the record of the meeting and ask staff/IT about missing email. HCL/Outlook calendar integration and calendar invitation files are deferred; an MBS email does not automatically create another calendar entry.

When reporting an issue, provide:

- Ordinary page path, such as /bookings/mine/, and approximate Nepal time.
- Action attempted, expected result and visible error text.
- Booking ID, if relevant, and whether it was Pending/Approved/another state.
- Browser name and whether a fresh page/reload reproduces it.

Do not send passwords, sign-in/check-in/setup links, email codes or .env contents. Contact Front Desk for booking decisions/room preparation and IT for access, mail or server issues. Diagnostic commands are in [troubleshooting](#).

4. Staff guide

Source: docs/staff-guide.md

Meeting Booking System (MBS) Transgate Tech | Binayak Bhandari Checked against application source on **8 October 2026**.

Front Desk and Administrators have **identical permissions**. The custom staff desk is the administration interface at <http://mbs.wdn.com.np/staff/>; there is no exposed /admin/ page. Employee booking screens remain available, but staff powers require password-plus-email-code verification. All displayed meeting dates/times use **Nepal time**.

Related documents: [employee guide](#), [features/use cases](#), [HTTP deployment](#), [troubleshooting and log commands](#).

Staff navigation

Menu	Direct page	Main task
Overview	Staff desk	Pending requests, meetings, no-shows, active rooms and failed mail
Pending requests	Pending queue	Review earliest upcoming requests
Bookings	Company bookings	Search/filter all records and open management actions
Rooms	Manage rooms	Add/edit/deactivate rooms
Closures	Room closures	Block/release unavailable periods
Rules	Booking rules	Office-wide booking/check-in policy
Holidays	Company holidays	Non-bookable dates
People	People and access	Profiles, active status, staff grants and setup email
Reports	Usage reports	Date-range reports and Excel export
Audit log	Action history	Read-only decisions and administration outcomes

The top **Rooms** menu opens the employee directory. To find the room edit control choose **Staff desk -> Rooms -> Edit** in the room's row.

1. First access and staff sign-in

Initial account setup

An existing verified staff member grants access in People, then sends a setup email. If no staff member can sign in, authorized IT uses this command **on the Linux server**, replacing the placeholder with the intended company mailbox:

BASH / Display wraps; copy complete commands from the Markdown source.

```
cd /opt/mbs/MeetingRoomSystemWDN
docker compose -f compose.prod.http.yaml exec web python manage.py create_staff_account
staff.member@wdn.com.np
```

This grants Active/Staff and sends a password setup link. It does not set a known/default password. It is intended for initial staff provisioning or authorized recovery, not bulk unreviewed access changes. Normal ongoing grants/revocations should use People.

1. Open the setup message in the intended mailbox.
2. Open its link within **one hour**.
3. Enter a password and matching confirmation, at least **12 characters**, passing the displayed password rules.
4. Save. The page directs you to staff sign-in.

"Staff enabled, but setup email failed" means access was granted even though the message failed. IT repairs delivery; the setup link can then be resent. Do not assume the failed command rolled back access.

Normal sign-in

1. Open Staff sign-in.
2. Enter company email and password.
3. After the password stage succeeds, keep the code page open in that browser.
4. Read the newest **eight-digit** code in that mailbox, including any leading zeroes.
5. Enter it within **10 minutes** and submit.
6. Check that the staff Overview opens.

Up to five failed code attempts are allowed; expired/exhausted challenges require a fresh password stage. A new successful password challenge invalidates older codes. Password requests are limited to five per email and 30 per source IP per 15-minute window. Session lasts eight hours from successful sign-in.

Employee email-link sign-in does not verify staff powers; it can leave a staff member with an employee session. If the staff desk redirects you to sign-in, complete the password/code workflow. A password reset or access change can invalidate existing sessions. Sign out on shared devices.

Password forgotten or setup expired

Ask another verified staff member to open **People**, find your active staff row and select **Email setup link**. Follow the one-hour link and set a new password. There is no public self-service password-reset form. If nobody has working staff access, IT uses the bootstrap command above after confirming the intended account.

2. Daily operating routine

1. Open **Overview** and check Pending requests, today's No shows and Failed emails.
2. Open **Pending requests** and review meetings before their start times.
3. Use **Bookings** to prepare for today's external guests, refreshments and equipment requests.
4. Open **Calendar** for day/week or staff-only month occupancy.
5. Monitor organizer check-in at meeting start; use Manual check-in only after confirming arrival and within the deadline.
6. Investigate failed mail with IT; missing reminders can cause missed check-ins even while meetings remain saved.
7. Review room closures and follow the company's manual senior-management priority process.

What the overview numbers mean

Indicator	Meaning
Pending requests	All records currently Pending; select card to review
Today's meetings	Approved, Checked in or Completed meetings with today's Nepal start date
Active rooms	Rooms marked Active
Today's no-shows	No show records with today's Nepal start date
Failed emails	Queued notifications currently Failed, including failures that may still be retrying
Upcoming meetings	Up to 12 future Approved meetings, earliest first

The overview is refreshed when loaded, not a push-updated live monitor. A zero Failed emails number does not prove all mail reached inboxes; messages can be Pending, accepted by relay but filtered, or login/setup delivery can have failed outside the booking notification queue.

3. Review a Pending request

1. Open **Pending requests**. It lists earliest meeting start first, then request time.
2. Use room, search and date filters if needed; Reset removes them.
3. Select the meeting title to open detail.
4. Review requester, requested timestamp, room/date/time, department, attendee count, guest company, refreshments, description and Front Desk notes.
5. Choose Approve, Reject, Edit or Cancel as appropriate.

Pending requests already hold room/time, including the normal post-meeting gap. Approval does not compete for the slot again with another ordinary request. Check-in instructions are issued only after approval.

Approve

1. Confirm the room/organizer remain active and the meeting has not started.
2. Press **Approve request** and confirm the prompt.
3. Check **Approved**, approval time and success message.

Approval records who approved it and queues confirmation to organizer, listed attendees and active staff. Only the organizer recipient gets its check-in link. If capacity, hours or holidays changed after request submission, review manually: approval does not revalidate every changed policy value.

Reject

1. Enter a clear reason, for example **Please request the larger room for the client group**.
2. Press **Reject request** and confirm.
3. Check **Rejected** and the reason.

Reason is required and limited to 500 characters. The organizer sees it, the occupied interval is released and organizer/staff rejection messages are queued. Rejected records cannot be edited or approved again normally; the employee makes a new request.

Expired/stale request

A request must be approved before its scheduled start. If still Pending at start, it is automatically Cancelled with an approval-window-expired reason. A stale page cannot approve a record already decided/cancelled or a meeting already started; reload and inspect status. Each recurring occurrence must be reviewed separately.

4. Search, edit, cancel and check in company bookings

Find a booking

Open **Bookings** and filter by:

- Search: meeting title, organizer email or room name.
- Status, including Pending, Approved, Rejected, Checked in, Completed, Cancelled or No show.
- Room, including inactive rooms for historical records.
- From/To: inclusive Nepal meeting start dates.

General Bookings is latest meeting first; use filters for forthcoming work. Pagination keeps filters. Select a title for private detail. Verified staff can read all meeting notes/attendees; employees can only open their own organizer records.

Edit

1. Open a **future Pending or Approved** record and select **Edit booking**.
2. Adjust fields, including room or organizer if appropriate.
3. Enter an override reason only if a justified rule exception is needed.
4. Save and inspect the result.

Staff edits to Approved retain Approved and queue change notices. A substantive staff edit to Pending keeps it Pending when the selected room requires approval; when the room is unchecked, it becomes Approved automatically and queues confirmation. An employee's substantive edit follows the selected room's current policy: checked means Pending with prior approval cleared, unchecked means Approved automatically. An unchanged form save retains the state, even if a Pending request's room was unchecked after submission.

Editing one series occurrence does not alter its siblings or recurrence pattern. Availability/capacity/current policy apply; failure leaves the saved booking intact. Old check-in links and stale pending notifications are superseded when substantive changes are saved. Removed attendees/former organizer receive change notices on an approved change.

Cancel

1. Open an eligible Pending, Approved or Checked in booking.
2. Enter an explanatory cancellation reason, such as **Room needed for approved priority meeting; organizer informed**.
3. Press **Cancel booking** and confirm.
4. Verify Cancelled state and reason; check replacement arrangements separately.

The interval is released and history retained. Existing check-in links are invalidated; cancellation notifications are queued. There is no undo/bulk-series cancel action. Cancel each affected occurrence deliberately.

Manual check-in

1. Confirm actual organizer/meeting arrival.
2. Open an Approved booking detail.
3. Within the configured check-in window, select **Manual check-in** and confirm.

4. Check Checked in state.

Default window is scheduled start through **before** start plus 15 minutes: a 10:00 meeting can be checked in at 10:00 but not at 10:15. The button may be visible before start; the server still rejects an early action. Manual check-in records your staff actor, invalidates email check-in tokens and queues check-in messages. No show records cannot be revived using it.

The worker normally reconciles every 30 seconds: Approved meetings missing the deadline become No show and release their intervals; checked-in meetings after end become Completed. Email failure does not extend the deadline. IT must investigate a delayed worker rather than changing timestamps directly.

5. Book on behalf and manage exceptions

1. From **Bookings**, select **Book for employee** +, or use **New booking** while in a verified staff session.
2. Complete **Schedule & details** as in the employee guide: date/duration/start, an available room, title, type, department, purpose, refreshments and special requirements. External and Internal + External meetings require a guest company.
3. For a justified policy exception, open **Staff booking options** and enter **Booking rule override reason**. Use custom/manual times for an exceptional schedule; preview and saving still enforce non-bypassable rules.
4. In **Attendees**, enter **Book on behalf of an employee** at @wdn.com.np or @transgate.com.np. Blank uses your own account. An eligible email not already registered creates an employee account; an inactive organizer is rejected.
5. Add email attendees/additional guests, check the headcount, and continue to **Review & confirm**. For recurrence, review every actual occurrence and its availability.
6. Press **Confirm booking** once.
7. Check Approved state and correct organizer; queued confirmation goes to that organizer/attendees/staff.

A newly created staff booking is Approved without the employee Pending stage. It still needs check-in. New recurring staff bookings create Approved occurrences; existing Pending requests stay Pending after a room-policy toggle or unchanged save, but a substantive valid edit in an unchecked room can approve them automatically.

Exception behavior

A nonempty reason (maximum 500 characters) bypasses normal horizon, weekend/holiday, office-hour, duration and increment checks. For example: **Board meeting approved by Front Desk outside normal office hours**. It records the reason and the staff booking/change action.

The following cannot be bypassed:

- End after start and future start time.
- Active room and active eligible organizer.
- Room capacity calculation.
- Conflicts with bookings, Pending holds, closures and buffers.
- Recurring creation limits of 15 occurrences and at most 366 days.

The configured post-meeting gap still applies. The calendar normally marks closed times as unavailable even for staff; an approved exception is entered through the booking form. Staff edit values, including original custom times and override reason, are prefilled; a substantive edit to Pending confirms it only if the selected room is unchecked; a checked-room request still needs explicit approval. Without JavaScript, the native form supports the same fields/validation, including staff organizer and override fields.

WDN third-floor senior-management priority

Priority is manual, with no automatic ranking/displacement. Review impacted meetings and communicate with organizers under company practice. Reject a Pending request, cancel an eligible meeting, or reschedule it to free the interval; then create the priority meeting. Put the explanation in the decision/cancellation reason and use an override reason if the replacement itself requires a rule exception. An override cannot occupy a still-held interval.

6. Add, edit or deactivate rooms

1. Open **Staff desk** -> **Rooms**. This list contains active and inactive records.
2. Select **Add room** +, or **Edit** in an existing row.
3. Enter name, location, floor, positive seat capacity, description and room instructions.
4. Enter facility names separated by commas, semicolons or newlines, such as **Projector, Whiteboard, Video conferencing**.
5. Optionally choose a real room **Photo**. Use a still JPEG, PNG or WebP no larger than 5 MiB and 12 million pixels; no photo is required.
6. Select **Active** to offer it for bookings; clear Active to retire it.
7. Set **Requires approval**: checked means employee requests require staff review; unchecked means valid employee requests are immediately Approved. New rooms default to checked.
8. Save and check **Room saved**, the updated row and room image in the booking wizard when appropriate.

Example: name **Room 1**, location **Kantipat**, floor **3**, capacity **8**, instructions **Leave the room tidy and switch off the projector**. Use actual office data; example rooms are not automatically imported into production.

Direct links: [staff room list](#) and [Add room](#). Room editing is on **Staff desk** -> **Rooms** -> **Edit**, not the employee Rooms page. It requires staff password/email-code sign-in.

Change approval directly from the room list

1. Open **Staff desk** -> **Rooms** in a verified staff session.
2. Find the correct room by name/location/floor and change its **Requires approval** checkbox.
3. With JavaScript, the row saves immediately when changed. Without JavaScript, press the **Save** button in that row.
4. Verify the saved checkbox and **Room approval setting saved** message. The audit log records the old/new setting when it changes.
5. Review existing Pending requests separately; use Approve/Reject, or make a substantive valid edit if appropriate.

Setting	New employee booking / substantive employee edit	Staff-created booking
Checked	Pending; hold retained until review/expiry	Approved immediately
Unchecked	Approved immediately; confirmation/check-in queued	Approved immediately

Changing this field does not rewrite existing Pending or Approved records. A Pending request remains Pending after an unchecked toggle or unchanged form save. A substantive valid edit to it in an unchecked room can confirm it automatically, whether made by its organizer or verified staff. Recurring creations follow the selected room's policy for each occurrence; no existing series is bulk-converted. The setting takes effect on subsequent saving without an application restart. Migration 0011 checks it for existing rooms to preserve their current review requirement.

Replace or remove a room photo

1. Open an existing room's **Edit** form.
2. To replace the photo, choose a new supported file. To remove it, select **Remove current photo** instead; do not select removal and upload together.
3. Save and verify the new preview, or its absence after removal. A browser cannot retain a selected upload after a failed form submission; reselect the file if correcting an error.
4. If the upload reports an unreadable, animated, oversized or unsupported file, choose a suitable still image. If saving reports a storage problem, ask IT to inspect the media volume; do not reset the database.

The application converts accepted files to JPEG up to 1920 pixels on the longest edge, strips supplied metadata and generates filenames. Replacing/removing deletes the previous file only after the room change commits. Employees can view active room photos while signed in; verified staff can preview inactive room photos too. There is no public media URL or photo gallery. IT must back up the photo volume together with room records, because a database dump alone cannot restore photos.

Name/location/floor must be unique together. Capacity must be positive. Facilities are free-text descriptive labels, up to 50 names and 120 characters each, deduplicated without case sensitivity. Description/instructions allow up to 10,000 characters. Changing a facility label changes search choices; it does not allocate inventory.

Deactivation prevents new selection/approval and removes the room from employee calendars/directory. It preserves records and does **not** cancel existing Approved meetings. Review them first and cancel/reschedule affected occurrences separately. Capacity/description changes do not rewrite or cancel old reservations. There is no room delete button; preserving the record keeps historical links intact.

7. Block or release unavailable periods

Add a closure

1. Open **Closures** -> **Block room** +.
2. Choose active room, future date and start/end.
3. Enter reason, for example **Projector replacement**.
4. Save and verify Blocked state and calendar occupancy.

Closures reserve exactly the chosen interval and do not add the meeting buffer. They require future start and end after start, on one date. They cannot overlap any active hold/booking/closure; resolve impacted bookings before blocking. Normal meeting horizon, office-hours, working-day, duration and server slot checks are not applied to closures. For a multi-day closure, add one entry per day.

Remove a closure

1. Find it under **Closures**.
2. Select **Remove** and confirm.
3. Check **Closure cancelled** and the released interval.

The record stays in closure history. There is no closure time-edit form; remove/recreate when a period changes. Review the calendar before assuming the removed period is available, because another reservation can occupy it.

8. Change booking rules

1. Open **Rules**.
2. Review existing values before editing; there is one office-wide policy.
3. Enter approved changes and save.

4. Check **Booking rules saved** and notify affected employees through office practice.
5. Review existing requests/meetings that could be affected.

Field	Initial value	Allowed form range / relationship
Opens at / Closes at	09:00 / 17:00	Ordered times aligned to configured increments
Minimum minutes	30	1-1440; fits office hours; whole increments
Maximum minutes	120	1-1440; at least minimum; whole increments
Slot minutes	15	1-60
Gap minutes	15	0-120; appended after each meeting
Advance days	14	1-366
Check-in minutes	15	1-1440 and no greater than minimum meeting duration

Example: keeping 15-minute slots and changing maximum from 120 to 180 is a valid whole-increment value if approved by office policy. Setting minimum to 20 with 15-minute increments is invalid; opening after closing is invalid. Working weekdays are fixed Monday-Friday in code and are not editable here.

Changed rules apply to new/edited bookings. Existing occupied buffers remain their stored values; changing the gap does not rebuild them. Check-in uses the **current** deadline when confirming and releasing meetings, so shortening it can affect already Approved meetings. Adding holidays/changing hours does not automatically cancel old bookings. Reports also use current rule assumptions, which can change historical utilization estimates.

9. Manage company holidays

1. Open **Holidays**.
2. Enter date and name, for example **Company holiday**.
3. Save. An existing date updates that holiday's name rather than adding a duplicate.
4. Review scheduled bookings on that date and handle them separately if needed.
5. To remove an entry, use its **Remove** action; the removal is audited.

Dates must be valid and name is required (up to 160 characters). Holiday entries are unique by date. Daily normal recurrence skips these dates; weekly/monthly series containing a holiday are rejected unless staff enters an exception. Existing meetings are not automatically cancelled. Removing a holiday does not open weekends, which remain closed independently.

10. Manage people and access

Add/update an employee

1. Open **People**.
2. Enter an eligible company email, names and department.
3. Select **Active account** for normal employee sign-in.
4. Select **Front Desk / Administrator access** only for staff who need full management powers.
5. Save and check the directory row.

Use Search for email/name/department and filter Everyone, Staff or Inactive. **Edit employee** loads an existing person's values into the form; **Clear form** returns to a new-entry form. Email is the lookup identity. Changing it creates/updates the account at the new address, rather than renaming the original account. Contact IT for an email-identity migration that must preserve ownership history.

Grant staff and send setup

1. Save the person with both Active and Front Desk / Administrator selected.
2. In that active staff row, select **Email setup link**.
3. Confirm **Password setup link sent**.
4. Ask the person to check their mailbox and complete the one-hour setup.

Saving staff access does not automatically send this email. The button sends directly through SMTP; a failure appears as **The setup email could not be sent**. It does not revert the access flag. It can also send a fresh reset link for an existing password.

Revoke or deactivate

1. Review future meetings and cancel/reassign them separately where necessary.
2. Find the person and select **Edit employee**.
3. Clear Staff to keep an active employee without management powers, or clear Active to prevent normal account access.
4. Save and check row status.

An access change invalidates existing login sessions and pending sign-in challenges. Password changes invalidate existing sessions as well. Records/bookings/audit history remain; account deactivation does not cancel bookings automatically. You cannot remove your own staff role or deactivate your own account through People. There is no separate administrator-only permission tier or grant approval chain.

11. Generate reports and export Excel

1. Open **Reports**.
2. Choose Today, This week, This month, This quarter or This year, or set From/To.
3. Press **Show report**. End points are inclusive Nepal meeting start dates; preset ends are today.
4. Read total booking records, room usage, department and organizer tables.
5. Select **Download Excel** for the same range.

Custom end minus start may span at most 366 days. Reversed/excessive ranges produce a message and today fallback on the page; invalid Excel requests are rejected. Default is current month's first day through today.

Interpret the numbers

- Booking counts include all statuses, such as Pending, Rejected, Cancelled and No show, and use meeting start date. Closures are not counted as meetings.
- Department/organizer counts include those same records. Blank department is Unspecified.
- Room rows include inactive rooms; most booked rooms appear first.
- Utilization counts checked-in/completed **scheduled minutes** clipped to current working hours minus current holidays/active closures. It does not count buffers, merely Approved time, or No shows.
- A checked-in meeting contributes its full scheduled interval, even before it finishes. This is not an actual attendance/early-departure measurement.

- Policy, holiday, closure and room activation history are not reconstructed. Historical percentages can change when current policy changes.
- The denominator is the full selected range; future dates add available hours. Use a completed or appropriate period for meaningful comparison.

Example under default rules: one room has 480 bookable minutes on a normal weekday. A checked-in 60-minute meeting contributes 12.5% utilization; a Pending request adds to booking count but not utilized minutes. Closures reduce the denominator by their overlap with working hours. Real percentages depend on current holidays/closures/range.

Excel contains **Bookings**, **Room usage**, **Departments** and **Organizers** sheets, with headers, filters and frozen first row. Booking rows include ID/date/start/end/room/organizer/department/status/title. Notes, full attendee lists, authentication tokens and audit records are not exported. Excel is a report, not a backup.

12. Use audit history

1. Open **Audit log**.
2. Select an action, for example booking_approved, booking_modified, user_access_modified or room_modified.
3. Optionally enter actor-email search.
4. Press **Filter** and inspect timestamp, actor, target type/ID and outcome.
5. Use Reset to clear filters; page through older events.

The UI is read-only and displays 50 events per page. "System" means no actor was attached to that automated/anonymous event. Audit supports investigation of booking decisions, access/password actions, room/rule/holiday changes and selected sign-in/mail failures. It does not log every page view, provide full email bodies or replace IT's service logs. There is no UI audit delete/export button or automatic one-year purge.

13. Email issues and escalation

Booking events are queued independently of saved reservation transactions. Login links, staff codes and setup messages are sent directly during the request, so their failure may appear in audit/logs rather than the Failed emails dashboard count.

For queued booking mail, the worker attempts at most five deliveries, with backoff. Some Failed messages are awaiting automatic retry; exhausted failures need authorized IT retry after repairing the cause. Obsolete messages are superseded when bookings change. Relay acceptance is not proof of final mailbox delivery.

If users report missing mail:

1. Confirm current booking state and correct organizer/attendee addresses.
2. Check dashboard Failed emails; do not conclude delivery succeeded from a zero count alone.
3. Ask IT to inspect SMTP, worker heartbeat and queue using [troubleshooting commands](#).
4. If the meeting has started and attendance is verified, perform Manual check-in **before** its deadline when needed.
5. Do not promise the system will extend the deadline during an email outage.

Record ordinary page/action, approximate Nepal time, booking ID and visible error/request reference if present. Keep passwords, email codes, token-bearing links and .env contents private. Direct HCL/Outlook calendar sync is deferred; missing entries in those calendars are not evidence of failed MBS confirmation delivery.

14. Handover and retention

Front Desk owns review, guest/preparation requests, manual priority arrangements, room records and appropriate access changes. IT owns server/worker/mail health, database credentials, backups/restores, Docker updates, logs/disk monitoring and technical incident diagnosis. Staff should escalate service errors rather than change database rows to bypass workflows.

Keep booking/audit history under the approved company retention policy, including any requirement to retain at least one year. The application currently has no automatic retention purge. Room inactivity, account inactivity and cancelled reservations retain their history. See [deployment](#) and [troubleshooting](#) for maintenance, diagnostics and recovery commands.

5. Office HTTP installation

Source: docs/deployment-http.md

Transgate Tech | Binayak Bhandari

This is the selected installation for `http://mbs.wdn.com.np`, on **Oracle Linux 9.8 x86_64**. It uses `compose.prod.http.yaml`, port 80, and no certificate or private CA. Browsers show **Not secure** because browser traffic is unencrypted. Keep access on the company network/VPN.

Production mode is independent of HTTPS: both production profiles disable the development email inbox and demo seeding, require generated secrets, and use the restricted PostgreSQL application role. Employee email links and staff email codes still require the real SMTP relay.

Before starting

Run the following commands on the Linux server as root. Git, Docker Engine, and the Compose plugin have already been installed and verified. For another server, install those prerequisites first. Use the same repository directory and Compose project on every update to preserve the database volume.

Internal DNS must resolve `mbs.wdn.com.np` to this server's confirmed office interface IP, `192.168.50.222`. Confirm the interface and DNS result:

SH / Display wraps; copy complete commands from the Markdown source.

```
ip -brief address
getent hosts mbs.wdn.com.np
```

Set `HTTP_BIND_IP=192.168.50.222`. The Docker bridge address `172.17.0.1` is not the office interface. The profile requires an explicit bind address, so it will not silently publish port 80 on all interfaces.

Port 80 must be free. Permit it only from the approved office network/VPN using IT's network and firewall policy. Keep SELinux enabled. The HTTP profile labels its application-specific configuration mounts for SELinux; there is no need to disable SELinux or `firewalld`.

The default Docker subnet is `172.29.16.0/28`. If it overlaps an office route or another Docker network, adjust `PROXY_NETWORK_SUBNET`, `PROXY_NETWORK_DYNAMIC_RANGE`, `PROXY_IP`, and `AUTH_TRUSTED_PROXY_CIDRS` together. The fixed proxy address must remain inside the subnet and outside the automatic address range. Only that proxy address may supply trusted client-IP headers. Web and PostgreSQL expose no host ports.

1. Get the HTTP deployment files

For the already cloned server:

SH / Display wraps; copy complete commands from the Markdown source.

```
cd /opt/mbs/MeetingRoomSystemWDN
git switch mbs-prod
git pull --ff-only
```

For a fresh directory instead:

SH / Display wraps; copy complete commands from the Markdown source.

```
mkdir -p /opt/mbs
cd /opt/mbs
git clone --branch mbs-prod https://github.com/BINAYAK016/MeetingRoomSystemWDN.git
cd MeetingRoomSystemWDN
```

Do not run the HTTP and HTTPS profiles simultaneously for this installation. If services already exist, use the update section and take a backup before changing their configuration.

2. Configure the domain, binding, and SMTP

SH / Display wraps; copy complete commands from the Markdown source.

```
dnf install -y python3 nano
python3 deploy/create-env.py --production --http
```

The generator creates independent passwords and a Django secret, writes `.env` with private permissions, and refuses to replace an existing file. If `.env` already exists from the earlier HTTPS preparation, preserve its generated passwords and secret and edit that same file:

SH / Display wraps; copy complete commands from the Markdown source.

```
nano .env
```

Set these values:

INI / Display wraps; copy complete commands from the Markdown source.

```
DJANGO_PRODUCTION=true
DJANGO_HTTPS=false
DJANGO_ALLOWED_HOSTS=mbs.wdn.com.np
DJANGO_PUBLIC_BASE_URL=http://mbs.wdn.com.np
HTTP_BIND_IP=192.168.50.222
EMPLOYEE_EMAIL_DOMAINS=wdn.com.np,transgate.com.np

DJANGO_EMAIL_BACKEND=smtp
DJANGO_FROM_EMAIL=mbs@wdn.com.np
SMTP_HOST=maildc01.wdn.com.np
SMTP_PORT=25
SMTP_USER=
SMTP_PASSWORD=
SMTP_USE_TLS=false
SMTP_USE_SSL=false
```

These SMTP values are the previously working relay configuration. The new server's source IP must also be allowed by the relay. Relay acceptance and actual mailbox delivery are verified below.

In nano, save with **Ctrl+O**, **Enter**, then exit with **Ctrl+X**. Keep all generated `POSTGRES_*` passwords and `DJANGO_SECRET_KEY`. The database administrator and application role must be different users; preserve the existing administrator credentials if the PostgreSQL volume already contains data.

SH / Display wraps; copy complete commands from the Markdown source.

```
chmod 600 .env
docker compose -f compose.prod.http.yaml config --quiet
```

Use `config --quiet` for validation; plain `config` prints expanded configuration including passwords. If SMTP is not ready, use `DJANGO_EMAIL_BACKEND=disabled` temporarily. The app can start, but sign-in email delivery is unavailable; file-email mode is rejected in production.

3. Build, migrate, and start

Stop if a command fails:

SH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml build --pull
docker compose -f compose.prod.http.yaml up -d db
docker compose -f compose.prod.http.yaml run --rm migrate
docker compose -f compose.prod.http.yaml run --rm db_setup
docker compose -f compose.prod.http.yaml up -d --wait --wait-timeout 180 web worker proxy
docker compose -f compose.prod.http.yaml ps
curl --fail --show-error http://mbs.wdn.com.np/healthz/
```

The worker establishes a heartbeat before web and proxy readiness succeed. `migrate` creates the complete schema, default booking policy, and PostgreSQL overlap protection. `db_setup` creates/updates the restricted application role and grants access to current tables/sequences. Run it after every migration. Migrations and backups use database

administrator credentials; application web/worker services use the restricted runtime role.

If internal DNS is not ready yet, check from the server using the configured office IP and correct Host header:

SH / Display wraps; copy complete commands from the Markdown source.

```
curl --fail --show-error -H 'Host: mbs.wdn.com.np' http://192.168.50.222/healthz/
```

Resolve DNS before testing email sign-in links or opening the site to employees.

4. Verify mail and create staff access

This sends one SMTP test email:

SH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec web python manage.py shell -c "from
booking.services.mail_delivery import deliver_mail; print('Accepted by relay:', deliver_mail('MBS server
SMTP test', 'Test email from the deployed MBS server.', ['binayak.bhandari@wdn.com.np']))"
```

Expected result: Accepted by relay: 1. Confirm the email actually arrives before creating staff access. If the relay rejects the sender or connection, ask IT to authorize this server's IP for the approved sender; keep passwords out of terminal output.

SH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec web python manage.py create_staff_account
binayak.bhandari@wdn.com.np
```

Open the emailed password setup link, set a password, then visit <http://mbs.wdn.com.np/staff/sign-in/>. Staff sign-in requires the password plus an emailed code. Repeat the account command for each approved Front Desk/Administrator user. If email setup fails, fix the relay and rerun it for the same address; it does not create duplicate accounts.

Add real rooms through **Staff desk -> Rooms -> Add room**, including an optional supported photo. No sample accounts or demo rooms are loaded. Review business hours, holidays, booking rules, room capacities/facilities and each room's **Requires approval** setting (checked by default). Employee access supports wdn.com.np and transgate.com.np.

5. Verify from an office laptop

Open <http://mbs.wdn.com.np/> using the HTTP prefix. Verify employee email sign-in and logout, staff password/code sign-in, room search, and the booking lifecycle:

- In a checked Requires approval room, employee creates a Pending request and the time slot is held.
- Staff approves the request; organizer receives the confirmation.
- Staff rejects another request with a visible reason; the slot becomes available.
- Employee substantively edits an Approved meeting in a checked room; it returns to Pending.
- Uncheck another room in Staff desk -> Rooms, verify the saved/audited setting, then submit a valid employee booking there; it starts Approved and queues confirmation/check-in immediately.
- Toggle a room containing an existing Pending request: its status stays Pending, and an unchanged edit stays Pending. A substantive valid edit in an unchecked room becomes Approved.
- Organizer cancels an eligible booking; the slot becomes available.
- Simultaneous requests for the same slot produce one reservation and one availability error.
- Check-in link works during the valid window; missed check-in releases the slot after the deadline.
- Three-step booking, attendee headcount, Internal + External guest company, all-occurrence availability and native form fallback work.

- Reports, audit history, recurring dates, holiday rules, closures, and room deactivation work with real office data.
- Optional staff-uploaded room photos display only while authenticated, survive recreation, and restore from the matching media archive.

Check that company laptops can reach the office IP and resolve the domain. Do not expose port 80 to the public internet. Existing HTTPS rehearsal results in `verification-report.md` remain evidence for that profile; verify this actual server's DNS, mail, firewall, and restore procedure before rollout.

Backups and updates

Take a backup using the matching HTTP Compose profile:

SH / Display wraps; copy complete commands from the Markdown source.

```
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
```

The script writes private custom-format database dumps to `deploy/backups/` and checks their archive listings. It does **not** include optional room photos. Web stores those in the named `room_media` volume at `/app/media`; back up/restore a matching photo archive using [Room photo backup and recovery](#). Ordinary container recreation preserves the volume. Encrypt and copy backups to company-approved separate storage with retention; store `.env` separately in approved secret storage. Perform a restore drill into an isolated database rather than the live one. An archive listing alone does not prove restoration. The shared [backup and restore procedure](#) applies, substituting `compose.prod.http.yaml` for each Compose command.

For an update, build the current branch before the outage, then stop writers and take a matching database/media recovery point while PostgreSQL remains running. Execute this from the installed repository; if its checked branch or clean-worktree gate fails, preserve the local changes and investigate before continuing. The subshell stops on any failure:

BASH / Display wraps; copy complete commands from the Markdown source.

```
(
set -eu
cd /opt/mbs/MeetingRoomSystemWDN
mbs() { docker compose -f compose.prod.http.yaml "$@"; }

test "$(git branch --show-current)" = "mbs-prod"
if [ -n "$(git status --porcelain)" ]; then
    echo "Local code changes found. Stop here and inspect git status --short."
    exit 1
fi

git pull --ff-only origin mbs-prod
mbs config --quiet
mbs build web
mbs up -d --wait --wait-timeout 90 db
mbs run --rm --no-deps -T --entrypoint sh web -c 'test -w /app/media/rooms/photos'

mbs stop proxy web worker
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
umask 077
mbs_media_backup="deploy/backups/room_media_$(date -u +%Y%m%dT%H%M%S).tar.gz"
mbs run --rm --no-deps -T --entrypoint sh web -c 'tar -C /app/media -czf - .' > "$mbs_media_backup"
tar -tzf "$mbs_media_backup" > /dev/null

mbs run --rm --no-deps migrate
mbs run --rm --no-deps db_setup
mbs up -d --no-deps --force-recreate --wait --wait-timeout 180 web worker proxy
mbs exec -T web python manage.py check
mbs exec -T web python manage.py showmigrations booking
mbs ps
curl --fail --show-error --resolve mbs.wdn.com.np:80:192.168.50.222 http://mbs.wdn.com.np/healthz/
git log -1 --oneline
)
```

Keep the printed database dump, media archive and release commit together as one recovery point and copy them to approved separate storage. Archive listing validates structure, not a successful restore. If a command fails after services were stopped, leave the writers stopped, inspect the failure and use the recovery runbook before resuming; do not delete volumes or start old code against a newer schema.

Preserve `.env`, the database credentials, Django secret, PostgreSQL volume and `room_media` photo volume. For an installation with photos, stop writers and take the paired database/photo backup before updating. This release requires committed migrations `0009_mixed_meeting_type`, `0010_room_photo` and `0011_room_requires_approval`; they preserve existing records, and no room reseeding is required. Migration `0011` checks Requires approval for existing rooms and does not change saved booking statuses. Verify all applied using `docker compose -f compose.prod.http.yaml exec web python manage.py showmigrations booking`. Migration `0008` preserves legacy confirmed meetings as approved; reversing it cancels pending/rejected records, so review rollback/restore separately. Stop web/worker writers before any migration.

Stop containers while preserving the database:

SH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml down
```

Never add `--volumes` unless intentionally deleting production data.

Troubleshooting and monitoring

SH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml ps
```

```
docker compose -f compose.prod.http.yaml logs --tail=100 web worker proxy db
```

`/livez/` reports the web process. `/healthz/` also requires PostgreSQL and a recent worker heartbeat. Docker restart policies restart exited processes; IT must monitor unhealthy containers, failed mail, backups, and disk usage.

- Connection refused: check `HTTP_BIND_IP`, port conflicts, container health, and the office firewall rule.
- Permission denied on a bind mount with SELinux enforcing: check the HTTP profile's `:z` labels, file ownership, and audit logs; keep SELinux enforcing.
- Sign-in links point to HTTPS: correct `.env` to `DJANGO_PUBLIC_BASE_URL=http://mbs.wdn.com.np`, recreate web/worker, then request a new email.
- Browser automatically changes HTTP to HTTPS: an earlier HTTPS test may have stored HSTS for this hostname. Remove that saved host policy on the affected test device or have IT arrange its expiry; the HTTP proxy does not send HSTS.
- Session-check/CSRF page: load a fresh form at the same HTTP hostname, confirm cookies are enabled, and use the latest image. The response must use `Referrer-Policy: same-origin`; `no-referrer` makes native browser POSTs send `Origin null`. After reproducing, `docker compose -f compose.prod.http.yaml logs --since 5m --tail=80 web` records a safe category (`missing_session`, `missing_token`, `invalid_token`, `origin_mismatch`, `referer_rejected`, or other) and request reference. Do not disable CSRF or trust null origins.
- Authentication email unavailable: review the relay host/port/sender and its authorization for this server's IP. SMTP-disabled mode intentionally denies mail-based sign-in.
- Database permission errors after an update: rerun committed migrations and `db_setup`; do not generate migrations on the server.

HTTP deliberately omits secure-cookie/HTTPS redirect/HSTS settings, so check `--deploy` can report transport warnings. Production-mode restrictions, CSRF checks, staff MFA, booking permissions, and database constraints remain active.

6. Operations and recovery

Source: docs/operations-runbook.md

Transgate Tech | Binayak Bhandari Applies to: the mbs-prod application and the selected private-network HTTP installation. **Reviewed:** 7 October 2026.

This handbook is for the person operating the Linux server. For screen-by-screen business work, read the [employee guide](#) and [staff guide](#). For symptom-based diagnosis, use [troubleshooting](#). The [HTTP deployment guide](#) covers the initial installation. The [HTTPS deployment guide](#) remains available for a future transport change.

Contents

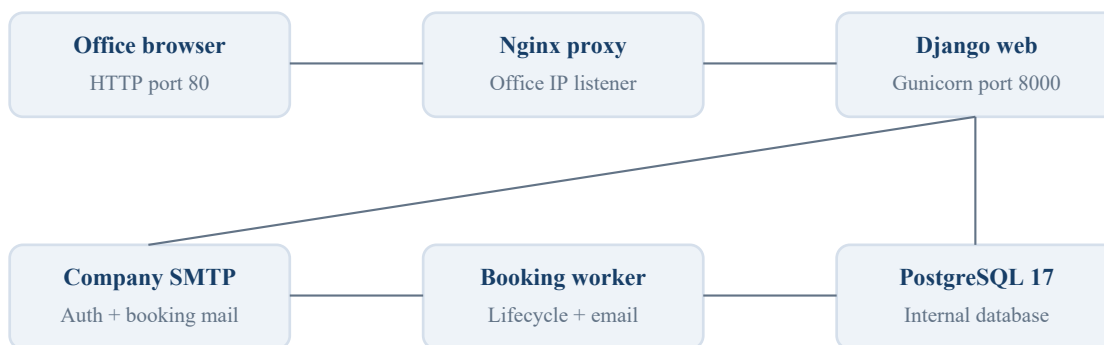
1. [Installation identity and architecture](#)
2. [Open an operator terminal](#)
3. [Configuration reference and safe checks](#)
4. [Starting, stopping, restarting, and updating](#)
5. [Staff access and password recovery](#)
6. [Service health, logs, and read-only database checks](#)
7. [Worker and notification operations](#)
8. [Backups, scheduling, and monitoring](#)
9. [Restore drill into a separate database](#)
10. [Approved live recovery](#)
11. [Daily, weekly, and release checklists](#)
12. [Incident handover and operational limits](#)
13. [Windows laptop differences](#)

1. Installation identity and architecture

Item	This office installation
Host OS	Oracle Linux 9.8, x86_64
Repository	https://github.com/BINAYAK016/MeetingRoomSystemWDN.git
Deployment branch	mbs-prod
Server directory	/opt/mbs/MeetingRoomSystemWDN
Compose profile	compose.prod.http.yaml
Public origin	http://mbs.wdn.com.np
Office IPv4 / published listener	192.168.50.222:80
Approved employee domains	wdn.com.np, transgate.com.np
SMTP relay	maildc01.wdn.com.np:25

Item	This office installation
Approved sender	mbs@wdn.com.np
Selected mail security	No relay username/password; TLS and SSL false
App time zone	Asia/Kathmandu; database timestamps are timezone-aware

The browser connects to Nginx on the office interface. Nginx forwards to Gunicorn/Django on container port 8000. Django and the background worker connect to PostgreSQL on container port 5432. Only the proxy publishes a host port; neither the web process nor PostgreSQL is directly published to the office network. HTTP uses no certificate mount, HTTPS redirect, HSTS, or Secure cookie flag. Browsers display **Not secure**. Keep this installation reachable through the approved office network/VPN.



Simplified relationships/components. The linked Markdown source includes the full Mermaid diagram.

Services and responsibilities

Service	Responsibility	Normal lifetime / health
proxy	HTTP listener, reverse proxy, safe request timing/status logs	Continuous; probes web /livez/
web	Employee and staff screens, authentication, booking transactions, reports, static files	Continuous; probes /healthz/
worker	Pending-request expiry, no-show release, completion, reminder scheduling, notification delivery	Continuous; probes recent worker heartbeat
db	Durable data, constraints, sessions, audit, mail queue	Continuous; pg_isready
migrate	Applies committed Django migrations with database administrator rights	One-off command; tools profile
db_setup	Creates/updates application role and grants current-schema privileges	One-off command; tools profile

Web and worker run as the image's non-root appuser (UID 1000), with read-only root filesystems, writable /tmp, dropped capabilities, and no-new-privileges. Web additionally has the writable persistent room_media volume at /app/media; uploads do not require write access to the application root. Static files are collected during the image build and served by WhiteNoise. Editing source on the server does not change an already-running image.

The production database network is internal. Web/worker also join the frontend network for SMTP access. Default frontend subnet 172.29.16.0/28 reserves 172.29.16.14 for the proxy outside the dynamic allocation range 172.29.16.0/29. Only 172.29.16.14/32 is trusted for client-IP headers. Review all four network settings together if a company route overlaps this range.

Database ownership

POSTGRES_USER/POSTGRES_PASSWORD are the PostgreSQL bootstrap administrator credentials. The migration, setup, backup, and restore tools use them. Web/worker receive POSTGRES_APP_USER and POSTGRES_APP_PASSWORD mapped to their connection variables instead.

The runtime role has login plus current-table SELECT/INSERT/UPDATE/DELETE and sequence USAGE/SELECT. It is NOSUPERUSER, NOCREATEDB, NOCREATEROLE, NOINHERIT. db_setup revokes UPDATE, DELETE, and TRUNCATE on audit_events from that role. Audit events remain insertable and readable. The administrator retains administrative power; this is not a cryptographic audit ledger.

Run db_setup after migrations and restores. It grants access to the current schema; it does not establish default privileges for future tables. PostgreSQL usernames/passwords in an initialized volume do not change merely because .env changes.

2. Open an operator terminal

The commands below are **Bash commands on the Linux server**, normally as the installation's existing root operator. Run this preamble in each new terminal:

BASH / Display wraps; copy complete commands from the Markdown source.

```
cd /opt/mbs/MeetingRoomSystemWDN
mbs() { docker compose -f compose.prod.http.yaml "$@"; }
```

mbs ps is exactly docker compose -f compose.prod.http.yaml ps. The function exists only in the current shell. Commands involving /etc, systemctl, and backup scheduling require root privileges.

Confirm the installation before changing it:

BASH / Display wraps; copy complete commands from the Markdown source.

```
pwd
git branch --show-current
git rev-parse --short HEAD
git status --short
docker --version
docker compose version
mbs ls
mbs ps -a
```

The expected branch is mbs-prod. Keep the same repository directory and Compose project identity. The existing default project is commonly meetingroomssystemwdn, but check mbs ls and container labels rather than assuming. A changed directory, -p, or COMPOSE_PROJECT_NAME can select a different project and therefore a different database volume. If a custom project name was used at installation, use that same name for every operation and backup; do not invent a new one for a routine restart.

Do not run local, production HTTP, and production HTTPS profiles against the same installation simultaneously. The production profiles use the same application image tag and service names.

3. Configuration reference and safe checks

Current values and precedence

`.env` is read by Compose to interpolate its service configuration. It is not automatically sourced into Bash. Existing shell environment values can override interpolation from `.env`; use a clean operator shell if unexpected values persist. In particular, an exported `COMPOSE_PROJECT_NAME` can change the selected installation. See [Docker's environment-variable reference](#).

Keep exactly one occurrence of every key. The earlier duplicate `DJANGO_PUBLIC_BASE_URL=https://...` and `DJANGO_HTTPS=true` lines must not remain in an HTTP installation. The production HTTP service definitions force `DJANGO_PRODUCTION=true` and `DJANGO_HTTPS=false`; the public URL must still be HTTP or the application fails its configuration checks. `DJANGO_ALLOWED_HOSTS` must be the single hostname used by this proxy, without a scheme, path, or wildcard.

Variable	Meaning / selected value
POSTGRES_DB	Live database name; generated template uses <code>meeting_rooms</code>
POSTGRES_USER	Existing bootstrap administrator role; preserve for initialized volumes
POSTGRES_PASSWORD	Administrator secret; preserve actual database credential until deliberately rotated
POSTGRES_APP_USER	Distinct runtime role; template uses <code>meeting_rooms_app</code>
POSTGRES_APP_PASSWORD	Runtime secret; applied by <code>db_setup</code> , then consumed by recreated web/worker
DJANGO_SECRET_KEY	Independent generated secret, at least 50 characters in production; preserve across routine updates
DJANGO_ALLOWED_HOSTS	<code>mbs.wdn.com.np</code> ; exact hostname
DJANGO_PUBLIC_BASE_URL	<code>http://mbs.wdn.com.np</code> ; full origin, no trailing path/query/credentials
DJANGO_PRODUCTION	<code>true</code> ; production safeguards are independent of transport
DJANGO_HTTPS	<code>false</code> in selected profile
HTTP_BIND_IP	<code>192.168.50.222</code> ; explicit host interface address required
EMPLOYEE_EMAIL_DOMAINS	<code>wdn.com.np,transgate.com.np</code> ; production Compose explicitly fixes these two domains
DJANGO_EMAIL_BACKEND	<code>smtp</code> for live login; disabled starts without mail; file is prohibited in production
DJANGO_FROM_EMAIL	<code>mbs@wdn.com.np</code> ; approved sender
SMTP_HOST	<code>maildc01.wdn.com.np</code>
SMTP_PORT	<code>25</code> ; valid range 1-65535
SMTP_USER / SMTP_PASSWORD	Empty for this IP-authorized relay
SMTP_USE_TLS / SMTP_USE_SSL	Both <code>false</code> here; never enable both together
PROXY_NETWORK_SUBNET	<code>172.29.16.0/28</code> unless deliberately changed with IT
PROXY_NETWORK_DYNAMIC_RANGE	<code>172.29.16.0/29</code>

Variable	Meaning / selected value
PROXY_IP	172.29.16.14; inside subnet, outside dynamic range
AUTH_TRUSTED_PROXY_CIDRS	172.29.16.14/32; trust the proxy only

Production Compose sets `POSTGRES_HOST=db` and `POSTGRES_PORT=5432` internally; a host-shell `.env` override does not alter those service definitions. `DEBUG=False`, SMTP timeout 10 seconds, the 120-second worker health threshold, and authentication lifetimes are application constants, not supported `.env` tuning knobs. Adding `DEBUG=True` to `.env` will not enable Django debug mode.

Changing production employee domains requires a reviewed change to the production Compose definition as well as appropriate application configuration. Editing only `EMPLOYEE_EMAIL_DOMAINS` in `.env` changes the local profile, not this production profile.

The backup script accepts `COMPOSE_FILE`, `ENV_FILE`, `BACKUP_DIR`, and normal Compose project selection. It defaults to the HTTPS profile, so HTTP backups must explicitly set `COMPOSE_FILE=compose.prod.http.yaml`.

Safe validation

BASH / Display wraps; copy complete commands from the Markdown source.

```
chmod 600 .env
mbs config --quiet
```

No output from `config --quiet` means the Compose configuration resolved. It does not prove that the database credentials authenticate, mail reaches a mailbox, or DNS works.

Do not paste `.env`, plain expanded `docker compose config`, `docker inspect` environment arrays, passwords, sign-in links/codes, mail bodies, or database dumps into a support chat. To check duplicate keys without printing their values:

BASH / Display wraps; copy complete commands from the Markdown source.

```
python3 - <<'PY'
from collections import Counter
from pathlib import Path
keys = []
for raw in Path('.env').read_text().splitlines():
    line = raw.strip()
    if line and not line.startswith('#') and '=' in line:
        keys.append(line.split('=', 1)[0].strip())
duplicates = sorted(key for key, count in Counter(keys).items() if count > 1)
print('Duplicate keys:', ', '.join(duplicates) if duplicates else 'none')
raise SystemExit(bool(duplicates))
PY
```

To print **effective nonsecret** Django settings from the running image:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec web python manage.py shell -c "from django.conf import settings; keys=('PRODUCTION_ENABLED', 'HTTP
S_ENABLED', 'PUBLIC_BASE_URL', 'ALLOWED_HOSTS', 'EMPLOYEE_EMAIL_DOMAINS', 'MAIL_MODE', 'EMAIL_READY', 'SESSION_C
OOKIE_SECURE', 'CSRF_TRUSTED_ORIGINS', 'SECURE_REFERRER_POLICY'); print({k:getattr(settings,k, 'MISSING') for
k in keys})"
```

Expected transport values are production true, HTTPS false, HTTP public origin, session Secure false, trusted origin `http://mbs.wdn.com.np`, and referrer policy same-origin. CSRF tokens live in the server-side session; a separate `csrftoken` cookie is not expected.

Credentials and rotation

The generator refuses to overwrite `.env`; use it only for a new installation:

BASH / Display wraps; copy complete commands from the Markdown source.

```
python3 deploy/create-env.py --production --http
```

For an existing installation, edit the preserved file with `nano .env`. If a secret was disclosed, plan its rotation with IT. Rotating the runtime password requires applying it with `db_setup` and recreating `web/worker`; avoid a window in which old workers use the old password. Rotating the administrator password requires changing the actual PostgreSQL role and its stored configuration together. Do not assume an `up` command changes the role password. Changing the Django secret invalidates existing authentication signatures; arrange a sign-in interruption and request fresh links/codes afterward.

4. Starting, stopping, restarting, and updating

Start in the background

Once schema and runtime grants already exist:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs up -d --wait --wait-timeout 180 db web worker proxy
mbs ps
curl --fail --show-error http://mbs.wdn.com.np/healthz/
```

`-d` runs detached. If you accidentally use attached `up`, use the displayed **d Detach** shortcut when available. `Ctrl+C` in an attached `up` stops its services; run the detached command again to start them.

Stop while retaining all data

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs stop proxy web worker
```

This stops the application and leaves PostgreSQL running for checks/backups. To stop PostgreSQL as well:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs stop db
```

To remove the selected project's containers/networks while retaining its named volumes:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs down
```

Do not add `--volumes` or use `docker volume rm`, `docker system prune --volumes`, or a global `prune` as a repair. They can delete the durable database or affect other applications.

Restart versus recreate

Situation	Operation
Same image/env; exited or stalled application process	Inspect logs first, then <code>mbs restart web worker</code>
<code>.env</code> changed	Recreate the affected services; <code>restart</code> retains old environment
Python, templates, CSS, or dependencies changed	Build a new image, then recreate <code>web/worker</code>
Nginx template changed	Recreate proxy; its startup renders the template
Database migration added	Stop writers, run <code>migrate</code> , then <code>db_setup</code> , before reopening
Docker daemon not running	Check <code>systemctl status docker</code> ; start the host service through approved host operations

After a mail/origin setting edit, with no schema change:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs config --quiet
mbs up -d --no-deps --force-recreate --wait --wait-timeout 180 web worker proxy
```

Do not use this shortcut for a runtime database password rotation: first stop web/worker, apply db_setup with the revised password, then recreate them. Existing authenticated sessions persist through ordinary process recreation because sessions are stored in PostgreSQL.

Change a conflicting Docker subnet

This is a planned network change with an outage. Confirm the new subnet/range/proxy address with IT and identify all writers. Preserve the project name, .env credentials and named database volume. Take a backup first; stop if it fails.

BASH / Display wraps; copy complete commands from the Markdown source.

```
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
mbs stop proxy web worker
mbs down
nano .env
```

Change PROXY_NETWORK_SUBNET, PROXY_NETWORK_DYNAMIC_RANGE, PROXY_IP and AUTH_TRUSTED_PROXY_CIDRS together. The proxy must be inside the subnet, outside the dynamic range; trust only its address. down removes the project's old containers/networks, retaining named volumes. Never add --volumes. Start with the same directory/project:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs config --quiet
mbs up -d --wait --wait-timeout 180 db web worker proxy
mbs ps
curl --fail --show-error http://mbs.wdn.com.np/healthz/
```

Verify employee access, effective trusted proxy configuration and SMTP reachability. A subnet change does not require new business records or a database reset.

Full application update

Use a maintenance window. Stop if any command fails; do not continue startup after a failed migration or role setup.

BASH / Display wraps; copy complete commands from the Markdown source.

```
git status --short
git rev-parse HEAD
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
mbs stop proxy web worker
git pull --ff-only
mbs build --pull
mbs run --rm migrate
mbs run --rm db_setup
mbs up -d --wait --wait-timeout 180 db web worker proxy
mbs ps
curl --fail --show-error http://mbs.wdn.com.np/healthz/
```

If room photos exist, take a paired database/photo backup from [Room photo backup and recovery](#) before the update; the database-only command above does not include files. Current releases add 0009_mixed_meeting_type, 0010_room_photo and 0011_room_requires_approval; apply the complete chain before starting new web code, then reapply db_setup. They preserve existing records and do not import/reset room inventory. Migration 0011 checks Requires approval for all existing rooms; room policy changes do not rewrite booking statuses. No additional environment variable or service is required. Web now mounts the named room_media volume, initialized for application UID/GID 1000.

Record the previous commit, new commit, backup filenames, migration results, and acceptance checks. git pull --ff-only deliberately stops if local commits/history conflict; review instead of forcing Git history. Preserve .env, secret storage, and the PostgreSQL volume.

Migrations must come from reviewed source control. Do not run `makemigrations` on production as a response to a missing table. Migrations use the administrator tool service, not the restricted web role. Migration 0008 preserves legacy confirmed meetings as approved; reversing it cancels pending/rejected records. A code rollback is not automatically a database rollback. Restore a tested compatible database and release together if rollback requires schema reversal.

For the CSRF/referrer fix, a code-only deployment after the documented release is sufficient: build web/worker, recreate them and proxy, then load a fresh form with Ctrl+F5. Do not disable CSRF or add `null` to trusted origins.

An orphan `mail_init` warning can come from the old local profile; it does not itself mean database failure. Inspect that container's labels and state before removing a named obsolete container. Do not apply a broad orphan-cleanup flag without verifying which project services it affects.

5. Staff access and password recovery

The supported staff UI is **Staff desk**, `/staff/`, not an exposed Django `/admin/` site. Front Desk and Administrators have identical permissions. Employee email-link authentication does not complete staff authentication, even for a user marked staff.

Initial setup from the server

The command below changes access and **sends a password setup email**. Use only for approved staff addresses:

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Examples only: substitute the actual individually approved staff addresses.
mbs exec web python manage.py create_staff_account person@wdn.com.np
mbs exec web python manage.py create_staff_account colleague@transgate.com.np
```

The command normalizes address case, creates or reuses the account, makes it active/staff, records `staff_bootstrap`, and sends a one-time password setup/reset link. It can therefore reactivate an existing inactive account; do not run it for a departed user. Re-running does not create a duplicate account. It does not replace the password until the recipient completes setup.

Staff enabled, but setup email failed: ... means access was granted but delivery failed. Repair mail, then rerun for the same approved user or use the existing staff UI's **Email setup link** action. Relay acceptance is not proof of mailbox arrival.

Normal staff sign-in

1. Open `http://mbs.wdn.com.np/staff/sign-in/`.
2. Enter company email and password.
3. Enter the newest eight-digit code from the company mailbox in the same browser session.
4. Staff verification lasts eight hours. A fresh login requires password and email code again.

Password setup links last one hour. The setup screen requires matching passwords of at least 12 characters and Django password validation. Codes last 10 minutes and allow at most five attempts. Starting another successful password challenge invalidates the previous unused challenge; use the newest email. Password attempts are throttled (five per account and 30 per observed client IP per 15-minute window). Do not erase throttle rows to bypass the protection.

Staff UI access management

Open **Staff desk** -> **People** (`/staff/users/`). Search for the address, edit names/department, enable/disable active status, and grant/revoke Front Desk/Administrator access. Save changes. Use **Email setup link** for an approved active staff user's forgotten password.

Access changes invalidate sessions and pending sign-in challenges. The UI prevents a staff member from removing their own staff access. User/booking history is retained; deactivation does not cancel their existing meetings. Review

affected meetings separately through the supported booking screens. Password setup/reset also revokes staff challenges and changes the session authentication version.

If all staff are locked out, an authorized server operator can bootstrap a designated approved staff address using the command above. Do not edit users/password hashes directly in SQL or enable a separate password-only admin bypass.

6. Service health, logs, and read-only database checks

Health meaning

Check	Healthy result	What it proves
/livez/	HTTP 200, ok	Web process can return a simple response
/healthz/	HTTP 200, JSON status/database/worker = ok	PostgreSQL query works and worker success heartbeat is within 120 seconds
check_worker_health	ok, exit 0	A recent successful reconciliation exists
DB health	Healthy from pg_isready	Server is accepting connections; not a full schema or role-permission check

Readiness email=configured means settings supply the relay host/sender. It does **not** connect to SMTP or prove delivery. Readiness can be healthy with email=unavailable if mail is intentionally disabled. Proxy health uses liveness, so a green proxy alone does not prove the database/worker are healthy. Docker restart policies restart exited processes; an unhealthy-but-running process needs alerting and diagnosis.

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs ps -a
mbs exec web python manage.py check
mbs exec worker python manage.py check_worker_health
curl --fail --show-error http://mbs.wdn.com.np/livez/
curl --fail --show-error http://mbs.wdn.com.np/healthz/
curl --fail --show-error -H 'Host: mbs.wdn.com.np' http://192.168.50.222/healthz/
```

The IP/Host-header check distinguishes DNS problems from application availability. It does not repair employee DNS resolution or email links. Check office-facing resolution and listeners:

BASH / Display wraps; copy complete commands from the Markdown source.

```
getent hosts mbs.wdn.com.np
ip -brief address
ss -ltnp 'sport = :80'
df -h
df -i
free -h
docker stats --no-stream
```

Logs

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs logs --since 15m --tail 200 web worker proxy db
mbs logs --follow --since 2m --tail 50 web worker proxy
```

Ctrl+C stops log following; it does not stop the containers when using logs --follow. Narrow the period around the reported incident, then expand only if necessary. Root's host-level diagnostics are:

BASH / Display wraps; copy complete commands from the Markdown source.

```
systemctl status docker --no-pager
journalctl -u docker --since '30 minutes ago' --no-pager
journalctl -k --since '30 minutes ago' --no-pager
getenforce
```

Keep SELinux enabled. Inspect relevant denial records/labels through IT's troubleshooting process; do not disable SELinux or firewalld as a blanket fix.

Application logs record exception type, source locations and, where available, SQLSTATE; they avoid exception values, request payloads, passwords, and token URLs. CSRF rejections record a safe category and request reference. Nginx logs time, method, status, byte count, duration, upstream status, and request ID without URI/query values. Token-bearing sign-in/check-in/password-setup routes deliberately suppress Nginx access logs. Absence of a token URL in proxy logs is expected.

A worker can be healthy and quiet: it prints `released=...` `completed=...` `reminders=...` `sent=...` only when there is activity. `released` counts no-show releases; `pending-request` expiry also runs but is not included in that counter. Use database status and audit events to investigate pending expiry.

Store incident excerpts privately:

BASH / Display wraps; copy complete commands from the Markdown source.

```
umask 077
MBS_INCIDENT_DIR="/var/log/mbs-incident-$(date -u +%Y%m%dT%H%M%S)"
mkdir -m 700 "$MBS_INCIDENT_DIR"
mbs logs --no-color --since 30m --tail 500 web worker proxy db > "$MBS_INCIDENT_DIR/containers.log" 2>&1
mbs ps -a > "$MBS_INCIDENT_DIR/services.txt"
git rev-parse HEAD > "$MBS_INCIDENT_DIR/revision.txt"
```

Review any host/third-party log excerpt before sharing; PostgreSQL/admin or operating-system messages can contain more detail than the application's sanitized logs. Do not use `set -x` during credential-bearing operations.

Committed migrations

These checks do not apply schema changes:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec web python manage.py showmigrations booking
mbs exec web python manage.py migrate --plan
```

All committed booking migrations, including `0008_booking_approval`, `0009_mixed_meeting_type`, `0010_room_photo` and `0011_room_requires_approval`, should be marked applied on a deployed current release. If changes are needed, follow the maintenance workflow using `mbs run --rm migrate` and `mbs run --rm db_setup`.

Runtime identity and privileges

This prints only role capabilities and a few privileges, not passwords:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec web python manage.py shell -c "from django.db import connection; c=connection.cursor();
c.execute('SELECT current_database(), current_user'); print('runtime:', c.fetchone()); c.execute('SELECT
rolname, rolsuper, rolcreatedb, rolcreatorole, rolinherit FROM pg_roles WHERE rolname=current_user');
print('role:', c.fetchone()); c.execute(\"SELECT
has_table_privilege(current_user, 'public.reservations', 'SELECT'),
has_table_privilege(current_user, 'public.reservations', 'INSERT'),
has_table_privilege(current_user, 'public.audit_events', 'INSERT'),
has_table_privilege(current_user, 'public.audit_events', 'UPDATE'),
has_table_privilege(current_user, 'public.audit_events', 'DELETE')\"); print('reservation SELECT/INSERT,
audit INSERT/UPDATE/DELETE:', c.fetchone())"
```

Expected runtime identity is the configured application role; all four role capability flags should be false. Expected privilege tuple is `(True, True, True, False, False)`. If schema permissions are missing after migration, inspect the successful migration/setup results and rerun the supported `db_setup` during maintenance.

Read-only database operational snapshot

The following uses database administrator access **only for SELECT diagnostics**, establishes a read-only transaction, and does not print meeting descriptions, email-token hashes, notification bodies, or passwords:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec -T db sh -c 'PGPASSWORD="$POSTGRES_PASSWORD" psql --no-psqlrc -v ON_ERROR_STOP=1 -U
"$POSTGRES_USER" -d "$POSTGRES_DB" <<'SQL'
BEGIN READ ONLY;
SELECT current_database() AS database, current_user AS operator_role, now() AS server_time;
SELECT id, last_success_at, last_error_at,
       round(extract(epoch FROM now() - last_success_at)) AS success_age_seconds
FROM worker_heartbeat;
SELECT status, COUNT(*) AS jobs, MIN(next_attempt_at) AS earliest_next_attempt
FROM notifications GROUP BY status ORDER BY status;
SELECT id, event_type, status, attempts, next_attempt_at, lease_expires_at, last_error
FROM notifications
WHERE status IN ('failed','sending')
ORDER BY next_attempt_at, id LIMIT 30;
SELECT status, COUNT(*) AS bookings
FROM reservations WHERE kind='booking' GROUP BY status ORDER BY status;
SELECT COUNT(*) AS active_rooms FROM rooms WHERE is_active;
SELECT COUNT(*) AS active_staff FROM users WHERE is_active AND is_staff;
SELECT opens_at, closes_at, minimum_minutes, maximum_minutes, slot_minutes,
       gap_minutes, advance_days, check_in_minutes FROM booking_policy WHERE id=1;
SELECT action, outcome, COUNT(*) AS recent_events
FROM audit_events WHERE created_at >= now() - interval '24 hours'
GROUP BY action, outcome ORDER BY action, outcome;
COMMIT;
SQL
```

`last_error_at` is the most recent failed cycle timestamp; a later successful cycle can mean the incident recovered. Notification `last_error` holds the exception class, not the SMTP response/body. Look at the matching safe worker logs and ask IT for relay diagnostics when a policy rejection needs its exact server-side reason.

Do not repair business state with SQL UPDATE/DELETE statements. Approval, cancellation, access changes, room changes, and retry decisions have invariants and audit behavior; use the supported UI/services/management commands.

7. Worker and notification operations

Timing and state

The worker loop runs about once per second, closes stale DB connections, and reconciles scheduling approximately every 30 seconds. A reconciliation:

- Cancels requests still pending at or after meeting start, with reason Approval window expired before the meeting started.
- Changes approved meetings whose check-in deadline passed to **No show**, releases their occupied slot, and records the cancellation reason.
- Marks checked-in meetings completed after their end.
- Queues one-hour reminders for eligible approved upcoming bookings.
- Updates singleton heartbeat `worker_heartbeat.id=1`.

Then it attempts at most one queued message per loop, with SMTP timeout 10 seconds. A slow send can delay the next reconciliation by up to a timeout; deadlines are enforced again by the check-in action, so stale processing does not extend the valid check-in window. Batch limits mean a large overdue backlog can take several reconciliation cycles.

Notification states are pending, sending, sent, failed, and skipped (superseded). Sending uses an exclusive row claim plus a five-minute lease. The lease supports recovery after a worker interruption. Each send increments attempts; maximum is five. After failures on attempts one through four, automatic retries wait 2, 4, 8, and 16 minutes. A failure on attempt five is terminal for automatic delivery and remains for operator review; its recorded next-attempt timestamp is 32 minutes later, but there is no automatic sixth send. There is no guarantee of exactly-once delivery if a process dies after the relay accepts mail but before the DB records success.

For queued booking mail, reservation_revision and current booking state prevent stale reminders/confirmations from being sent after edits/cancellation. skipped is usually deliberate supersession, not a mail outage. A reminder includes the organizer's check-in link only for the organizer; attendees receive meeting timing and are told the organizer handles check-in.

Employee login links, staff codes, and password setup messages are sent synchronously, outside the booking notification queue. Retrying queued booking mail does not recreate an expired login challenge. Request a fresh login/setup message through its supported flow after fixing SMTP.

Disabled mail

DJANGO_EMAIL_BACKEND=disabled, or SMTP mode missing a relay host/sender, leaves delivery unavailable. The worker still reconciles booking lifecycle/heartbeat; pending booking notifications are not consumed by the disabled backend. Employee/staff email sign-in cannot succeed, and staff setup mail fails. Do not switch to production file mail or treat a healthy heartbeat as mail verification.

Send an intentional relay test

The following **sends one real test email** to an authorized recipient entered at the prompt. Use when verifying a planned mail repair:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec web python manage.py shell -c "from booking.services.mail_delivery import deliver_mail;
recipient=input('Authorized test recipient: ').strip(); print('Accepted by relay:', deliver_mail('MBS
operations test', 'Test message from the MBS server.', [recipient]))"
```

Expected output is Accepted by relay: 1. Confirm mailbox arrival, sender identity, and any quarantine/spam routing with IT. The new server must be allowed by the relay based on its actual SMTP source IP; its HTTP listener address is not necessarily the relay-visible source in a routed/NAT network.

Retry eligible failed booking mail

First correct mail settings, recreate web/worker if settings changed, and verify delivery. Inspect failed IDs with the read-only snapshot. To retry one identified failed notification:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec web python manage.py retry_failed_mail --id 123
```

Replace 123 with the actual numeric notification ID. To review all failed jobs after a general mail outage:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec web python manage.py retry_failed_mail --all
```

One of --id or --all is required. The command checks booking status, revision, and useful lifetime; eligible jobs become pending with attempts reset, obsolete jobs become skipped, and audit events are appended. Output is requeued=N superseded=N. It does not instantly deliver mail; the normal worker sends the requeued jobs. Do not repeatedly requeue while the relay remains broken.

Avoid running an extra run_booking_worker --once as a read-only test. It can send real mail, expire requests/no-shows, and complete bookings. Normal health checks use check_worker_health and do not execute business jobs.

8. Backups, scheduling, and monitoring

What is backed up

deploy/backup.sh dumps the selected database in PostgreSQL custom format, verifies pg_restore --list, and writes a private file under deploy/backups/. It uses a unique .partial file, removes it if the operation fails, sets directory mode 700/file mode 600, then atomically renames a verified archive to .dump. Database dumps contain company data and authentication/session records; protect them like production data.

A database dump does not include uploaded room photos, PostgreSQL cluster roles/passwords, .env, Git source, Docker images, host DNS/firewall settings, or future certificate files. Photo files require a separate matching room_media archive as described below. Keep .env separately in company-approved secret storage; keep release commit identifiers and installation instructions with recovery records. Store original administrator credentials for an existing volume and current runtime credentials for role recreation. db_setup can recreate the application role in a recovered cluster.

Create and verify a backup

BASH / Display wraps; copy complete commands from the Markdown source.

```
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
ls -lh deploy/backups/*.dump
```

Record the exact Backup written: filename. Choose that file explicitly; do not automatically restore whichever filename sorts last. Example selection below must be changed to the real existing filename:

BASH / Display wraps; copy complete commands from the Markdown source.

```
MBS_BACKUP='/opt/mbs/MeetingRoomSystemWDN/deploy/backups/REPLACE_WITH_ACTUAL_FILENAME.dump'
test -f "$MBS_BACKUP"
test -s "$MBS_BACKUP"
mbs exec -T db pg_restore --list < "$MBS_BACKUP" > /dev/null
sha256sum "$MBS_BACKUP"
```

Stop if any check fails. Archive-list success is structural validation, not proof that the archive restores. Perform the separate-database drill below.

pg_dump obtains a consistent database snapshot while normal transactions continue. Live record counts taken afterward can legitimately differ from its snapshot. For an exact count comparison, use a maintenance window with web/worker stopped and capture counts alongside that backup. Account for externally connected approved administrative tools as writers too.

Room photo backup and recovery

The web service mounts named volume room_media at /app/media; accepted images are stored under rooms/photos/. Local, HTTP and HTTPS Compose profiles all preserve this volume across container recreation/ordinary down. The actual Docker volume name includes the existing Compose project prefix. Keep the same project identity and never use down --volumes as a repair. Worker/database containers do not store the files.

The image initializes media directories for application UID/GID **1000**; normal uploads use directory mode **0750** and file mode **0640**. Do not replace the named volume with an empty host directory or run application uploads as root to work around an ownership problem. A database row with a missing photo file cannot recreate the image.

For a paired snapshot, pause staff uploads by stopping web and stop worker while taking the matching database snapshot. Use the current checkout/image and the mbs function from section 2. Commands below run a one-off tar process with the normal web volume; they do not start the application or send mail. Stop if any step fails and preserve the partial archive for diagnosis rather than calling it a verified backup.

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs stop proxy web worker
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
umask 077
MBS_MEDIA_BACKUP="$(mktemp "$(pwd)/deploy/backups/room_media_$(date -u +%Y%m%dT%H%M%S).XXXXXX.tar.gz)"
mbs run --rm --no-deps -T --entrypoint tar web -C /app/media -czf - rooms > "$MBS_MEDIA_BACKUP"
test -s "$MBS_MEDIA_BACKUP"
tar -tzf "$MBS_MEDIA_BACKUP" > /dev/null
chmod 600 "$MBS_MEDIA_BACKUP"
sha256sum "$MBS_MEDIA_BACKUP"
mbs up -d --wait --wait-timeout 180 web worker proxy
```

Record the printed database dump filename, photo archive filename/checksum and release commit as **one recovery point**, then copy both to approved separate storage. Archive-list success is structural validation; a restore drill must

verify that restored room rows resolve to visible restored photos. The database-only backup scheduler below does not archive media automatically; IT must include this additional snapshot/copy step in its approved complete-backup process.

For an actual approved recovery, restore the matching database according to sections 9-10 and keep proxy/web/worker stopped. Restore only a trusted MBS archive into the intended recovery installation's media volume. The following extraction writes files; use a separate recovery project/volume for a drill, never the live project's volume. Substitute the exact selected archive, review its rooms/photos/ entries and preserve the existing volume before recovery:

BASH / Display wraps; copy complete commands from the Markdown source.

```
MBS_MEDIA_BACKUP='/opt/mbs/MeetingRoomSystemWDN/deploy/backups/REPLACE_WITH_ACTUAL_PHOTO_ARCHIVE.tar.gz'
test -f "$MBS_MEDIA_BACKUP"
test -s "$MBS_MEDIA_BACKUP"
tar -tzf "$MBS_MEDIA_BACKUP"
mbs run --rm --no-deps -T --entrypoint tar web --no-same-owner -xzf - -C /app/media < "$MBS_MEDIA_BACKUP"
mbs run --rm --no-deps -T --entrypoint sh web -c 'id; test -w /app/media/rooms/photos'
```

Extraction runs as UID 1000, so newly restored files belong to the application. It does not remove unrelated existing files; use a reviewed clean recovery volume when a precise full recovery is required. Restore the database and photo archive from the same point, then check each restored photo through staff/booking screens before reopening. If ownership/extraction fails, keep writers stopped and investigate the selected volume rather than deleting it. Replaced/removed photos are deleted after a successful room change; an older database snapshot can refer to a file that no longer exists in today's volume, which is why the matching archive is required.

Copy and retention

Copy the verified dump and checksum to company-approved **separate** storage using its approved encrypted transfer method. Do not place passwords in an scp command. A template for an already configured SSH destination is:

BASH / Display wraps; copy complete commands from the Markdown source.

```
scp "$MBS_BACKUP" APPROVED_BACKUP_USER@APPROVED_BACKUP_HOST:/APPROVED_PRIVATE_MBS_DIRECTORY/
```

Replace all destination placeholders with IT's real approved target; verify the destination host key through IT. Verify checksum and restore readability on the stored copy. Local-only backups do not survive loss of this server/volume. Ensure the backup destination and secret storage have separate access controls.

IT should record a retention schedule, storage owner, off-host transfer method, and recovery targets. An example for a small office is seven daily, four weekly, and twelve monthly verified backups; confirm the actual policy rather than enforcing a sample by deleting files. Booking/audit history needs at least the agreed one-year business retention. The app implements no automatic history purge.

To preview old local dumps without deleting anything:

BASH / Display wraps; copy complete commands from the Markdown source.

```
find /opt/mbs/MeetingRoomSystemWDN/deploy/backups -maxdepth 1 -type f -name 'meeting_rooms_*.dump' -mtime +30 -print
```

Delete only explicitly reviewed files after confirming off-host retention and a tested restore, using your approved retention process. Avoid wildcards or recursive deletion as incident cleanup.

Schedule one daily backup on Oracle Linux

This example schedules 02:00 in the **server's configured local time zone**, not automatically Nepal time. Check the time zone and choose a company-approved slot. Oracle documents cron scheduling in its [Oracle Linux 9 task automation guide](#).

BASH / Display wraps; copy complete commands from the Markdown source.

```
timedatectl
dnf install -y cronie
systemctl enable --now crond
install -d -m 700 /var/log/mbs
```

Create a wrapper outside the Git checkout. This changes only backup scheduling when the operator executes it:

BASH / Display wraps; copy complete commands from the Markdown source.

```
cat > /usr/local/sbin/mbs-backup <<'SH'
#!/bin/sh
set -eu
umask 077
cd /opt/mbs/MeetingRoomSystemWDN
printf 'Backup started: %s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
date -u +%Y-%m-%dT%H:%M:%SZ > /var/log/mbs/backup-last-success
printf 'Backup completed: %s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
SH
chmod 700 /usr/local/sbin/mbs-backup
```

For an installation with a custom Compose project name, add its **actual existing** COMPOSE_PROJECT_NAME to the wrapper's backup invocation. The default checkout uses its directory-derived project name; leave it unchanged otherwise.

Run and verify the wrapper once before scheduling:

BASH / Display wraps; copy complete commands from the Markdown source.

```
/usr/local/sbin/mbs-backup
cat /var/log/mbs/backup-last-success
```

Create the cron entry; flock prevents overlapping scheduled runs:

BASH / Display wraps; copy complete commands from the Markdown source.

```
cat > /etc/cron.d/mbs-backup <<'CRON'
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
0 2 * * * root /usr/bin/flock -n /run/lock/mbs-backup.lock /usr/local/sbin/mbs-backup >>
/var/log/mbs/backup.log 2>&1
CRON
chmod 644 /etc/cron.d/mbs-backup
```

No notification delivery is built into this cron example. IT monitoring must alert on a stale/missing success marker or failed job; logs alone do not notify an operator. Add approved off-host copy/verification separately; a local backup success marker does not prove off-host transfer.

Configure rotation for this operator log if logrotate is installed:

BASH / Display wraps; copy complete commands from the Markdown source.

```
cat > /etc/logrotate.d/mbs-backup <<'ROTATE'
/var/log/mbs/backup.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
    create 0600 root root
}
ROTATE
```

Check schedule and latest success:

BASH / Display wraps; copy complete commands from the Markdown source.

```
systemctl status crond --no-pager
cat /etc/cron.d/mbs-backup
tail -n 40 /var/log/mbs/backup.log
python3 - <<'PY'
from pathlib import Path
import time
marker = Path('/var/log/mbs/backup-last-success')
if not marker.exists():
    raise SystemExit('ALERT: no successful scheduled backup marker')
age = time.time() - marker.stat().st_mtime
print('Hours since latest local backup success:', round(age / 3600, 2))
raise SystemExit(1 if age > 26 * 3600 else 0)
PY
```

Docker logs also need bounded host retention under IT's logging policy; this repository does not configure a rotating Docker log driver. `docker system df` is safe to inspect space, but do not run a global prune to solve growth.

9. Restore drill into a separate database

This drill **creates a new database and restores business data there**. It does not restore over or delete the live database. It shares the same PostgreSQL server/storage, so schedule it when extra disk/CPU usage is acceptable; a separate recovery host is stronger isolation for a full disaster rehearsal. Do not launch a worker against the drill database: it could send duplicate real company mail or alter restored booking states.

Only restore trusted company-created backups. PostgreSQL custom archives contain executable schema definitions. The flags below fail on restore errors and intentionally skip original ownership/ACLs, so the restoring administrator owns objects and `db_setup` reapplies application permissions. See [PostgreSQL 17 pg_restore](#).

9.1 Choose and validate the archive and target

Run after selecting the actual `MBS_BACKUP` in section 8:

BASH / Display wraps; copy complete commands from the Markdown source.

```
test -f "$MBS_BACKUP"
test -s "$MBS_BACKUP"
mbs exec -T db pg_restore --list < "$MBS_BACKUP" > /dev/null
MBS_RESTORE_DB="mbs_restore_$(date -u +%Y%m%d_%H%M%S)"
case "$MBS_RESTORE_DB" in
    mbs_restore_[0-9]*) ;;
    *) printf '%s\n' 'Invalid restore target'; exit 1 ;;
esac
printf 'Separate restore database: %s\n' "$MBS_RESTORE_DB"
```

Validate that it differs from the live database before creation:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec -T -e MBS_RESTORE_DB="$MBS_RESTORE_DB" db sh -c '
set -eu
test "$MBS_RESTORE_DB" != "$POSTGRES_DB"
case "$MBS_RESTORE_DB" in
    mbs_restore_*) ;;
    *) exit 1 ;;
esac
export PGPASSWORD="$POSTGRES_PASSWORD"
createdb -U "$POSTGRES_USER" --maintenance-db="$POSTGRES_DB" "$MBS_RESTORE_DB"
'
```

`createdb` fails if that name already exists; do not reuse, clean, or drop it automatically. Choose a new unique target and understand why the first creation failed. Continue only if creation succeeded.

9.2 Restore into the empty target

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec -T -e MBS_RESTORE_DB="$MBS_RESTORE_DB" db sh -c '
set -eu
test "$MBS_RESTORE_DB" != "$POSTGRES_DB"
export PGPASSWORD="$POSTGRES_PASSWORD"
pg_restore -U "$POSTGRES_USER" -d "$MBS_RESTORE_DB" --exit-on-error --single-transaction --no-owner
--no-privileges
' < "$MBS_BACKUP"
```

Do not add `--create`, `--clean`, or `--disable-triggers`. `--create` can restore to the database name embedded in an archive instead of the intended target. This procedure selects the target explicitly.

9.3 Inspect restored schema, constraints, and counts

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs exec -T -e MBS_RESTORE_DB="$MBS_RESTORE_DB" db sh -c 'PGPASSWORD="$POSTGRES_PASSWORD" psql --no-psqlrc
-v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$MBS_RESTORE_DB" <<'SQL'
BEGIN READ ONLY;
SELECT current_database() AS restored_database;
SELECT 'users' AS table_name, COUNT(*) AS rows FROM users
UNION ALL SELECT 'rooms', COUNT(*) FROM rooms
UNION ALL SELECT 'reservations', COUNT(*) FROM reservations
UNION ALL SELECT 'booking_series', COUNT(*) FROM booking_series
UNION ALL SELECT 'booking_attendees', COUNT(*) FROM booking_attendees
UNION ALL SELECT 'notifications', COUNT(*) FROM notifications
UNION ALL SELECT 'audit_events', COUNT(*) FROM audit_events;
SELECT name FROM django_migrations WHERE app='booking' ORDER BY name;
SELECT extname FROM pg_extension WHERE extname='btree_gist';
SELECT conname, contype, convalidated FROM pg_constraint
WHERE conrelid='public.reservations'::regclass ORDER BY conname;
SELECT status, COUNT(*) FROM reservations GROUP BY status ORDER BY status;
COMMIT;
SQL
```

Confirm the expected snapshot's migration list, row counts, known rooms/bookings, audit history, and overlap exclusion constraint `reservation_room_occupied_no_overlap`. Compare exact counts with a same-snapshot/writer-stopped manifest if one was recorded; do not demand equality with a still-changing live database.

9.4 Reapply grants and verify the restricted role

This affects privileges on the separate drill database, leaving `.env` and live database selection unchanged:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs run --rm -e POSTGRES_DB="$MBS_RESTORE_DB" db_setup
mbs run --rm --no-deps -e POSTGRES_DB="$MBS_RESTORE_DB" web python manage.py shell -c "from django.db
import connection; from booking.models import Room,Reservation,AuditEvent; c=connection.cursor();
c.execute('SET default_transaction_read_only=on'); c.execute('SELECT current_database(), current_user');
print(c.fetchone()); print('rooms/reservations/audit:', Room.objects.count(), Reservation.objects.count(),
AuditEvent.objects.count()); c.execute(\"SELECT
has_table_privilege(current_user,'public.audit_events','INSERT'),
has_table_privilege(current_user,'public.audit_events','UPDATE'),
has_table_privilege(current_user,'public.audit_events','DELETE')\"); print('audit INSERT/UPDATE/DELETE:',
c.fetchone())"
```

The query must report the drill database plus application role, readable records, and audit privileges (`True`, `False`, `False`). `db_setup` also sets the configured application-role password at cluster level; use the unchanged current `.env` credentials so live web/worker credentials remain consistent.

Record archive checksum, release commit, restored target, counts, constraint/role results, duration, date, and reviewer. Leave the drill database untouched until IT approves its cleanup and verifies the target name is the drill database, not production. No automatic deletion command is included here.

10. Approved live recovery

This is a **maintenance action that changes the live database selected by the application**. Use only after IT approves a specific recovery point and accepts loss of transactions after that backup. It is not a troubleshooting shortcut. Review release/schema compatibility and identify all writers before executing it.

The safer procedure restores into a new database and switches the application to it; it retains the original live database for investigation. It does not overwrite/drop the original or erase the PostgreSQL volume.

1. Record the failure time, current release, current database name, approved archive/checksum, last known good backup, and approved recovery target. Check the destination host/project/directory. Inform users of maintenance.
2. Stop browser access and every writer. Keep PostgreSQL running:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs stop proxy web worker
mbs ps -a
```

3. Take a **pre-recovery** backup of the original database if PostgreSQL is accessible, and copy it off-host. If it cannot be backed up, record the failure and preserve the existing volume rather than deleting it:

BASH / Display wraps; copy complete commands from the Markdown source.

```
COMPOSE_FILE=compose.prod.http.yaml sh deploy/backup.sh
```

4. Follow section 9 with the explicitly approved archive to create a **new empty recovery database**, restore, verify counts/schema/constraints, and verify runtime permissions. Do not start the restored worker during verification. Make sure the target is different from the original live name and no other service currently uses it.
5. Back up the current .env privately before switching its database name. This file contains secrets; keep it out of Git and support messages:

BASH / Display wraps; copy complete commands from the Markdown source.

```
umask 077
MBS_ENV_RECOVERY_COPY="/root/mbs-env-before-recovery-$(date -u +%Y%m%dT%H%M%S)"
install -m 600 .env "$MBS_ENV_RECOVERY_COPY"
nano .env
```

Set **only** POSTGRES_DB to the verified recovery database's exact name. Preserve administrator/runtime usernames/passwords, Django secret, selected HTTP settings, and mail settings. Remove duplicates. Do not change the database role credentials as part of a restore unless the reviewed recovery plan requires it.

6. Restore the matching photo archive to the intended recovery room_media volume as described in section 8, with writers stopped. Keep the original volume/paired backup available for investigation; a database switch alone cannot recover missing photo files. Then validate/recreate DB service configuration against the same existing named volume and run the compatible release's committed migrations and grants:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs config --quiet
mbs up -d --wait --wait-timeout 180 db
mbs run --rm migrate
mbs run --rm db_setup
```

The official PostgreSQL image skips initialization on an existing volume; this does not create the selected recovery database. It must already exist from the completed restore. Stop if any command fails.

7. Before workers resume, review how old state affects queued notifications, future bookings, and access. A recovered queue can contain mail that was already accepted after the backup and might resend. An old heartbeat is expected. Once staff approves resume, start the application:

BASH / Display wraps; copy complete commands from the Markdown source.

```
mbs up -d --wait --wait-timeout 180 web worker proxy
mbs ps
curl --fail --show-error http://mbs.wdn.com.np/healthz/
```

8. Verify runtime database identity with section 6, staff/employee login, room/booking history, future pending/approved requests, queues, check-in handling, reports, and audit. Request fresh login links/codes rather than reusing restored old messages. Inform users of the actual recovery point and reconcile meetings/actions performed after that point.
9. Update backup monitoring for the new selected database; create/copy a fresh backup. Keep the original database and pre-recovery backup through IT's approved investigation/retention period.

If rollback is needed, stop every writer again and review which database has accepted new transactions. Switching back after recovered writes can lose those writes; do not casually alternate database names or attempt to merge rows directly. Use an explicit reviewed recovery/reconciliation plan.

11. Daily, weekly, and release checklists

Daily operator checks

- `mbs ps: db/web/worker/proxy` running and healthy.
- `/healthz/`: DB/worker healthy; note whether email is configured.
- Read-only notification snapshot: failed jobs, exhausted attempts, overdue sending leases, oldest pending work.
- Staff desk: pending requests reviewed before their starts; operational no-shows understood.
- Latest scheduled backup marker, successful off-host transfer, free disk/inodes.
- Review recent worker/DB error counts; do not treat every superseded notification as an outage.

Weekly checks

- Verify one stored off-host archive/checksum and retention inventory.
- Review staff access, departed employees, SMTP relay permission, and active room inventory with owners.
- Inspect Docker log/dump/storage growth and backup scheduler failures.
- Confirm server clock/time synchronization and office DNS resolution from a real employee device.
- Review audit events for denied authentication, access changes, overrides, retry operations, and booking decisions.
- Confirm room closures/holidays/policy are current; do not silently rewrite historic reservations.

Monthly / after infrastructure changes

- Perform and document a separate-database restore drill; periodically rehearse on a separate host too.
- Review approved backup recovery point/objectives and measured restore duration.
- Test employee email-link login, staff password/code login, confirmation/reminder delivery, and email check-in on representative company devices/mailboxes.
- Recheck firewall/VPN scope, SELinux, and relay source-IP authorization after network changes.

Release acceptance

- Record approved release commit; check clean checkout and compatible schema.
- Verified backup exists off-host; recovery plan prepared before migration.
- Stop all writers for migrations, apply committed migrations, rerun runtime grants.
- All health checks return expected status; inspect safe startup/error logs.
- Employee can request/confirm a login; staff can authenticate with password/code.
- Room search/details/calendar work; the booking wizard previews every selected occurrence and preserves edit values. Staff can add/edit room photos under Staff desk -> Rooms.

- Existing inventory remains; migrations 0009/0010/0011 are applied, existing rooms initially require approval and named media volume is writable by UID 1000. Paired database/photo restore has been checked when photos exist.
- Submit a checked-room test workflow: Pending hold -> approval/rejection, employee edit requiring review, cancellation, recurring occurrence horizon, valid check-in/no-show handling.
- Verify an unchecked room creates Approved employee bookings, including recurrence; toggling the setting does not convert existing Pending/Approved records, and unchanged saves preserve their status. Verify current selected-room policy on substantive edits and an audited staff checkbox change.
- Confirm reports/audit/access controls and actual mailbox delivery; do not infer all this from HTTP 200 alone.
- Keep test meetings distinguishable and cancel eligible test bookings through the UI after verification.

12. Incident handover and operational limits

Send IT enough evidence

Provide the incident time/time zone, affected ordinary URL path or named screen, user role, exact visible error, whether fresh/incognito browser reproduces, release commit, mbs ps, health results, effective nonsecret settings, and relevant sanitized logs/request IDs. Include SMTP host/port/from address and the relay-visible server source IP when mail is involved. Do not provide raw sign-in/check-in/password-reset links, codes, passwords, full .env, notification bodies, or dumps.

Identify whether the failure is:

- **Client/DNS/network:** cannot resolve/reach office HTTP address; IP/Host-header server test works.
- **Proxy/web:** liveness fails, proxy returns 502/504, web exits/unhealthy.
- **Database:** readiness reports unavailable DB, role authentication/permission/schema errors.
- **Worker:** DB is okay but success heartbeat missing/stale; lifecycle jobs/notifications stop advancing.
- **SMTP:** app is healthy but sign-in/setup fails or queued sends accumulate.
- **Business rule:** request denied by overlap, policy, horizon, inactive room, authorization, or expired check-in; use staff workflows rather than infrastructure restarts.

Known boundaries

- This is a single-server application; Docker restart policies and backups are not high availability.
- A healthy site does not prove SMTP delivery, office DNS/firewall, backup copy, or restore readiness.
- Calendar is built into the app; direct HCL/Outlook integration is deferred.
- Recurrence creates explicit occurrences only within the permitted horizon (default 14 days). It does not reserve future months automatically; organizers book later eligible months manually.
- Front Desk/Administrators share powers; WDN third-floor senior-management priority is handled manually.
- Worker recovery after an outage can expire already-started pending requests and mark overdue approved meetings no-show. Staff manual check-in also obeys the valid window; it cannot retroactively bypass the deadline.
- Policy, holiday, room deactivation, and account deactivation changes do not automatically rewrite/cancel all existing bookings. Staff reviews affected future meetings.
- Reports use current policy/holidays for utilization and are operational estimates, not a historically reconstructed accounting system.
- There is no built-in alerting, automatic off-host backup copy, automatic data purge, or exactly-once SMTP guarantee.

- The HTTP production profile intentionally generates transport warnings in Django check --deploy; do not disable CSRF/auth safeguards to silence them.

13. Windows laptop differences

Laptop development uses compose.yaml, loopback http://127.0.0.1:8000, a file-email development inbox, and the administrator DB role. It is separate from the company production profile. Start Docker Desktop with its Linux engine first. An npipe/dockerDesktopLinuxEngine connection failure means the Docker engine is not available; no migration has run successfully through that failure.

In **PowerShell**, from your actual local checkout:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
Set-Location 'C:\Users\Dell\Documents\Codex\2026-09-28\files-mentioned-by-the-user-meeting\outputs\meeting
-room-booking'
docker info --format '{{.ServerVersion}}'
docker compose -f compose.yaml config --quiet
docker compose -f compose.yaml up -d db
docker compose -f compose.yaml run --rm migrate
docker compose -f compose.yaml up -d web worker
docker compose -f compose.yaml ps
docker compose -f compose.yaml logs --since 10m --tail 100 web worker db
```

For a fresh local .env, the generator without production flags selects .env.example; use your installed Python 3.12 command:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
python deploy/create-env.py
```

It refuses to overwrite an existing file. Keep local DJANGO_PRODUCTION=false, DJANGO_HTTPS=false, DJANGO_PUBLIC_BASE_URL=http://127.0.0.1:8000, and suitable localhost/127.0.0.1 hosts. Production settings copied into the local profile can disable the demo inbox or make generated URLs point at the office server. Use the same loopback address consistently across form and email link.

Local file-email messages are visible at /dev/mail/; it is intentionally unavailable in production. Local mail_init prepares file-mail volume ownership; that service does not exist in either production profile. Demo room seeding is available only locally and rejected in production.

PowerShell does not use Bash's mbs() function, <<'SQL' heredocs, or inline COMPOSE_FILE=value command syntax. For production diagnosis on your laptop, SSH into the Linux server and use the Bash sections there. Do not run production restore instructions against your laptop accidentally.

For a local SQL SELECT, pass a PowerShell literal here-string into Docker rather than interpolating \$POSTGRES_PASSWORD on the Windows host:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
@'
BEGIN READ ONLY;
SELECT status, COUNT(*) FROM notifications GROUP BY status ORDER BY status;
COMMIT;
'@ | docker compose -f compose.yaml exec -T db sh -c 'PGPASSWORD="$POSTGRES_PASSWORD" psql --no-psqlrc -v
ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
```

Do not redirect a binary database dump through older Windows PowerShell text redirection. Run the documented backup script on Linux, or use a reviewed binary-preserving export method. Keep local and production data/credentials distinct.

7. Troubleshooting and logs

Source: docs/troubleshooting.md

Transgate Tech | Binayak Bhandari

Production commands use **Oracle Linux**, `/opt/mbs/MeetingRoomSystemWDN`, branch `mbs-prod`, and <http://mbs.wdn.com.np> at **192.168.50.222**. Run Linux commands in that repository. Laptop commands use PowerShell and the local `compose.yaml`; they are identified separately. Use [operations](#) for planned changes, backups and restoration, [HTTP deployment](#) for installation, and [staff instructions](#) for business actions.

1. First response

Collect the approximate time/timezone, page **path** such as `/staff/sign-in/`, action, visible message and support reference. Describe whether one person, everyone, one room or one action is affected. Never share an email sign-in/setup/check-in URL, password, code, cookie, `.env` or dump in a ticket. Check the booking's current status before assuming a failed email or stale screen means the booking failed.

Start with these read-only commands:

BASH / Display wraps; copy complete commands from the Markdown source.

```
cd /opt/mbs/MeetingRoomSystemWDN
date -Is
git branch --show-current
git rev-parse --short HEAD
docker compose -f compose.prod.http.yaml ps -a
docker compose -f compose.prod.http.yaml logs --since 15m --tail=200 --timestamps web worker proxy db
curl --silent --show-error --max-time 15 --write-out '\nHTTP %{http_code}\n' http://mbs.wdn.com.np/livez/
curl --silent --show-error --max-time 15 --write-out '\nHTTP %{http_code}\n'
    http://mbs.wdn.com.np/healthz/
```

Observation	Meaning	Next section
Connection refused/timeout; no HTTP status	DNS, binding, routing, proxy or host	Network
<code>/livez/</code> is ok, <code>/healthz/</code> is 503	Web responds; database or worker readiness failed	Health
403 session-check page	CSRF validation failed	CSRF
400/rejected hostname	Allowed-host/public-origin mismatch	Configuration
502/504	Proxy could not complete upstream request	Network
Branded 500/503	Application failure or unavailable database	Application errors
Email missing after sign-in request	SMTP/relay/mailbox problem; generic success message alone proves no delivery	Email
Booking still pending	Review required when submitted, or saved Pending retained after policy changed	Booking rules
Release/reminder late	Check worker, deadline, booking state and outbox	Health , email

A successful sign-in-page GET does not prove POST, SMTP, staff MFA or check-in works. Readiness does not prove inbox delivery.

2. Logs and support evidence

Recent/live logs and incident windows

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Recent records with Docker timestamps.
docker compose -f compose.prod.http.yaml logs --since 30m --tail=300 --timestamps web worker proxy db

# Ctrl+C stops viewing; containers keep running.
docker compose -f compose.prod.http.yaml logs --follow --since 5m --tail=100 web worker proxy

# Replace these example UTC times with the actual incident window.
docker compose -f compose.prod.http.yaml logs --since '2026-10-07T10:15:00Z' --until
'2026-10-07T10:25:00Z' --timestamps web proxy

# Bounded search for failure categories.
docker compose -f compose.prod.http.yaml logs --since 1h --no-color web worker proxy db | grep -E
'ERROR|WARNING|failed|unavailable|CSRF|TIMEOUT|status=5[0-9][0-9]|status=403'
```

Docker's [Compose logs reference](#) explains these options. `grep` exits 1 if no matches; that is not proof of health.

Service	Logged information
proxy	Method, HTTP status, bytes, duration, upstream status, request ID. No URL/query, cookies or form body. Sensitive token routes omit access logging.
web	Gunicorn startup; safe HTTP failures, resolved view, exception class, optional SQLSTATE, sanitized source locations and request ID; CSRF/authentication categories.
worker	released=N completed=N reminders=N sent=N; notification ID/attempt/error class; cycle failure class. Quiet periods with no work are normal.
db	PostgreSQL startup/shutdown/recovery, authentication and database errors. Review operational details before sharing.

Normal application logs intentionally omit raw exception messages, traceback locals and secret-bearing requests. They are not a full SMTP transcript or business audit. Use `/staff/audit/` for business actions. Proxy times include an offset; PostgreSQL/Gunicorn commonly log UTC; screens use Nepal time (UTC+05:45).

Correlate a support reference

Use the displayed 32-character support reference from a 500/503 page, never the token from an email URL:

BASH / Display wraps; copy complete commands from the Markdown source.

```
REQUEST_REF='REPLACE_WITH_32_CHARACTER_SUPPORT_REFERENCE'
docker compose -f compose.prod.http.yaml logs --since 1h --no-color web proxy | grep -F "$REQUEST_REF"
```

No result can mean a short time window, old recreated container or sensitive route with no proxy access record. X-Request-ID also carries the reference. Capture evidence **before** recreating containers, which removes their old locally accessible logs.

Save a private incident log

BASH / Display wraps; copy complete commands from the Markdown source.

```
umask 077
mkdir -p /root/mbs-support
chmod 700 /root/mbs-support
INCIDENT_FILE="/root/mbs-support/mbs-$(date -u +%Y%m%dT%H%M%S).log"
docker compose -f compose.prod.http.yaml logs --since 30m --tail=1000 --timestamps --no-color web worker
proxy db > "$INCIDENT_FILE" 2>&1
chmod 600 "$INCIDENT_FILE"
printf 'Saved locally: %s\n' "$INCIDENT_FILE"
```

Review/redact before sharing. Never commit logs, dumps or .env. No external collector is included; IT chooses retention/rotation/monitoring. Compose logs only retrieves records retained by Docker's current logging driver.

3. Health and worker problems

Check	Expected	Meaning
/livez/	200 text ok	Web can answer a simple request.
/healthz/	200 JSON status: ok, database: ok, worker: ok	Database query works; worker reconciliation succeeded within 120 seconds.
/healthz/, database unavailable	503, database unavailable	Database query failed.
/healthz/, worker unavailable	503, database ok, worker unavailable	Heartbeat absent/stale.
check_worker_health	ok, exit 0	Same heartbeat check via management command.

503 may occur briefly during startup. email: configured means host/sender configured, not relay acceptance; readiness can be 200 with email unavailable.

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec worker python manage.py check_worker_health
docker compose -f compose.prod.http.yaml logs --since 15m --tail=200 worker db
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
from django.utils import timezone
from booking.models import WorkerHeartbeat
h = WorkerHeartbeat.objects.filter(pk=1).values('last_success_at', 'last_error_at').first()
print('now_utc=', timezone.now().isoformat())
print('heartbeat=', h)
PY
```

The loop attempts one due notification per iteration, normally about one second; reconciliation runs approximately every 30 seconds. SMTP has a ten-second timeout. Large queues do not all send in one reconciliation. Correct database/network/configuration failures before restarting. If the worker remains stuck after diagnosis:

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Recovery mutation: restart and resume queued work.
docker compose -f compose.prod.http.yaml restart worker
docker compose -f compose.prod.http.yaml logs --since 2m --tail=100 worker
docker compose -f compose.prod.http.yaml exec worker python manage.py check_worker_health
```

run_booking_worker --once is **not read-only**: it can cancel requests, release no-shows, complete meetings, create reminders and send email. Use check_worker_health for diagnosis. Avoid extra permanent interactive workers.

restart: unless-stopped acts when a process exits; it does not restart a container merely marked unhealthy. IT should alert on readiness, container state and heartbeat. Stale worker readiness can also prevent web/proxy startup readiness succeeding even when Gunicorn started.

4. Network, DNS and proxy

Linux server checks

BASH / Display wraps; copy complete commands from the Markdown source.

```
hostname -I
ip -brief address
getent ahostsv4 mbs.wdn.com.np
ss -ltnp '( sport = :80 )'
docker compose -f compose.prod.http.yaml port proxy 80
docker compose -f compose.prod.http.yaml logs --since 10m --tail=100 proxy web

# Bypass DNS, retaining the correct Host header.
curl --silent --show-error --max-time 15 --resolve mbs.wdn.com.np:80:192.168.50.222 --write-out '\nHTTP
  %{http_code}\n' http://mbs.wdn.com.np/livez/
```

Office IPv4 is 192.168.50.222; 172.17.0.1 is Docker's bridge. If --resolve works while ordinary curl fails, inspect DNS. If both fail, inspect binding/proxy/routing. Raw-IP browser URLs can fail allowed-host validation and change cookies; keep the canonical hostname.

Employee laptop checks (PowerShell)

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
Resolve-DnsName mbs.wdn.com.np
Test-NetConnection mbs.wdn.com.np -Port 80
curl.exe --silent --show-error --max-time 15 --write-out '\nHTTP %{http_code}\n'
  http://mbs.wdn.com.np/livez/
```

If server-local tests work but laptops fail, IT checks routing/VPN/DNS/firewall. Inspect services without disabling them:

BASH / Display wraps; copy complete commands from the Markdown source.

```
systemctl is-active docker firewalld
firewall-cmd --get-active-zones
firewall-cmd --list-all-zones
getenforce
ip route
docker network ls
```

Docker creates published-port filtering rules and integrates with firewalld. A host INPUT rule alone does not establish an office-only allowlist for forwarded Docker traffic. Verify access restrictions at the correct network/Docker forwarding layer; see [Docker packet filtering](#). Keep SELinux, firewalld and Docker filtering enabled.

Symptom	Check/correction
address already in use	Identify port-80 owner with ss; review the intended conflicting service before changing it.
cannot assign requested address	Correct HTTP_BIND_IP to an actual office-interface address.
Subnet/allocation conflict	Check office routes/Docker networks; update subnet, dynamic range, proxy IP and trusted CIDR together; follow operations network-recreation instructions.
502 after recreation	Check web health/logs/current image; current Nginx dynamically resolves web:8000 through Docker DNS.
504/slow requests	Check database/web logs and resource pressure, not just timeouts.
Browser changes to HTTPS	Check HTTPS-only browser settings or old HSTS from an HTTPS trial. The HTTP proxy sends no HSTS.

Symptom	Check/correction
SELinux mount denial	Verify committed :ro,Z, file ownership, <code>ls -lZ deploy/*.sh deploy/*.template</code> and <code>ausearch -m AVC -ts recent</code> if installed. Correct the specific access/label.
Orphan mail_init warning	A local-profile service absent from production, not itself a failure. Verify service/project identity before targeted cleanup; preserve volumes.

5. Session check and CSRF 403

Let's start fresh means the POST failed CSRF validation, not that the email/password was wrong. Session-backed CSRF protects login and every state-changing form.

1. Open a fresh form at `http://mbs.wdn.com.np/sign-in/` or `/staff/sign-in/`; submit in that same session and hostname.
2. Avoid stale tabs/back-button forms and hostname/IP/localhost switching; allow site cookies.
3. If fresh Chrome Incognito also fails, inspect the server/header rather than repeatedly clearing cookies.

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Filtered headers omit Set-Cookie.
curl --silent --show-error --dump-header - --output /dev/null http://mbs.wdn.com.np/sign-in/ | grep -Ei
'^^(HTTP/|Referrer-Policy:|X-Request-ID:|Location:)'
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
from django.conf import settings
keys = ('PRODUCTION_ENABLED', 'HTTPS_ENABLED', 'PUBLIC_BASE_URL', 'ALLOWED_HOSTS',
        'SESSION_COOKIE_SECURE', 'SESSION_COOKIE_DOMAIN', 'CSRF_USE_SESSIONS',
        'CSRF_TRUSTED_ORIGINS', 'SECURE_REFERRER_POLICY')
print({k: getattr(settings, k, 'MISSING') for k in keys})
PY
docker compose -f compose.prod.http.yaml logs --since 5m --tail=100 web
```

Expected: production true, HTTPS false, public URL `http://mbs.wdn.com.np`, session secure false, session domain None, session CSRF true, trusted origins exactly that origin, referrer policy **same-origin**.

Earlier no-referrer made native browser POSTs send `Origin: null`; Django rejected them. The correction is in 392dfa5 and later versions. Verify the response header; pulling source alone does not update running Python code. See [verification report](#) and [Fetch origin-header rules](#).

Safe category	Likely reason	Action
missing_session	Missing/expired CSRF session	Fresh form, cookie/hostname/environment check.
missing_token	Form token absent	Current form/templates/image.
invalid_token	Stale/mismatched token/session	Reload after sign-in/out or access/password change.
origin_mismatch	Wrong scheme/host/port or null Origin	Canonical URL, exact trusted origin, Host header, referrer policy.
referer_rejected	HTTPS referer validation failed	Alternative HTTPS profile: check origin/proxy scheme/context.
other	Other rejection	Correlate request ID and deployed version.

Never disable middleware/tokens, enable DEBUG, trust arbitrary/null origins in production. The local opaque-origin accommodation is restricted to loopback development.

6. Configuration and stale containers

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Validate without printing expanded passwords.
docker compose -f compose.prod.http.yaml config --quiet

# Duplicate NAMES/counts only, never values.
awk -F= '/^[A-Za-z_][A-Za-z0-9_]*={/{count[$1]++} END {for (k in count) if (count[k]>1) print k, count[k]}'
.env

# Explicitly nonsecret runtime settings only.
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
from django.conf import settings
keys = ('DEBUG', 'PRODUCTION_ENABLED', 'HTTPS_ENABLED', 'PUBLIC_BASE_URL',
        'ALLOWED_HOSTS', 'EMPLOYEE_EMAIL_DOMAINS', 'MAIL_MODE', 'EMAIL_READY',
        'EMAIL_HOST', 'EMAIL_PORT', 'DEFAULT_FROM_EMAIL', 'EMAIL_USE_TLS',
        'EMAIL_USE_SSL', 'AUTH_TRUSTED_PROXY_CIDRS', 'TIME_ZONE')
print({k: getattr(settings, k, 'MISSING') for k in keys})
PY
```

Do not publish plain Compose config, full `docker inspect`, `printenv`, `.env` or Django database/authentication settings. Remove duplicate keys so each appears once. Environment names are SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_USE_TLS, SMTP_USE_SSL, DJANGO_FROM_EMAIL; internal Django EMAIL_* settings are derived, not the environment contract.

Production profiles force production true, their own transport, and wdn.com.np, transgate.com.np. Editing `.env` alone cannot add a production employee domain. File-email mode, `/dev/mail/` and demo seeding are unavailable in both production profiles.

For a reviewed email/origin `.env` change only:

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Configuration mutation with brief service interruption.
docker compose -f compose.prod.http.yaml config --quiet
docker compose -f compose.prod.http.yaml up -d --no-deps --force-recreate --wait --wait-timeout 180 web
worker proxy
```

restart retains the old environment. Python/templates/static changes need a build and recreation; use [controlled updates](#). Documentation-only changes need no image rebuild. Existing-volume **Skipping initialization** is normal: changing bootstrap credentials in `.env` does not change stored PostgreSQL roles. Preserve working credentials/data and use deliberate rotation.

7. Email sign-in and SMTP

Selected relay and connectivity

Previously accepted company settings:

INI / Display wraps; copy complete commands from the Markdown source.

```
DJANGO_EMAIL_BACKEND=smtp
DJANGO_FROM_EMAIL=mbs@wdn.com.np
SMTP_HOST=maildc01.wdn.com.np
SMTP_PORT=25
SMTP_USER=
SMTP_PASSWORD=
SMTP_USE_TLS=false
SMTP_USE_SSL=false
```

Acceptance from one source does not prove authorization of a new server IP or inbox arrival. IT must permit this sender/source. Credentials are used when relay policy requires them; adding a password cannot automatically fix sender rejection.

BASH / Display wraps; copy complete commands from the Markdown source.

```
getent ahostsv4 maildc01.wdn.com.np
docker compose -f compose.prod.http.yaml logs --since 15m --tail=150 web worker

# Actual application network: DNS, connection and EHLO only; no AUTH/mail.
# For the selected plain-SMTP port-25 relay, not implicit SSL port 465.
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
import smtplib
import socket
from django.conf import settings
try:
    addresses = socket.getaddrinfo(settings.EMAIL_HOST, settings.EMAIL_PORT, type=socket.SOCK_STREAM)
    print('addresses=', sorted({a[4][0] for a in addresses}))
    with smtplib.SMTP(settings.EMAIL_HOST, settings.EMAIL_PORT, timeout=10) as smtp:
        code, _ = smtp.ehlo()
        print('ehlo_status=', code)
        print('capabilities=', sorted(smtp.esmtp_features))
except Exception as exc:
    print('smtp_error_class=', type(exc).__name__)
    print('smtp_status=', getattr(exc, 'smtp_code', None))
PY
```

EHLO success proves connectivity, not sender/recipient authorization. Use a matching connection type for an alternative IT-approved TLS relay.

Send one deliberate test

This **sends real email**: enter an authorized recipient, run once, check Inbox/Junk. It never prints credentials. Backend acceptance does not prove inbox delivery.

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec web python manage.py shell -c "from django.conf import
settings; from django.core.mail import send_mail; recipient=input('Authorized test recipient: ').strip();
print('backend accepted count=', send_mail('MBS relay test', 'One test message from the company Meeting
Booking System. Please confirm receipt.', settings.DEFAULT_FROM_EMAIL, [recipient], fail_silently=False))"
```

Exceptions from an interactive test may show sender/recipient and policy text; review before sharing. Never enable SMTP debuglevel, which can expose AUTH/message contents.

Error	Likely cause	Next action
gaierror	Relay DNS failure	Company DNS/VPN and container resolver.
ConnectionRefusedError	No listener/wrong port/active reject	Confirm endpoint/port.
TimeoutError, SMTPServerDisconnected	Routing/firewall/relay availability or protocol mismatch	Compare container EHLO and relay logs with IT.
SMTPSenderRefused, 554 policy rejection	Envelope sender/source policy denied	IT authorizes <code>mbs@wdn.com.np</code> from this server or supplies approved sender/relay; a password may not fix it.
SMTPRecipientsRefused	Recipient/domain/relay permission rejected	Exact mailbox and accepted destination domains.
SMTPAuthenticationError	Wrong credentials/AUTH policy	IT confirms account and permitted AUTH/encryption; password stays in private <code>.env</code> .
SMTPNotSupportedError	Requested STARTTLS/AUTH not advertised	Match settings to approved capabilities.
SSLError	TLS mode/certificate/trust mismatch	Correct endpoint/mode/company trust.

Error	Likely cause	Next action
Disabled-backend warning	Disabled or missing host/sender	Configure/recreate services; queued jobs consume no retries while disabled.
Accepted but no Inbox message	Routing/quarantine/spam/mailbox after acceptance	Provide IT sender, recipient, test subject and timestamp for relay tracking.

The selected relay uses both TLS/SSL false. Never enable both; that prevents startup. Mailbox domain need not equal the relay hostname, but organizer/staff identities need an allowed company domain.

Authentication versus queued notifications

Employee links, staff codes and setup links send synchronously; failed delivery denies authentication. They are **not** recovered by `retry_failed_mail`. Repair SMTP and request a fresh link/code/setup. The employee check-email screen intentionally avoids account enumeration.

Booking events/reminders are worker-delivered outbox records. Inspect metadata without bodies/subjects/tokens/recipient lists:

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
from django.db.models import Count
from booking.models import Notification
print('counts=',
      list(Notification.objects.values('status').annotate(total=Count('id')).order_by('status')))
rows = Notification.objects.filter(status__in=['pending', 'sending',
      'failed']).order_by('next_attempt_at').values('id', 'reservation_id', 'reservation_revision',
      'event_type', 'status', 'attempts', 'next_attempt_at', 'last_error')[:50]
print('queue_metadata=', list(rows))
PY
```

pending awaits delivery; sending holds a five-minute lease; sent means backend acceptance; failed retains error class; skipped displays **Superseded** for obsolete events. Failed attempts retry after approximately 2, 4, 8 and 16 minutes. The fifth failure is terminal until requeued. Its timestamp may be 32 minutes ahead, but there is no automatic sixth attempt. Interrupted leases recover after expiry; an interrupted final attempt becomes `WorkerInterrupted`. Ambiguous SMTP outcomes may cause duplicate delivery.

After SMTP repair, intentionally requeue eligible failures:

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Mutates outbox; the worker may send real emails.
docker compose -f compose.prod.http.yaml exec web python manage.py retry_failed_mail --id 123
# Alternatively all failed jobs:
docker compose -f compose.prod.http.yaml exec web python manage.py retry_failed_mail --all
```

Replace 123 with the diagnosed notification ID. Output is `requeued=N superseded=N`; eligible failures reset attempts and the action is audited. Obsolete revisions/events are skipped. This cannot recreate login tokens. Never update the outbox in SQL or resend obsolete check-in links.

8. Booking availability and business rules

Symptom	Explanation/resolution
Unavailable after visible meeting end	Gap applies: default 10:00-11:00 occupies through 11:15.
Pending request blocks another person	Intended slot hold; staff approves/rejects. Expired requests cancel at start.
No approved meeting but room unavailable	Pending requests, closures, gap, holidays, activity or horizon.
Recurring series fails entirely	All occurrences save atomically; inspect every date/conflict.
Monthly recurrence gives one occurrence	Only dates within default 14-day horizon; later months booked manually.
Approved becomes pending after edit	Substantive employee edit in a room currently requiring approval; selected room's current setting applies. Staff edits of Approved retain approval.
Unchecked room still has Pending requests	A toggle never rewrites bookings. Existing Pending needs explicit staff review or a substantive valid edit in the unchecked room; an unchanged save preserves Pending.
Checkbox does not save	Staff password/code session required; with JavaScript it submits on change, otherwise use that row's Save button. Reload and check audit. A 403 requires CSRF investigation; do not disable CSRF.
Unchecked room booking confirmed but no email	Approved is a saved booking state, not proof of delivery. Inspect worker/outbox/relay as in section 7; the check-in deadline remains.
Old approval screen fails	Reload; changed/expired/cancelled/rejected request cannot be revived.
Date/time invalid	Future Nepal start, Mon-Fri hours, holidays, duration/increments/horizon; current policy may differ from defaults.
Staff override still conflicts	Never bypasses overlap, room/organizer activity or capacity. Move/cancel through UI or select another slot.
Senior-management third-floor priority	Front Desk decides/moves/cancels manually with reasons/notifications; no automatic preemption.
Deactivated room still has bookings	Deactivation prevents new reservations, not retroactive cancellation; staff reviews existing meetings.
New holiday/policy leaves meetings unchanged	No retroactive cancellation; staff reviews affected records. Current check-in deadline applies to existing approved meetings.
Other employee details denied	Ownership or verified staff required; expected privacy behavior.
Only one room listed	Staff must configure actual active rooms; production never seeds demo data.
Wizard shows unavailable for a free first date	All eligible recurring occurrences must be free, including gaps; inspect later dates in the preview/review.
Selected Available room fails final submission	Preview does not hold space; final save detects intervening conflicts, inactive rooms and capacity/policy changes. Retain entries and choose a valid alternative.
External or Internal + External rejected	Guest company is required for both; count email-listed people and additional no-email guests once each.

Symptom	Explanation/resolution
Wizard controls absent or room cards never update	Native form remains usable. Inspect browser console/Network for the static wizard asset and /bookings/availability/ response; rebuild/recreate a stale image and load a fresh page. Share status/error text, not full request values.
Invalid form cannot be retried	Load current assets; client validation must not leave a prevented submission locked. Server errors retain data for correction.

Read-only current policy/status totals:

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
from django.db.models import Count
from booking.models import BookingPolicy, Reservation, Room, CompanyHoliday
fields = ('opens_at', 'closes_at', 'minimum_minutes', 'maximum_minutes', 'slot_minutes', 'gap_minutes',
'advance_days', 'check_in_minutes')
print('policy=', BookingPolicy.objects.filter(pk=1).values(*fields).first())
print('rooms=', list(Room.objects.values('is_active',
'requires_approval').annotate(total=Count('id')).order_by('is_active', 'requires_approval')))
print('reservations=', list(Reservation.objects.values('kind',
'status').annotate(total=Count('id')).order_by('kind', 'status')))
print('holiday_count=', CompanyHoliday.objects.count())
PY
```

Inspect individual bookings/audit in Staff desk. Direct SQL status/time edits bypass locking, notifications, revisions and audit.

Room photo upload or display

Symptom	Check
Upload rejected	Still JPEG/PNG/WebP only; at most 5 MiB and 12 million pixels. An extension alone does not prove readable image contents.
Form correction loses selected file	Browsers do not retain the file picker selection; select the upload again before resubmitting.
Replace/remove error	Choose a new upload or Remove current photo, not both.
Photo save reports storage problem	Check named room_media volume mounted at /app/media, directory ownership UID/GID 1000 and writable volume; application root remains read-only.
Photo request redirects to sign-in	Images require an authenticated session. Ordinary employee sessions cannot view inactive room photos.
Room has no photo	Optional and valid; staff can add one under Staff desk -> Rooms -> Edit.
Stored photo returns 404 after recovery	Database has only a filename; restore the matching photo archive, not only the dump. Missing files are not recreated from room records.
New schema field/endpoint fails after upgrade	Apply committed 0009/0010/0011 using the maintenance workflow, rerun db_setup, rebuild/recreate services; do not reseed rooms.

Read-only storage diagnosis (no photo bytes or credentials printed):

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml exec -T web python manage.py shell <<'PY'
import os
from pathlib import Path
from django.conf import settings
from booking.models import Room
root = Path(settings.MEDIA_ROOT)
photos = root / 'rooms' / 'photos'
print('process_uid_gid=', (os.getuid(), os.getgid()))
print('media_root=', str(root), 'photos_directory_exists=', photos.is_dir(), 'writable=',
      os.access(photos, os.W_OK))
if photos.exists():
    info = photos.stat()
    print('directory_uid_gid_mode=', (info.st_uid, info.st_gid, oct(info.st_mode & 0o777)))
references = list(Room.objects.exclude(photo='').values_list('photo', flat=True))
print('room_photo_references=', len(references))
print('missing_photo_files=', sum(not (root / name).is_file() for name in references))
PY
docker compose -f compose.prod.http.yaml logs --since 10m --tail 80 web
```

Expected application UID is 1000 and the photo directory is writable. Existing named volumes retain their own permissions; rebuilding the image cannot repair a separately misowned volume. Have IT inspect the selected project/volume and restore ownership through its reviewed maintenance process. Keep writers stopped during recovery; do not delete/reset the volume. See [photo backup and recovery](#).

Check-in/release

Check-in needs an approved meeting, at/after start and **before** the current deadline. Exactly start+15 minutes is too late under defaults. Opening a link displays confirmation; pressing the button changes status. The link proves organizer mailbox access and needs no earlier employee session. Expired/consumed/rescheduled/cancelled links fail.

Verified staff can check in through the booking page within the same window; they cannot late-check-in a no-show. If a new meeting is needed, create a new eligible booking. Release sets no_show, keeps history and releases occupancy; it does not delete. Stale workers can delay release while late check-in is already rejected. Booking writes reconcile overdue records; calendar GETs do not. Check heartbeat/logs and do not promise an exact release second.

9. Staff access and room editing

Admin is the custom **Staff desk**, not /admin/. Front Desk/Administrators have identical powers. Staff accounts using employee email login get only employee capabilities; staff actions need password plus code.

Need/problem	Action
Room Edit	/staff/rooms/ via Staff desk, Edit in row; /rooms/ is employee directory.
Staff pages	/staff/sign-in/, password, eight-digit email code at /staff/code/.
New staff	Authorized operator bootstrap below; user sets their own password from setup email.
Invalid/expired code	10-minute code, five guesses; request fresh code. Password/access changes invalidate challenges.
Expired setup	One-hour link; another staff sends fresh setup via People, or operator reruns bootstrap.
Access update invalidates session	Sign in again with current role.
Inactive account	Authentication denied; staff reviews/reactivates only when authorized.
Self-revoke refused	People prevents you from removing your own staff/activity. There is no separate last-staff concurrency guarantee; retain authorized operator recovery access.

Need/problem	Action
Forgotten password	Another staff sends setup email or authorized operator bootstrap; SMTP must work.

BASH / Display wraps; copy complete commands from the Markdown source.

```
# Privileged mutation: grants/reenables staff and sends setup email.
# Replace with the approved person's allowed company address.
docker compose -f compose.prod.http.yaml exec web python manage.py create_staff_account person@wdn.com.np
```

Email case is normalized; rerunning does not duplicate users. The role may be enabled before delivery failure; repair SMTP and rerun the same address. Avoid shared passwords or technical superusers as business onboarding. See [staff guide](#).

10. Application errors and static assets

Record time/path/support reference. A refresh succeeding does not identify the earlier cause. An old screenshot without matching failure context cannot establish a specific server defect.

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.http.yaml logs --since 20m --tail=300 web worker db proxy
docker compose -f compose.prod.http.yaml exec web python manage.py check
docker compose -f compose.prod.http.yaml exec web python manage.py showmigrations booking
docker compose -f compose.prod.http.yaml exec web python manage.py migrate --check
docker compose -f compose.prod.http.yaml images
```

showmigrations and migrate --check do not apply migrations. Schema changes require backup, stopped writers, committed migrations and db_setup via operations. Never generate migrations on the server or run web/worker permanently as database administrator.

Older sync Gunicorn could time out on idle browser connections. Current image uses two threaded workers/four threads each and no unused control socket, supporting the read-only filesystem. If logs show WORKER TIMEOUT/socket startup errors, verify current image/build; resource pressure/database slowness need separate diagnosis.

CSS/logo/static manifest errors may indicate stale/mismatched builds or asset paths. Assets collect/fingerprint during image build. Rebuild/recreate through controlled updates and hard-refresh; do not write files into a read-only production container. Branding/fonts are local/system assets; no CDN font dependency is required.

Alternative HTTPS deployment uses its own Compose/certificate guide. Selected HTTP intentionally has no certificate, redirect, HSTS or secure-cookie transport flag. check --deploy may report these transport warnings; maintain the chosen profile consistently while troubleshooting. [Django deployment checklist](#).

11. Database, disk and Docker

BASH / Display wraps; copy complete commands from the Markdown source.

```
systemctl status docker --no-pager
journalctl -u docker --since '30 minutes ago' --no-pager
df -h / /opt /var/lib/docker
df -i / /opt /var/lib/docker
docker system df
docker stats --no-stream
docker compose -f compose.prod.http.yaml logs --since 15m --tail=200 db
docker compose -f compose.prod.http.yaml exec db sh -c 'pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
```

Inspect disk/inodes/database before repeated restart. Never erase volumes/database files, globally prune Docker or delete business/audit history to free space. Review image/log/backup retention deliberately with protected off-server copies.

Safe runtime identity/privileges:

BASH / Display wraps; copy complete commands from the Markdown source.

```
docker compose -f compose.prod.yaml exec -T web python manage.py shell <<'PY'
from django.db import connection
with connection.cursor() as c:
    c.execute('SELECT current_database(), current_user')
    print('identity=', c.fetchone())
    c.execute('SELECT rolsuper, rolcreatedb, rolcreaterole FROM pg_roles WHERE rolname=current_user')
    print('superuser_createdb_createrole=', c.fetchone())
    c.execute("SELECT has_table_privilege(current_user, 'audit_events', 'SELECT'),
has_table_privilege(current_user, 'audit_events', 'INSERT'), has_table_privilege(current_user,
'audit_events', 'UPDATE'), has_table_privilege(current_user, 'audit_events', 'DELETE')")
    print('audit_select_insert_update_delete=', c.fetchone())
PY
```

Expect application role, role flags all false and audit privileges (True, True, False, False). Administrators retain controlled recovery powers; do not replace the restricted runtime role.

Error	Investigation
Password authentication	Existing-volume role password vs .env; preserve working credentials, use controlled recovery/rotation.
db unreachable	DB container/project/internal network; no need to publish PostgreSQL.
Missing relation	Image/schema/migration state; maintenance migration.
Table/sequence permission	db_setup after migration, distinct admin/runtime roles.
Exclusion conflict	Expected overlap guard; choose another slot, keep constraint.
No space	Filesystem/inodes; protect data before targeted cleanup.
Recovery/not ready	PostgreSQL/storage logs; allow normal recovery, investigate repeat failures.
Daemon unavailable	Start/check Docker; laptop Linux engine below.

Failed migrations/lost databases need a tested backup and controlled restore in [operations](#). Archive listing alone is not restoration proof.

12. Laptop problems (PowerShell)

Run local commands in the local repository, without production -f:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
Set-Location 'C:\Users\Dell\Documents\Codex\2026-09-28\files-mentioned-by-the-user-meeting\outputs\meeting
-room-booking'
docker version
docker compose version
docker info --format '{{.ServerVersion}}'
docker compose ps -a
docker compose logs --since 15m --tail=150 web worker db
Invoke-WebRequest http://127.0.0.1:8000/healthz/
```

Missing dockerDesktopLinuxEngine pipe means Docker Desktop's Linux daemon is not ready. Start Desktop, wait for running state, ensure Linux containers, rerun `docker info` before Compose. Check `ws1 --status/ws1 --list --verbose` if needed; do not reset/uninstall and discard data as the first fix.

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
# Normal restart after initial setup.
docker compose up -d db
docker compose up -d web worker
```

Use 127.0.0.1 consistently. Local file-mail links/codes appear at /dev/mail/; production has no such inbox. Seed demo rooms only into an empty local directory or add through staff UI; README has initial setup. Linux <<'PY' snippets belong on Linux; operations shows PowerShell here-string alternatives. Replace local log grep with Select-String.

13. Reports, audit and verification

Utilization counts scheduled meeting minutes for checked-in/completed bookings, against the **current** Mon-Fri policy/holiday calendar minus active closures. Reports include currently inactive rooms. Gaps, unchecked approved/pending, no-shows and cancellations are not used minutes. Historical rule changes can alter results; this is an operational estimate, not immutable past-policy reconstruction.

Excel requires staff access: use the report page link to /staff/reports.xlsx, review date order/range and safe logs. Formula-looking user text is literal in export. Compare [feature definitions](#) before comparing personal spreadsheet totals. Audit is append-only for runtime, with no UI delete/edit or automatic purge. GET page views are not a complete access audit.

Local automated checks

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
docker compose build
docker compose run --rm test
docker compose run --rm web python manage.py check
docker compose run --rm web python manage.py makemigrations --check --dry-run
```

Tests create/destroy a separate test database and need local administrator configuration. Never run with production credentials. [Verification report](#) records the earlier 210-test PostgreSQL pass and browser/restore rehearsals; these do not replace an office pilot.

Close an incident

Verify the original action on an affected device, readiness/heartbeat, current booking status and required email receipt. Record cause, remedy, commit/configuration version, time and remaining limitations. Review failed outbox before retrying; confirm backup/monitoring after recovery. A refresh or running container alone is not sufficient evidence.

8. Architecture

Source: docs/architecture.md

Transgate Tech | Binayak Bhandari

Status: Implementation verified with 281 PostgreSQL tests (8 October 2026) and isolated browser/deployment/restore rehearsals. The owner has installed company services on Oracle Linux 9.8 at 192.168.50.222 using `compose.prod.http.yaml` and `http://mbs.wdn.com.np`. HTTP was explicitly selected; no certificate is part of this installation. Actual device access, relay/mailbox delivery, records, backups and pilot acceptance must be checked on the company installation. Start with the [documentation index](#); schema is in [database design](#).

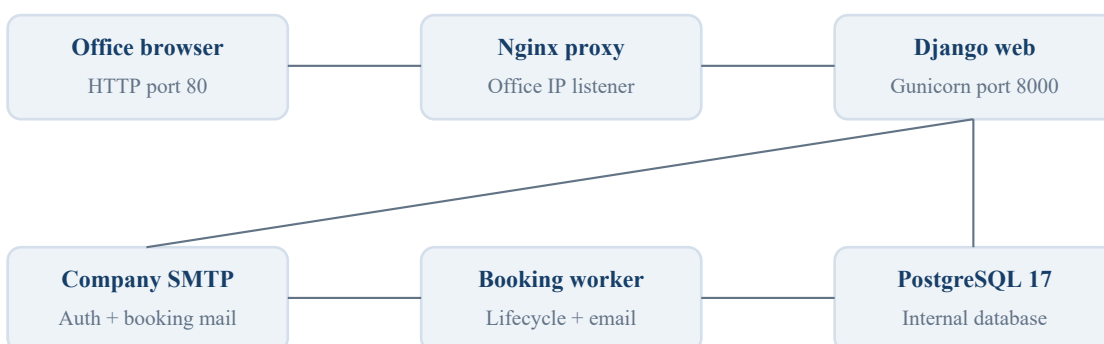
Sources: Meeting Room Booking System Requirements Questionnaire (23 September 2026) and the project owner's answers in this conversation, including the production approval workflow (7 October 2026) and configurable per-room approval (8 October 2026). The questionnaire supplies stakeholder requirements; the owner's answers resolve or amend them.

Purpose and launch scope

An internal web application lets approximately 90-95 employees reserve five company meeting rooms without overlapping reservations. It includes daily and weekly availability, searchable room details and facilities, optional authenticated room photos, one-time email sign-in, profiles, a three-step booking wizard with real whole-series availability, internal/external/mixed meetings, individual and recurring bookings, employee changes and cancellations, email check-in, automatic cancellation after a missed check-in deadline, Front Desk management, room closures, email notifications, basic reports with Excel export, and audit history. Staff have a month view and searchable, paginated booking/user/audit lists. Direct HCL/Outlook calendar integration will follow the first launch.

Bookings use Nepal time. Initial configurable defaults are Monday-Friday, 09:00-17:00 excluding configured holidays; 30-120 minute meetings; 15-minute increments/gap; and a 14-day horizon including recurrence. Check-in is allowed from start until before the current deadline, initially 15 minutes later. Staff policy changes are audited. History has no automatic purge; the requested minimum one-year retention and company backup/retention arrangements remain operational decisions.

Architecture



Simplified relationships/components. The linked Markdown source includes the full Mermaid diagram.

Single application. Django renders employee and staff screens and enforces permissions and booking rules. One repository contains the web app and worker. PostgreSQL holds application records, sessions, and pending notification jobs. This is an appropriate starting architecture for five rooms and approximately 95 employees; capacity still needs pilot verification.

Booking interface and preview. Server-rendered fields remain the authoritative POST contract. JavaScript stages Schedule & details, Attendees and Review & confirm, derives duration/start choices from current policy, and fetches a read-only authenticated availability endpoint. Preview and creation share schedule/recurrence expansion, including holiday skips and stored occupied intervals; edit excludes only the authorized occurrence. The endpoint returns room metadata and occupied ranges without other meeting titles/emails/notes. Selecting a room does not reserve it; final writes recheck under database locks. Native form fallback retains the same server validation when JavaScript is unavailable. External and mixed meetings require guest company information.

Room photos. Verified staff upload, replace or remove optional still photos through room management. Files are decoded/re-encoded as bounded JPEGs with generated names; supplied metadata is discarded. Authenticated delivery serves active room photos to employees and inactive rooms to verified staff, with no public media server. Database stores filenames; the web-only named `room_media` volume stores bytes under `/app/media/rooms/photos`. Normalized upload files belong to UID/GID 1000. Replaced/removed files are deleted after the room transaction commits. Recovery pairs a database snapshot with its photo archive.

One conflict model. PostgreSQL is confirmed. Both meeting occurrences and maintenance closures reserve time through the same database relation so that one exclusion constraint protects against conflicts between either type. The shared write path locks the room row, checks availability, and writes in a transaction; a conflicting concurrent write returns an availability error. Later staff workflows must use that same write path. PostgreSQL documents exclusion constraints specifically for non-overlapping room reservations. [PostgreSQL range constraints](#)

Time and buffers. Store timezone-aware timestamps as UTC instants and apply working hours and recurrence rules in Asia/Kathmandu. Store meeting times separately from the occupied interval. Count the 15-minute gap once: a meeting from 10:00 to 11:00 reserves the room until 11:15, so another meeting may start at 11:15. Front Desk can add a room closure for extra preparation time. A rule change does not rewrite existing occupied intervals.

Room approval policy. `Room.requires_approval` defaults to true, including existing rooms after migration 0011. Verified staff manage it through the room list checkbox or add/edit form. Employee creations and substantive edits in a checked room become Pending; in an unchecked room they become Approved automatically. Both states participate in the same occupancy/exclusion constraint as closures. Saving reads the selected room's current policy under its row lock; browser flags cannot select a status or bypass rules. Automatic approval has a timestamp but no fabricated reviewer. Staff creations are Approved; staff edits retain Approved, while substantive edits to Pending in an unchecked room can confirm it. Unchanged saves and room-policy toggles leave saved statuses intact.

Staff review. Verified staff approve a future Pending request or reject it with a visible reason. Both transitions lock the current room and reservation, revalidate state and permissions, update audit metadata, and queue mail atomically. Rejected/cancelled requests release occupancy. Pending requests expire at the meeting start and cannot check in. Approval cannot revive rejected/cancelled/expired records. Recurring creations use the room policy for every occurrence; Pending occurrences are reviewed separately. Turning off review does not bulk-approve the existing queue or demote approved records when turned back on.

Finite recurring bookings. Expand a recurring request into actual occurrences inside the two-week horizon, with a shared series identifier. Initial series creation is one transaction: either all requested occurrences are saved or the user receives an availability error. Later monthly dates are booked manually, as agreed. A monthly pattern within two weeks ordinarily produces only one occurrence. Recurrence does not require an ongoing process to reserve future dates.

Manual priority. Front Desk will decide when senior management receives priority for the WDN third-floor room. If staff must move or cancel an existing booking, that change must be audited and the affected organizer notified. Staff cannot bypass the database conflict constraint and leave two active bookings for the same room and time.

Email identity and privacy. Employee links use `wdn.com.np` or `transgate.com.np`; token records and pending session values contain hashes. Organizer check-in links are generated for outgoing mail during delivery, not persisted in notification body snapshots. Outbox records still contain sensitive recipient/meeting content; restrict database/backup access. Safe logs omit private links and request values. Database throttles access to account/IP; only the configured

proxy supplies trusted IP headers. Login and check-in tokens have separate purposes. A check-in token authorizes only its occurrence and needs no prior employee login; edits/cancellation invalidate old links. Other employees cannot see titles, purpose, attendees or notes. Detail/change endpoints enforce ownership or verified staff access.

Safe use of email links. Opening a link displays a confirmation page; a deliberate button press performs login or check-in using a protected POST request. Merely fetching or previewing a URL must not consume the token or check in a meeting. This also reduces accidental actions from email link inspection. Django requires safe HTTP methods such as GET to be free of state-changing actions. [Django CSRF guidance](#)

Staff privileges. Front Desk and Administrators have identical permissions through one staff role, with individual named accounts for auditability. Staff sign-in requires a password and an eight-digit one-time code sent to the company mailbox. Guess counts are persisted and locked, and codes are bound to the current password/access version. If a staff member uses the employee email-link route, that session has employee capabilities only. Staff endpoints check the stronger session flag and current authentication version. Access changes invalidate all of the affected person's sessions and pending sign-in challenges, including after deactivation/reactivation. Profiles may change only names and department.

Email delivery. Reservation changes, audit and outbox jobs commit together. Worker delivers booking events and one-hour reminders; pending-request/rejection notices omit attendees. Eligible organizer confirmation/change/reminder mail includes check-in links. Failures allow five attempts; leases recover interruptions. Current state/revision supersedes obsolete jobs. Disabled SMTP preserves retries and blocks authentication email. Backend acceptance does not prove inbox arrival, and ambiguous outcomes can duplicate mail. Selected relay is maildc01.wdn.com.np:25, sender mbs@wdn.com.np, no relay AUTH/TLS/SSL; verify the deployed source and actual receipt. See [email operations](#).

Check-in and automatic release. Check-in/release use transactional locks and current-state checks. After the deadline an unchecked approved booking becomes no_show, releases occupancy and records audit/no-show mail together. History remains; late check-in fails even before reconciliation runs. Pending requests cannot check in and expire at their start.

Worker. One Docker-managed Django worker reconciles every 30 seconds: it cancels expired pending requests, releases overdue no-shows, marks finished meetings, and queues reminders. It sends one due email at a time between reconciliations, with a ten-second SMTP timeout. Booking writes also reconcile expired pending requests and overdue no-shows. Calendar GET requests retain occupied intervals until the database actually releases them and mark past, closed, holiday, or out-of-window starts unavailable. Release happens on the next successful reconciliation, with a slow send adding up to one timeout. Worker success is recorded in a singleton heartbeat. Readiness requires a successful cycle within two minutes; IT monitoring must alert on stale heartbeats and unhealthy containers.

Deployment and recovery. One company server hosts Nginx, Gunicorn, worker and PostgreSQL. Selected HTTP publishes only 192.168.50.222:80; an alternative HTTPS profile remains available for a future planned change. Both force production safeguards. App services share one image; web/worker are non-root with read-only application filesystems, and web mounts a writable persistent photo volume. Web/DB publish no host port. Migration/setup use administrator credentials; runtime has no superuser, database/role/schema creation or audit update/delete privileges. Nginx resolves recreated web addresses through Docker DNS. IT restricts office/VPN access. SMTP failure affects authentication/check-in-link delivery; staff can manually check in within the valid window. [Operations/recovery](#) covers backups, restore drills and updates.

Browser and operational safety. Forms use CSRF and escaped templates. Session cookies are HttpOnly/SameSite Lax; Secure is transport-dependent and false in the selected HTTP profile. Same-origin referrer policy supports native form origin checks. Content security policy, frame denial and no-store caching apply; Excel treats user text literally. Branded safe errors/logs omit secrets; unavailable databases return 503. Rooms prefetch facilities, calendars group reservations and management lists paginate. The admin interface is custom /staff/, with no configured /admin/ route.

Reports. Utilization clips checked-in/completed meeting intervals to the configured working calendar and subtracts the union of room closures. Cancelled/no-show meetings and preparation gaps do not count as used meeting time.

Results use the current holiday/policy settings, so historical changes can alter the denominator. Reports are operational estimates, not historical policy reconstruction.

The single server is a shared point of failure. IT must agree acceptable downtime and data loss, provide scheduled backups on another system, and test restore before rollout. Monitor application health, scheduled-job freshness, failed mail, database/storage availability, and backup completion. This provides a recoverable initial installation; uninterrupted service would require additional infrastructure. Pin supported software versions, keep migrations in source control, and document upgrades and recovery.

Main user journeys

1. An employee requests an email sign-in link, verifies ownership of the company address, then follows Schedule & details -> Attendees -> Review & confirm. The wizard checks every proposed occurrence and room capacity before submission. The slot is rechecked when saving; selection alone does not reserve it.
2. Saving rechecks policy/occupancy atomically. A checked room creates Pending employee holds/request notices; staff review each occurrence. An unchecked room creates Approved employee meetings/confirmation immediately. Staff-created bookings start Approved for either policy. Manual approval also queues confirmation; rejection requires a reason and frees occupancy.
3. For a recurring series, the application expands occurrences within the two-week window, checks every occurrence, and commits them in one transaction if every slot is valid. The organizer books later monthly occurrences manually; the system will not automatically reserve dates beyond two weeks.
4. The organizer opens the private email link and presses the check-in button before the configured deadline (initially 15 minutes after start). An unchecked Approved meeting becomes No show and releases occupancy; history remains.
5. Front Desk staff see full booking and preparation details and may manage bookings and room closures. Employees see only limited information about others' bookings.

Current installation and remaining decisions

1. Employee name/department source, official holiday calendar, five room records, and exact WDN third-floor priority procedure. Employee email addresses must use wdn.com.np or transgate.com.np. Front Desk will handle room priority manually; the workflow still needs definition.
2. Server OS/IP/domain and HTTP choice are known; the owner supplied Docker startup/health logs. The selected relay previously accepted a test. IT still verifies deployed-server relay permission/mailbox receipt, device DNS/routing, backup destination/retention/recovery objectives and monitoring. Staff password plus email code is approved. No certificate is required for selected HTTP.

Acceptance checks before rollout

The pilot must demonstrate verified employee ownership; authorization of changes and private details; prevention of staff-MFA bypass through employee login; pending, approved, rejected, cancelled, and recurring bookings; simultaneous requests for one slot; conflicts between bookings and room closures; exact buffer boundaries; email confirmation/change/cancellation/reminder; failed email retry without lost bookings; safe email-link previews; check-in racing against cancellation or rescheduling; scheduled-job restart and recovery; Front Desk operations; accurate basic reports; audit records; and a tested database restore. Representative concurrent usage must be tested on the chosen deployment configuration. IT and management will review the results before company-wide rollout.

See [features/use cases](#) for business definitions/limits and [verification evidence](#) for tests versus office acceptance. No production credentials are included in this repository.

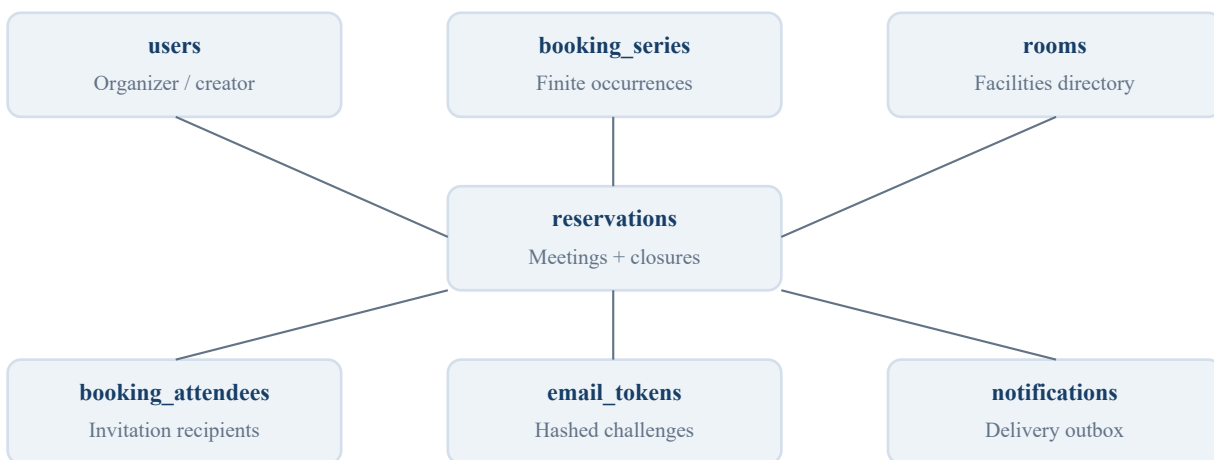
9. Database design

Source: docs/database-design.md

Transgate Tech | Binayak Bhandari

The application uses PostgreSQL. Each meeting occurrence and room closure reserves time in the same reservations table, allowing one database constraint to reject overlapping occupied periods for a room. All displayed booking times use Asia/Kathmandu; timestamps are stored as timezone-aware instants. Booking screens, email delivery, check-in jobs, and staff workflows use this schema.

ERD



Simplified relationships/components. The linked Markdown source includes the full Mermaid diagram.

Photo files and database boundaries

rooms.photo stores an optional generated filename, not image bytes. Normalized JPEG files live under /app/media/rooms/photos in the web container's persistent room_media volume. Photo replacement/removal is tied to the database commit; after successful replacement the former file is removed. Migration 0010_room_photo adds this optional field without seeding/resetting existing rooms. Database backups must be paired with photo archives from the same recovery point; a database dump alone cannot restore file contents. See [photo backup and recovery](#).

Tables and keys

Table	Key fields and relationships	Purpose
users	id PK; email unique including case-insensitive constraint; is_staff, is_active, auth_version; name and department	One identity per company address. Authentication version participates in the session hash and invalidates existing sessions when access changes.
rooms	id PK; unique (location, floor, name); capacity, description, instructions, optional photo filename, active flag, requires_approval default true	Company-configured rooms managed through staff controls; production does not seed sample records. Inactive rooms cannot receive new bookings.

Table	Key fields and relationships	Purpose
room_facilities	id PK; room_id FK; unique (room_id, name)	Room equipment and facilities. A simple room-specific list is sufficient for five rooms.
booking_series	id PK; room_id, organizer_id, created_by_id FKs; frequency, first/last dates	Groups the actual occurrences of a daily, weekly, or monthly request within the two-week advance window. It is not a promise to reserve later dates.
reservations	id PK; room_id FK; optional series_id, organizer_id, created_by_id FKs; kind, status, meeting type, guest company, approval/rejection actor/timestamp/reason, revision, actual start/end, occupied start/end	Shared schedule for meetings and staff room closures. A meeting has organizer and title; a closure has a reason. Rejected, cancelled, and no-show meetings remain in history but release occupancy.
booking_attendees	id PK; reservation_id FK; unique (reservation_id, email)	Email recipients and attendee count. Only meeting reservations may have attendees.
email_tokens	id PK; optional user_id, reservation_id FKs; token hash unique, purpose, expiry, consumed time, attempts	Short-lived employee login, staff email code, or per-occurrence check-in. This table and pending session values store hashes. Staff challenge guesses are counted under a row lock.
authentication_throttles	id PK; unique (purpose, key); window start, attempts	Atomic account/IP limits with HMAC identity keys and indexed request windows.
notifications	id PK; optional reservation_id FK; recipient, event type, reservation revision, content snapshot, status, attempts, next attempt, lease UUID/expiry	Database outbox for email delivery and retries. Snapshots contain sensitive meeting/recipient data; restrict access. Check-in links are generated for outgoing delivery, not persisted in snapshots. Status distinguishes pending/sending/sent/failed/skipped (Superseded). Leases recover interrupted delivery.
audit_events	id PK; optional actor_id FK; action, target, outcome, timestamp, safe details	Append-only application history, including system actions. No passwords, raw tokens, or SMTP secrets.
booking_policy	Singleton id=1 PK; working hours, limits, slot size, gap, check-in deadline; optional updated_by_id FK	Configurable company-wide booking rules. Application logic validates rules on every write and records staff changes.
company_holidays	id PK; date unique, name	The official non-bookable working calendar, with staff exceptions recorded on affected reservations.
worker_heartbeat	Singleton id=1 PK; last successful cycle and last error timestamps	Worker readiness and restart/failure monitoring without exposing exception details.
Django session/auth tables	Django-managed session key/expiry/data and inherited authentication relations	Database sessions with hashed pending tokens; passwords use Django's password hashers. No external SSO or public REST API is required.

There is no separate `roles` table because the confirmed application has exactly two permission levels: employee and staff. Django's `is_staff` flag identifies the shared Front Desk/Administrator level. Its built-in superuser flag is reserved for technical administration. A separate role catalog can be introduced if the business later requires distinct staff permissions.

Reservation state and validation

`kind=booking` permits `pending`, `approved`, `rejected`, `checked_in`, `completed`, `cancelled`, or `no_show`. `kind=block` permits `blocked` or `block_cancelled`. The database checks the `kind/status` pairing and requires the appropriate meeting or closure fields. Meeting start precedes end, and the occupied interval contains the meeting interval. A normal meeting reserves from `starts_at` through `ends_at` + configured gap. A closure uses its own specified interval. Pending and approved requests hold occupancy. Checked-in/completed meeting occupancy remains in historical conflict checks; rejected/cancelled/no-show/withdrawn closures release it. Rejection requires a nonempty reason. Approval/rejection metadata and a reservation revision record review and invalidate stale queued mail. Policy bounds, room capacity, series date ordering, and token purpose/target also have database checks.

The first migration enables PostgreSQL's `btree_gist` extension. An exclusion constraint on (`room_id` equality, `tstzrange(occupied_from, occupied_until, '[')` overlap) applies to active statuses. This rejects concurrent double booking and conflicts between a booking and a closure. Adjacent intervals may meet at the boundary, so a meeting ending at 11:00 blocks through 11:15 and the next meeting may start at 11:15. The application also locks the room row, checks availability, and writes in one transaction, making simultaneous requests for the same room wait their turn. The database constraint remains the final guard. All later edit, cancel, check-in, and no-show workflows must use the same transaction discipline. A conflict is shown to the user as an availability error.

The room approval flag is not a booking status: changing it does not bulk-update reservations. New employee requests/substantive edits use the selected room's current flag under a row lock. Automatic approval records `approved_at` with `approved_by=NULL`; staff creation/manual review records the staff actor. Unchanged saves preserve the existing status/metadata. Approval policy does not relax capacity, conflicts, ownership or date validation.

Rules depending on the current policy, room activity, official holidays, staff overrides, allowed email domains, and attendee ownership cannot be fully expressed as simple row constraints. The application enforces them server-side. Staff activity and rule overrides are audited.

Indexes and deletion

The exclusion constraint supplies the room/time search index. Explicit indexes cover reservation starts by room/organizer/status and pending creation time, token expiry and requests, due notifications, throttle windows, and recent audit actions by actor. Foreign keys protect users/rooms referenced by booking history; deactivate instead. History is retained, with no automatic purge. IT defines retention and maintenance of expired sessions/throttle windows under company policy.

Production `db_setup` grants runtime data access using a separate `NOSUPERUSER`, `NOCREATEDB`, `NOCREATEROLE` role and revokes schema creation. Audit records allow `SELECT/INSERT` only; `UPDATE/DELETE/TRUNCATE` are denied to runtime. There is no audit edit/delete application endpoint. Database administrators still have privileged recovery access and must be controlled separately. Reapply runtime permissions after migrations.

Migration and verification strategy

Numbered Django migrations are committed with the code and applied by a separate deployment step after a database backup. They are never generated on the production server. Schema changes must preserve existing history; destructive changes require a reviewed data migration and restore plan. Tests run against PostgreSQL, including overlap, adjacency, room closure conflict, cancellation release, concurrent booking attempts, and the employee email-link flow.

Migrations 0001-0004 are preserved. Additive migrations 0005-0007 add policy bounds, authentication versions/throttles/guess counts, and notification leases/worker heartbeat. The complete chain is tested on an empty PostgreSQL database. Existing invalid policy values must be corrected before applying its new constraint; migration failure leaves the prior schema intact. An isolated production rehearsal also verifies restricted runtime permissions and actual dump/restore record counts.

SMTP/email credentials must be supplied by the system administrator during production deployment. No production email credentials are included in this repository.

Approval migration

Migration `0008_booking_approval` runs in one PostgreSQL transaction: it replaces status/exclusion checks, maps existing confirmed bookings to approved, and installs the occupancy constraint with pending requests included. Existing approved records do not receive fabricated approval actors/timestamps. Notification snapshots start at revision 1. Each subsequent change increments the revision. Runtime permissions must be reapplied after migration.

Migration `0009_mixed_meeting_type` adds Internal + External (mixed) and extends the guest-company constraint: both external and mixed meetings require a company. Existing internal/external records are retained; reversing 0009 maps mixed to external while retaining company information. Migration `0010_room_photo` then adds the optional room photo field. Migration `0011_room_requires_approval` adds the non-null Boolean room setting with default true: all existing rooms retain their review requirement, and saved reservation statuses/history are not rewritten. Apply the committed chain during the supported upgrade; do not create migrations or reseed rooms on the server. Removing the photo field during rollback does not constitute a file-volume backup or recovery.

Take a backup and stop writers before an update. Reversing 0008 maps approved back to confirmed and pending/rejected to cancelled; it releases unapproved requests rather than silently confirming them. Unsent request/rejection emails are superseded during reversal because the old worker does not support those states. Rollback changes request status and requires a reviewed restore plan; stop web and mail workers before changing either code or schema.

10. Laptop setup and development

Source: README.md

Transgate Tech | Binayak Bhandari

An internal meeting room application for employees with `wdn.com.np` or `transgate.com.np` email addresses. Django renders the employee and staff interface; PostgreSQL stores records, sessions, notification jobs, and audit history. A worker sends email and reconciles reminders, completed meetings, and missed check-ins. Docker Compose runs the application, worker, and persistent PostgreSQL database; production adds an Nginx proxy on the company network. Separate deployment profiles support HTTPS and the explicitly selected private-network HTTP installation.

The interface uses the Transgate navy/red palette and logo, with Times New Roman as approved. Front Desk and Administrators have identical staff permissions. Staff use a password plus a company-email code; employees use a company-email link.

Complete documentation

Start with the [documentation index](#). Detailed guides cover all features and use cases, employee and staff tasks, server operations, backups/restoration, and troubleshooting with log/diagnostic commands. A printable [complete handbook](#) accompanies the Markdown guides. [Build instructions](#) explain regeneration.

Features

- Room search, details, capacity/facilities, optional staff-managed real photos, and availability calendar.
- Three-step booking flow: Schedule & details -> Attendees -> Review & confirm, with real room availability for every selected recurring occurrence. Native form fallback keeps booking usable without JavaScript.
- Internal, External and Internal + External meetings; guest company is required for external/mixed meetings.
- Own bookings and profile; booking requests, modification, and cancellation.
- Per-room approval: checked **Requires approval** rooms create Pending employee requests; unchecked rooms confirm them immediately. Pending requests reserve the slot; staff approval confirms the meeting, rejection records a reason and releases the slot.
- Daily, weekly, and monthly recurrence within the two-week advance window. Later monthly dates are reserved manually.
- Business hours, holidays, meeting duration, time slots, and room gap enforced on the server. PostgreSQL exclusion constraints prevent conflicting concurrent reservations, including room closures.
- Request, approval, rejection, modification, cancellation, reminder, check-in, and no-show notifications for the organizer, attendees, and staff.
- Organizer check-in links in confirmation, change, and reminder emails. Check-in opens at the start; the worker releases missed check-ins after 15 minutes.
- Staff room management, closures, employee/access management, booking on behalf of employees, reasoned rule overrides, audit history, statistics, and Excel export.

Direct HCL/Outlook calendar integration remains a later phase. WDN third-floor senior-management priority is handled by Front Desk manually.

Stack and requirements

Python 3.12, Django 5.2 LTS, PostgreSQL 17, Gunicorn, WhiteNoise, openpyxl, and Nginx. Install Docker Desktop with the Linux engine on a laptop, or Docker Engine plus the Compose plugin on a Linux server. Docker handles Python and application dependencies; a separate local Python installation is optional.

Local setup

In PowerShell, from this repository, create a fresh `.env` with generated secrets:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
docker run --rm -v "${PWD}:/workspace" -w /workspace python:3.12-slim python deploy/create-env.py
docker compose build
docker compose up -d db
docker compose run --rm migrate
docker compose up -d web worker
Invoke-WebRequest http://127.0.0.1:8000/healthz/
```

The environment generator refuses to replace an existing `.env`. For an existing installation, preserve its database password and Django secret, review `.env.example` for new variables, then build and migrate. A fresh worker heartbeat may take a few seconds before readiness returns HTTP 200.

For an existing demo, back up its database first and stop both writers before migrating to `mbs-prod`. Do not run old demo code against the approval schema:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
git switch mbs-prod
git pull --ff-only
docker compose stop web worker
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc -f
/tmp/pre-mbs-prod.dump'
docker compose cp db:/tmp/pre-mbs-prod.dump ./pre-mbs-prod.dump
docker compose build
docker compose run --rm migrate
docker compose up -d web worker
```

Protect the dump; backup archives are Git-ignored. Preserve `.env`, its database password, Django secret, database volume and optional `room_media` volume. A database dump does not contain uploaded photos: back up/restore that volume alongside the matching database snapshot. Existing confirmed meetings become approved. Current upgrades apply `0009_mixed_meeting_type`, `0010_room_photo` and `0011_room_requires_approval` after `0008`; they preserve existing rooms/bookings and do not seed rooms. Migration `0011` sets Requires approval to checked for existing rooms, preserving their review requirement. See [photo backup/recovery](#) before updating an installation with uploads.

Open `<http://127.0.0.1:8000/>`. Local `.env.example` explicitly selects `DJANGO_EMAIL_BACKEND=file`; messages are written to a Docker volume and appear in the development inbox at `<http://127.0.0.1:8000/dev/mail/>`. This local convenience does not verify mailbox ownership. It is unavailable in either production profile and must never be used for company rollout.

Optional demonstration rooms, only in an empty local directory:

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
docker compose exec web python manage.py seed_demo_rooms
docker compose exec web python manage.py create_staff_account your.name@wdn.com.np
```

Open the staff setup link in the development inbox, set a password, and use `/staff/sign-in/`. Its email code appears in the same development inbox. Actual room data is entered through the staff interface.

Stop containers and preserve data with `docker compose down`. Restart with `docker compose up -d web worker`. Database, optional room photos (`room_media`) and local email persist in named volumes. Never add `--volumes` to the stop command unless intentionally deleting the data.

Environment and email

`.env.example` describes local configuration. `.env.production.example` describes HTTPS deployment; `.env.production.http.example` describes private-network HTTP deployment. Actual `.env*` files, certificate keys, and backup archives are excluded from Git and Docker build contexts.

Variable	Purpose
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Database name and bootstrap/migration administrator. Preserve these for existing volumes.
POSTGRES_APP_USER, POSTGRES_APP_PASSWORD	Production application role with database data access and no superuser, role-creation, or database-creation privileges.
DJANGO_SECRET_KEY	Independent generated secret; production requires at least 50 characters. Preserve across updates.
DJANGO_ALLOWED_HOSTS	Allowed hostnames. Production proxy expects one internal hostname without scheme/port.
DJANGO_PUBLIC_BASE_URL	Matching origin used in email links, e.g. <code>https://meetings.company.invalid</code> or <code>http://mbs.wdn.com.np</code> for the selected HTTP profile.
DJANGO_PRODUCTION	Both production Compose profiles force true; development inbox and demo seeding remain disabled independently of transport.
DJANGO_HTTPS	Secure cookies and HTTPS redirect. <code>compose.prod.yaml</code> forces true; <code>compose.prod.http.yaml</code> forces false.
HTTP_BIND_IP	HTTP profile only: required server office-interface IP. For this installation, <code>192.168.50.222</code> .
EMPLOYEE_EMAIL_DOMAINS	Comma-separated local employee domains; defaults to <code>wdn.com.np, transgate.com.np</code> . Production allows these two domains.
DJANGO_EMAIL_BACKEND	<code>file</code> for local HTTP development, disabled while unconfigured, or <code>smtp</code> for the approved relay.
AUTH_TRUSTED_PROXY_CIDRS	Networks/IPs allowed to supply X-Real-IP; production trusts only its fixed proxy IP.
PROXY_NETWORK_SUBNET, PROXY_NETWORK_DYNAMIC_RANGE, PROXY_IP	Production proxy network; keep the fixed proxy outside the automatic address range and update the trusted CIDR consistently if changed.

Example files intentionally omit relay credentials. The selected office configuration is documented in the HTTP installation/runbook: `maildc01.wdn.com.np:25`, sender `mbs@wdn.com.np`, no relay AUTH, TLS/SSL false. Disabled mode allows startup, logs a configuration warning, preserves queued notifications without consuming retries, and rejects sign-in email delivery. Selecting `smtp` without host/sender also disables delivery with a warning. No message is marked sent unless the backend accepts it. The local file mode records a message locally and performs no external delivery.

SMTP variable	Deployment value
SMTP_HOST	Company SMTP relay hostname/IP.
SMTP_PORT	Approved relay port, commonly 587 for STARTTLS or 465 for implicit TLS.

SMTP variable	Deployment value
SMTP_USER, SMTP_PASSWORD	Relay credentials if authentication is required; leave empty only when IT approves an IP-authorized relay.
DJANGO_FROM_EMAIL	Approved sender address/name.
SMTP_USE_TLS	true for STARTTLS.
SMTP_USE_SSL	true for implicit TLS; never enable simultaneously with STARTTLS.

After supplying these, set `DJANGO_EMAIL_BACKEND=smtp` and recreate web/worker. Synchronous employee/staff authentication reports unavailable delivery without granting a session. Notification failures keep bookings intact, retry up to five attempts, and remain visible to staff. SMTP delivery acceptance does not guarantee inbox arrival; IT verifies relay and mailbox behavior during pilot testing. An ambiguous SMTP result may cause duplicate delivery after a retry.

Verification and builds

POWERSHELL / Display wraps; copy complete commands from the Markdown source.

```
docker compose build
docker compose run --rm test
docker compose run --rm web python manage.py check
docker compose run --rm web python manage.py makemigrations --check --dry-run
```

Tests require PostgreSQL and cover email configuration/failure, secure authentication, authorization, room management, booking changes, concurrent conflicts, recurrence, check-in, worker health, and page rendering. The test runner creates and destroys a separate test database; use local configuration rather than production credentials.

Development tools in `requirements-dev.txt` provide `ruff check .` and `ruff format --check .`; CI runs these alongside PostgreSQL tests, migration checks, Django checks, and the Docker build. Python syntax can also be checked with `python -m compileall -q booking config`. No separate static type checker is configured for this Django project. Run `pip-audit -r requirements.txt` to check the pinned application dependencies against public vulnerability advisories.

Builds collect and fingerprint static assets, then run Gunicorn as a dedicated non-root account. The production web/worker containers use a read-only filesystem and temporary `/tmp`; web has a writable named `room_media` volume at `/app/media` for optional room photos (application UID/GID 1000). The [HTTPS deployment guide](#) includes clean setup, restricted runtime database permissions, migrations, initial staff creation, TLS, verification, backups, and restore. For the selected `mbs.wdn.com.np` installation on Oracle Linux, follow the [private-network HTTP guide](#). It uses port 80 and requires no certificate or private CA; browsers show **Not secure** because browser traffic is unencrypted.

The [verification report](#) records the production-branch test results, browser checks, deployment rehearsal, security fixes, and remaining deployment checks.

Approval and booking lifecycle

Staff select **Requires approval** per room in **Staff desk -> Rooms** or the room add/edit form; it defaults to checked. Employee submissions for checked rooms start **Pending** and hold the room/time, including the configured gap. Front Desk or Administrators open **Pending requests**, review details, and approve or reject. Rejection requires a reason visible to the requester. Employee submissions for unchecked rooms start **Approved** immediately and queue confirmation/check-in instructions. Capacity, availability and booking rules apply to both paths. Manual approval requires a verified staff session.

The wizard checks date/time/recurrence availability without creating a reservation and submits one server-validated request at the final step. A selected available card is not a hold; final saving rechecks room activity, capacity, schedule and conflicts.

Staff-created bookings are **Approved** immediately. A substantive employee edit follows the selected room's current policy: checked means **Pending**; unchecked means **Approved**. Staff edits retain Approved; a substantive staff edit to Pending can confirm it automatically only when the selected room is unchecked. An unchanged save preserves the existing state. Toggling a room does not rewrite saved bookings, so existing Pending requests remain Pending until review or a substantive eligible edit. Each recurring occurrence follows its room's policy and is reviewed separately when Pending. Owners and staff may cancel eligible bookings; cancelled and rejected records remain in history but release occupancy. Pending requests that have not been approved by the meeting start are cancelled automatically. Approval cannot revive rejected, cancelled, or expired records. Approved meetings proceed through **Checked in**, **Completed**, or **No show** after a missed check-in deadline.

Operations and limits

`/livez/` checks the web process. `/healthz/` checks PostgreSQL and a worker heartbeat from the last two minutes, returning HTTP 503 when either is unavailable. The endpoints expose no credentials or configuration. Docker checks web, worker, proxy, and database. An unhealthy container needs operator attention; Docker restart policies restart exited processes, not merely unhealthy ones.

The worker reconciles approximately every 30 seconds and sends one message at a time with a ten-second SMTP timeout. A slow send can delay the next reconciliation by that timeout. Failed jobs and interrupted delivery leases recover after restart; messages whose reminders have become obsolete are labelled superseded rather than sent.

After SMTP is repaired, `python manage.py retry_failed_mail --all` requeues eligible failures that exhausted retries; `--id NUMBER` targets one. It supersedes obsolete booking messages and records the action in audit history.

This remains a single-server application. The owner has started company services on the selected HTTP installation. IT must verify employee-device DNS/routing, deployed-server relay authorization/mailbox receipt, representative usage, health monitoring and tested off-server backups. Certificate monitoring applies only if the alternative HTTPS profile is adopted. The agent's isolated test/rehearsal evidence does not replace company pilot acceptance.

Guides: [all documentation](#), [features/use cases](#), [operations](#), [employees](#), [staff](#), [troubleshooting](#), [architecture](#), and [database design](#).

SMTP/email credentials must be supplied by the system administrator during production deployment. No production email credentials are included in this repository.

11. Verification evidence

Source: docs/verification-report.md

Transgate Tech | Binayak Bhandari

Delivery and scope

Production work is on mbs-prod, created from clean main at 22caa31. Earlier production review preserved the existing demo branch, running containers, database, SMTP settings and company accounts. The 8 October local upgrade is recorded separately below. The final commit IDs and remote push result are reported in the handover message. This report covers locally verified code and isolated deployment rehearsals. The owner has separately installed the company HTTP services; these local checks do not establish company-server acceptance.

Inspection covered every URL pattern, templates/static assets, forms, identity and staff permissions, model relations/constraints/indexes, migrations, booking services, recurring occurrences, calendar, worker/outbox, reports/Excel, audit history, configuration, Docker/TLS/runtime database permissions, seed/bootstrap and backup tools. This application serves HTML forms, authenticated read-only availability JSON and readiness/liveness JSON; it has no public REST API.

Approval workflow

- Employee creation in a room with **Requires approval** checked produces **Pending**; an unchecked room produces **Approved** automatically. Pending requests hold the complete occupied interval and gap through the PostgreSQL exclusion constraint. New/previously configured rooms default to checked in migration 0011.
- Verified staff review a dedicated queue and approve a future request or reject it with a required reason visible to its requester. Approval/rejection actors and timestamps are stored and audited.
- Staff creation is Approved. Employee substantive edits follow the selected room's current setting: checked -> Pending; unchecked -> Approved automatically. Staff edits retain Approved; substantive edits to Pending in unchecked rooms can confirm automatically. Previous links/notifications are invalidated after substantive changes. Unchanged edits and room toggles preserve saved statuses.
- Rejected/cancelled records release their slot and remain in history. Invalid transitions cannot revive them. Unapproved requests expire as cancelled at the meeting start; approved meetings use the existing check-in/completed/no-show lifecycle.
- Recurrence remains atomic and finite inside the two-week horizon; each occurrence follows its room policy and is reviewed separately when Pending. Front Desk handles room priority manually.

Reproduced failures and fixes

Defect	Evidence and fix
Idle TCP connections causing HTTP 500 before Django	Two sync Gunicorn workers with timeout 3 seconds and two idle sockets returned two HTTP 500s, with <code>WORKER TIMEOUT</code> and no <code>URI read</code> . The same five-second probe with four threads per worker returned no 500/timeouts; subsequent requests returned 200. Deployment now uses gthread and disables the unused control socket for read-only startup. A regression exercises the actual Docker CMD against an isolated WSGI fixture.

Defect	Evidence and fix
Database session load/save failure	Injected failures during staff sign-in reproduced the recovery path. Outer safety middleware and database-free error rendering now return safe 503, preserve the session cookie, and recover to rooms 200 after the dependency recovers. Unexpected exceptions still return 500 and retain safe diagnostics.
Malformed URL input	Unicode numeric filters previously reached <code>int()</code> and raised <code>ValueError</code> ; invalid UTF-8 password UID raised <code>UnicodeDecodeError</code> . Bounded ASCII checks and decoding guards now return safe pages/errors.
Calendar room activation race	A room activated between the room and reservation queries could cause a <code>KeyError</code> . Reservations now use the same room-ID snapshot; a regression reproduces activation between the queries and verifies load and refresh.
HTTPS public-port CSRF failure	Actual HTTPS POSTs on port 9444 failed because proxy Host omitted the public port. Django now trusts exactly the validated HTTPS public origin. Valid origin/referer/token POSTs pass; wrong/null origins and missing tokens remain rejected. Final real HTTPS approval/rejection passed.
Native browser form CSRF failure	Company GET returned <code>Referrer-Policy: no-referrer</code> ; Chrome native employee submission reproduced 403 while a scripted request with explicit correct Origin passed 200. <code>no-referrer</code> causes native form POSTs to serialize Origin as null. The application now uses <code>same-origin</code> ; native Chrome employee and staff submissions passed against an isolated production HTTP instance. Null/wrong origins and missing/invalid tokens remain denied.
Approval/access and stale mail races	Sorted user locks precede room/reservation locks and revalidate current role/activity. Concurrent decisions commit one outcome; organizer deactivation prevents waiting approval. Reservation revisions supersede stale mail, and same-time reapproval can queue a fresh reminder.

The user reported intermittent staff sign-in/rooms errors that cleared after refresh, without a timestamp or screenshot. Available demo logs contained startup lines only. The defects above were reproduced and fixed; **the exact cause of those earlier individual incidents remains unconfirmed**. Future 500/503 responses display a support reference matching safe server logs and the X-Request-ID header.

Security and database

Backend tests cover employee IDOR, foreign edit/cancel attempts, forged approval/staff flags, MFA bypass, stale/deactivated sessions, single-use token/code expiry and guesses, password setup, CSRF, escaped XSS content, literal Excel values, invalid form fields and malicious-looking filters. ORM parameterization is retained; no raw user-built SQL or permissive CORS is added.

Both production profiles retain `HttpOnly/SameSite` cookies, CSRF checks, CSP/frame denial, no-store private/error responses, bounded requests/timeouts, exact email domains, trusted proxy IP, disabled debug/development inbox, nonroot/read-only containers and an internal database network. The HTTPS profile additionally enables HTTPS redirects, secure cookies and HSTS. The selected HTTP profile explicitly omits those transport settings. Error logs retain view, exception class, safe source locations/SQLSTATE and request reference without exception values, tokens, credentials or request URLs. Proxy access logs retain status/timing/reference without query values or personal headers.

Migration `0008_booking_approval` atomically maps existing confirmed records to approved, adds review metadata/revision fields, a pending index, rejection validation and an exclusion constraint including pending. No reviewer is invented for legacy meetings. Fresh migrations and `0007 -> 0008` data preservation passed. Round-trip rollback tests passed: approved -> confirmed, pending/rejected -> cancelled, unsent request/rejection mail superseded, sent history retained. Stop writers, back up, and reapply runtime grants during updates. Runtime cannot create

schemas or modify/delete/truncate audit history.

Automated gates

Gate	Latest result / earlier evidence
Full PostgreSQL suite	281 passed in 43.153 seconds on 8 October 2026 ; separate test DB created and removed. Prior wizard/photo baseline: 244 passed in 40.439 seconds; earlier browser-CSRF baseline: 210 passed in 31.580 seconds; earlier HTTPS/HTTP support: 186/202 passed.
Fresh migration chain	All Django migrations and booking 0001-0011 passed; the existing local installation upgraded through 0011 without resetting records or booking statuses.
Forward/reverse migration tests	Legacy data, new guards, rollback mail and re-upgrade passed
Concurrency/transactions	Conflicting pending creates/edits, buffer/closure conflicts, atomic recurrence, competing decisions, access changes, login/code and check-in races passed
Email	Disabled/configured backend, failures/zero delivery, retry/lease, request/rejection/approval/cancellation, revision/reminder checks and local fake SMTP relay passed; no company mail sent
Ruff lint/format	Full repository <code>ruff check</code> . passed; <code>ruff format --check</code> . passed for 77 Python files on 8 October 2026.
Syntax/system checks	Current JavaScript syntax and Django system checks passed. Python compilation passed in the earlier recorded baseline.
Migration drift	Current <code>makemigrations --check --dry-run</code> reported no changes.
Docker build	Current room-approval build and collectstatic passed. Prior wizard/photo build: 135 static files copied, 401 post-processing outputs .
Dependency audit	Earlier 8 October 2026 wizard/photo release: pip-audit 2.10.1 completed full application-requirement resolution, including Pillow 12.3.0, with exit 0 and no known vulnerabilities found at audit time.
Earlier HTTPS production Django check	Exit 0; unsilenced W005/W021 for HSTS subdomains/preload, pending IT domain-wide policy
Static type checker	Not configured; no formal type-check success is claimed
GitHub Actions	CI workflow covers lint, migrations, tests, audit and build; hosted runner result is separate from this local report

Negative tests intentionally log denied access, injected errors and mail failures. These are assertions, not production errors suppressed to pass a gate.

Actual browser checks

Manual browser work used a separate database and local file email at port 8020. Employee email-link login, staff password/email-code login, logout/relogin and persistence passed. Verified combined room search/capacity/location/facility filtering, details, overlap rejection against pending occupancy, employee pending creation, My bookings/history, profile save, staff authorization redirect, approval and required rejection reason, owner rejection feedback, employee edit requiring reapproval, owner cancellation, staff cancellation, room add/edit/deactivation and inactive-room exclusion. Confirm dialogs support Go back, keyboard focus and Escape; confirmed approve/reject/cancel actions were exercised. A real Excel file downloaded; the final browser console had no warning/error entries.

Direct URL plus refresh checks covered:

Employee/common	Staff
Overview; rooms; room details; calendar day/week; new request; My bookings/history; request detail/edit; profile; sign-in/confirmation/logout	Staff sign-in/code; dashboard; pending/all bookings; room list/add/edit; closure list/new; rules; holidays; people; reports; audit; calendar month

Back/forward navigation passed. Desktop 1280, mobile 390 and tablet 768 viewports were inspected; tested page widths stayed within the viewport and table/navigation overflow remained inside scrollable containers. Viewport overrides were reset. Native JavaScript confirmation initially stalled the browser test tool; the interface now uses an accessible in-page dialog, and its actions were retested successfully. Password setup, access grants/revocations, closure/holiday/rule mutations, check-in/deadline jobs and pagination are covered by integration tests; they are not all claimed as manual browser mutations.

Earlier isolated HTTPS production deployment rehearsal

Actual production Compose ran with synthetic secrets, disabled SMTP, isolated DB/network/volume, a trusted test certificate and loopback HTTPS 9444. Verified:

1. Fresh migrations, runtime role setup, healthy proxy/web/worker/database, UID 1000 and read-only application filesystem with writable /tmp.
2. Runtime ordinary DML and audit insert allowed; schema creation/audit UPDATE/DELETE/TRUNCATE denied (SQLSTATE 42501).
3. HTTPS/TLS, HSTS, CSP, no-store/nosniff and secure cookies; 19 authenticated routes each direct/repeated GET 200.
4. Valid-CSRF employee approval denied without changing pending state; real HTTPS employee create -> staff approve -> owner Approved and create -> staff reject -> owner Rejected/reason passed on the final image.
5. Actual backup.sh produced a validated 88,530-byte custom dump. pg_restore into a separate database succeeded; room/user/reservation/audit/notification counts matched 1/2/1/1/2 at the backup point.
6. DB outage readiness 503/liveness 200 and DB recovery without app restart; naturally stale worker after 127 seconds readiness 503 and recovery after restart.
7. Recreated web changed frontend IP while nginx continued running; Docker DNS re-resolution restored all probes. Hashed CSS/JS returned 200 with immutable cache headers.

Rehearsal containers/networks were stopped and removed without deleting volumes. Synthetic evidence/backups stayed outside Git. The existing demo was not modified.

Selected HTTP production deployment rehearsal

After the user selected HTTP for the office network, a separate final-image rehearsal ran at `http://127.0.0.1:9120`, with a fresh isolated database/network, synthetic secrets and a local SMTP capture service. No company relay was contacted. The existing demo remained untouched.

Verified on the final HTTP-support image:

1. Full PostgreSQL suite: **202 tests passed** in 33.125 seconds, including 16 new HTTP-production settings/authentication tests. Django system checks, migration drift, Ruff lint/format, Docker build, and both Compose profiles' quiet configuration checks passed.
2. Fresh migrations through `0008`, restricted runtime role setup, and healthy database/web/worker/proxy startup. Runtime role superuser/database-creation/role-creation flags were all false.
3. Actual HTTP employee email-link confirmation and staff password plus emailed-code POSTs succeeded using captured synthetic SMTP messages. Employee approval was denied; staff approval and rejection persisted with the organizer-visible rejection reason.
4. Session cookies remained `HttpOnly/SameSite Lax` without `Secure`. HTTP produced no HTTPS redirect or HSTS, and the proxy mounted no certificate. Development inbox access returned 404 even on loopback in production mode.
5. Wrong, null and different-port origins, and missing CSRF tokens, returned 403. Valid origin and session-token POSTs passed. Both production profiles reject file-email mode; local development retains its separate email inbox.
6. The HTTP profile requires an explicit bind IP; blank `HTTP_BIND_IP` failed configuration validation. The environment generator selected the HTTP template, generated independent secrets, rejected `--http` without `--production`, and refused to replace an existing `.env`.
7. Actual `deploy/backup.sh`, selected with `COMPOSE_FILE=compose.prod.http.yaml`, produced a verified **89,358-byte** custom-format dump. Restore into a separate database succeeded and matched room/user/reservation/audit counts of **1/2/2/6**.

All rehearsal containers/networks were stopped and removed without deleting volumes. Synthetic SMTP evidence, secrets and backups stayed outside Git. This verifies the local HTTP profile; company-server DNS, network access, SMTP authorization and mailbox delivery still require the operator checks in [the HTTP deployment guide](#).

Native-browser CSRF correction

The new company-server screenshot was reproduced in a separate Chrome tab. Effective running HTTP settings were correct, and a fresh scripted session with an explicit correct Origin passed. The real response still used `Referrer-Policy: no-referrer`. Per the [Fetch standard's Origin-header algorithm](#), this policy makes native form submissions send `Origin: null`, which Django rejects. Earlier scripted production rehearsals explicitly supplied Origin and did not exercise this browser-generated header.

The setting is now `SECURE_REFERRER_POLICY="same-origin"`, retaining same-origin referrers while withholding them from other sites. Actual native Chrome employee submission reached the normal request-received page, and staff submission reached normal credential validation, on a separate production-mode HTTP instance at `mbs-csrf.localhost:9120`. Both used dummy invalid credentials; no company email or successful sign-in is claimed for these probes. The company server still requires pulling and rebuilding the update.

The final image passed **210 PostgreSQL tests in 31.580 seconds**, Django check, migration drift, Ruff lint/format and the Docker build. New regression coverage checks employee/staff response policy plus safe CSRF rejection diagnostics. Failures log only fixed categories and a validated request reference, never raw reasons, Origin/Referer values, cookies, credentials or tokens. The error page now gives site-neutral guidance and returns staff forms to staff sign-in. CSRF enforcement is unchanged.

Browser screenshot evidence is retained outside Git as `outputs/mbs-csrf-fixed.jpg`. Isolated review containers/networks were removed without deleting volumes; the existing demo was preserved. Company booking/staff records were not changed; company probes used only anonymous form sessions.

Deployment handover

The selected company installation is <http://mbs.wdn.com.np>, bound to **192.168.50.222:80**, using `compose.prod.http.yaml`. Follow [the HTTP deployment guide](#); no certificate or private CA is required. The following commands describe the retained HTTPS profile.

Required: `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_APP_USER`, `POSTGRES_APP_PASSWORD`, `DJANGO_SECRET_KEY`, `DJANGO_ALLOWED_HOSTS`, and matching HTTPS `DJANGO_PUBLIC_BASE_URL`. Review proxy subnet/trusted IP settings if the defaults conflict. The HTTPS profile forces HTTPS; the HTTP profile forces HTTP. Both force production mode and allow `wdn.com.np`, `transgate.com.np`.

IT supplies SMTP separately: `DJANGO_EMAIL_BACKEND=smtp`, `SMTP_HOST`, `SMTP_PORT`, `DJANGO_FROM_EMAIL`, `SMTP_USER`/`SMTP_PASSWORD` when required, and mutually exclusive `SMTP_USE_TLS`/`SMTP_USE_SSL`. Keep mail disabled while unconfigured; startup/bookings remain usable and jobs are not falsely marked delivered. Mailbox sign-in requires working delivery. No production credentials are committed.

On the company Linux server, after internal DNS/firewall/TLS and `.env` are prepared:

SH / Display wraps; copy complete commands from the Markdown source.

```
git clone --branch mbs-prod https://github.com/BINAYAK016/MeetingRoomSystemWDN.git
cd MeetingRoomSystemWDN
docker run --rm -v "$PWD:/workspace" -w /workspace python:3.12-slim python deploy/create-env.py
--production
# Edit .env; install approved fullchain.pem and privkey.pem under deploy/certs.
chmod 600 .env deploy/certs/privkey.pem
docker compose -f compose.prod.yaml config --quiet
docker compose -f compose.prod.yaml build --pull
docker compose -f compose.prod.yaml up -d db
docker compose -f compose.prod.yaml run --rm migrate
docker compose -f compose.prod.yaml run --rm db_setup
docker compose -f compose.prod.yaml up -d web worker proxy
curl --fail https://YOUR-INTERNAL-HOST/healthz/
# Once approved SMTP is configured:
docker compose -f compose.prod.yaml exec web python manage.py create_staff_account
APPROVED-PERSON@wdn.com.np
sh deploy/backup.sh
```

See [deployment](#) for exact updates, mail enablement, setup, monitoring, backup scheduling and restore commands; [README](#) for laptop setup; [employee](#) and [staff](#) for operation.

Remaining verification

Company-server DNS, firewall/VPN, actual SMTP authorization and both domains' mailbox delivery, backup storage/retention, an on-server restore drill and representative office load await operator verification. The server OS, internal domain and office-interface IP have been supplied; the isolated local rehearsal does not prove those company-server checks. The specific earlier intermittent incident remains unattributed. No company deployment or sustained capacity benchmark is claimed. Direct HCL/Outlook integration remains deferred by agreement. Single-server failure, periodic worker release and possible duplicate SMTP delivery after an ambiguous response remain documented operating limits. For the optional HTTPS profile, HSTS domain-wide flags require IT approval.

Documentation-only release verification - 7 October 2026

This release changes documentation and adds an optional ReportLab handbook builder, with no application behavior or database migration changes. Existing application test evidence above remains the baseline; the full application suite was not rerun for prose changes.

- Reviewed feature/employee/staff documentation against models, forms, views, services, templates and worker: permission/state/notification matrices, complete URL catalogue and 28 use cases.
- Validated all 13 Markdown documents: 106 local links/anchors, balanced fences, 90 Bash/sh code blocks, nine embedded Python snippets and 12 PowerShell blocks. Syntax-only checks did not execute operational examples.
- The feature catalogue covers all 43 configured URL patterns, including protected actions and generated private-link routes.
- Ruff lint and formatting pass across all 70 Python source files, including the optional builder; Git whitespace checks pass.
- Ran the six read-only troubleshooting Django diagnostics against a separate PostgreSQL 17 project with SMTP disabled. Confirmed configured policy, queue metadata, restricted runtime identity and audit SELECT/INSERT-only permissions.
- Exercised the documented separate-database restore pattern on that isolated project: a trusted 87,294-byte custom-format dump restored one user, room, reservation, notification and audit event each; all five counts matched. Verified restored runtime audit INSERT/UPDATE/DELETE privileges (True, False, False) and the validated overlap constraint. The live-target name guard rejected creation against the selected source database.
- No worker was started for the restore drill and no external email, company account or company server mutation was performed. Isolated containers/networks were removed with the QA volume retained; the existing local demo was left healthy.
- Generated the complete handbook from 11 maintained guides, rendered its pages and checked contents/bookmarks, tables, diagrams, command displays, margins and glyphs. Long command displays wrap for print; Markdown remains the copyable source.

The optional documentation dependency is separate from application requirements. Rebuild the handbook after source-guide changes and review the latest rendered pages before committing it. Company-side acceptance, monitoring and recovery arrangements remain the operator's responsibility.

Booking wizard and room photos - 8 October 2026

The following records the prior wizard/photo baseline, before the subsequent per-room approval release.

This release changes application behavior as well as documentation: Schedule & details -> Attendees -> Review & confirm, real whole-series room availability, Internal + External meetings, and optional staff-managed room photos. Committed migrations `0009_mixed_meeting_type` and `0010_room_photo` preserve existing records. No room reseeding is part of the upgrade. A local database backup was taken before upgrading the existing laptop installation.

Automated verification

- Final clean PostgreSQL run: **244 tests passed in 40.439 seconds**; Django system check reported no issues. Early runs exposed historical migration-test cleanup that restored only schema 0008, leaving later photo-model tests on an incomplete schema; cleanup now restores the current migration graph leaf. The final result is the clean rerun, not those earlier failed attempts.
- Availability coverage checks authenticated GET-only access, active room metadata, generic occupied ranges without private meeting content, all occupying/released statuses, both meeting buffers, every recurrence occurrence, working-day skips, edit ownership/state and single-occurrence exclusion, staff verification/overrides, read-only behavior, page configuration/prefills, short-office-hour defaults and final-save revalidation after preview.
- Mixed meeting coverage requires guest company in form, service and database. Switching recurrence back to One meeting ignores a stale until date instead of rejecting an otherwise valid single meeting.
- Photo coverage checks optional upload, authenticated viewing, staff second factor, inactive-room privacy, supported/corrupt/animated/oversized images, metadata removal and image resizing, replace/remove commit

cleanup, new-file cleanup on database failure, preservation on storage failure, conflicting remove/upload, and arbitrary-path/symlink escape rejection.

- Full repository `ruff check` . passed and `ruff format --check` . passed for 75 Python files. `node --check` passed for wizard and shared application JavaScript. The current `pip-audit 2.10.1` run resolved the full application requirements, including Pillow 12.3.0, and reported no known vulnerabilities with exit 0.

Actual browser and persistence checks

A separate local QA instance used file-email login; no company SMTP relay or production account was used. Browser actions verified:

1. Schedule initialization and required-title/guest-company validation; selecting Internal + External and refreshments Yes.
2. Invalid attendee email blocked progression. Organizer addresses and duplicates were counted once; nine people in an eight-seat room were rejected, while three people were accepted.
3. One final submission created a Pending request with the entered meeting information. Edit preserved mixed type, guest company and attendees, displayed the no-photo placeholder, and excluded its own occupied slot from availability.
4. Daily Friday 9 October through Tuesday 13 October previewed exactly Friday, Monday and Tuesday, skipping the weekend. Switching back to One meeting removed the additional dates from review.
5. All three steps were inspected at 390×844 after the mobile-grid correction; page width was 375 pixels, within the 390-pixel viewport. Desktop was inspected at 1500×1100 against the supplied layout reference.
6. A write probe in the isolated named `room_media` volume survived web-container recreation, then was removed. This establishes volume persistence for that local QA project; it is not a production photo backup/restore rehearsal.

Existing laptop upgrade

The current local installation was backed up, rebuilt and migrated through 0009/0010. Pre/post counts matched: **five rooms, two bookings and three users**. Database, web and worker were healthy; `/healthz/` returned HTTP 200, and media storage was writable. The existing local file-email backend remained unchanged. Employee file-email login succeeded against the updated local instance; the desktop booking screen was captured at 1500 pixels, with no browser-console errors. Migration drift reported no changes; Git whitespace checks passed. QA was signed out/closed and the temporary viewport override was reset. These checks concern the laptop installation, not the company server.

The guides now cover all 45 configured routes, native-form fallback, real preview limitations, staff photo paths, media ownership/persistence, paired database/photo backups and migrations 0009/0010. Syntax checks of documented archive/extraction commands do not claim an executed photo restore. Actual staff photo upload is covered by integration tests; the browser review used existing rooms with no photo and does not claim a manual uploaded-photo walkthrough.

Company-server deployment of this release, actual mailbox delivery, employee-device behavior, representative load and a paired database/photo restore drill still require IT's acceptance checks. The local QA data and evidence do not establish those outcomes. Rebuild/recreate and migrate the company installation through the documented maintenance procedure; preserve existing database and media volumes.

Per-room approval - 8 October 2026

This release adds Requires approval to staff room rows/add/edit and exposes Staff approval required or Automatic approval in employee room/booking screens. Migration `0011_room_requires_approval` follows 0010, defaults all existing rooms to checked and leaves saved booking status/history intact. Employee creations and substantive edits use the selected locked room policy; staff creation remains Approved. No-op saves and policy toggles do not convert an existing Pending queue.

Automated verification

- Final PostgreSQL run: **281 tests passed in 43.153 seconds**, including 23 added booking-policy tests and 14 added staff-management tests. Django system checks reported no issues. The first run exposed an unchecked default on a new room form; it was corrected and the full suite rerun successfully.
- The complete migration chain through **0011** passed, including existing-data preservation. Tests exercise checked/unchecked creation and recurrence, selected-room edits, no-op retention, current policy under locks, normal validation/conflicts, confirmation/change mail, check-in link invalidation, approval metadata and audit. Staff tests cover verified access, CSRF, strict input, add/edit defaults, toggle persistence, retained booking states and audit.
- Full repository `ruff check .` and `ruff format --check .` passed (**77 Python files**). Node syntax checks for both application/wizard scripts passed. The Docker image build and collectstatic passed. No new company-server or live-SMTP result is implied.

Actual browser checks

An isolated QA instance used local file email. Staff completed password-plus-email-code sign-in, unchecked Room 4 directly in Staff desk -> Rooms, and the automatic POST persisted with the saved message. An employee completed email-link sign-in, selected that room and pressed **Confirm booking**: the resulting booking was **Approved**, identified as automatically approved. The same employee substantively edited it into checked Room 2: it became **Pending approval** with the earlier approval actor cleared. The staff checkbox screen was captured for the handover. These were real browser workflows; additional service/security edge cases are covered by tests rather than claimed as manual walkthroughs.

Existing laptop upgrade and documentation

The laptop database was backed up in validated custom format, migrated through 0011 and web/worker recreated. Health returned HTTP 200; both services were healthy. Pre/post inventory matched: **five rooms, two bookings and three users**. Booking 1 remained Cancelled and booking 2 remained Pending; all five existing rooms defaulted to Requires approval checked. Local file email remained unchanged. No production server was accessed and no company email was sent.

Documentation now explains both room policies, checkbox auto-save/Save fallback, review queue retention, substantive/no-op edits, confirmation/check-in, recurrence, audit and migration 0011. The feature directory covers all **46 configured routes**. The HTTP update example builds before the outage, stops writers, takes paired database/media snapshots with database running, applies migrations/runtime grants, then recreates services. Documented command syntax/link checks validate examples; they do not claim the production update or photo restore has run.